



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE GRADO

Integración de la inteligencia artificial en el E-Commerce para la mejora de la experiencia de usuario y optimización de ventas.

Realizado por
Antonio Silva Gordillo

Para la obtención del título de
Grado en Ingeniería informática - Tecnologías informáticas

Dirigido por
Dr. Juan Antonio Ortega Ramírez

En el departamento de
Lenguajes y sistemas informáticos

Convocatoria de junio, curso 2024-2025

"Recuerda: la tecnología no es nada. Lo importante es tener fe en la gente, que son básicamente buenos e inteligentes, y si les das herramientas, harán cosas maravillosas."

— Steve Jobs

Agradecimientos

Quiero expresar mi más sincero agradecimiento a mi tutor, el Dr. Juan Antonio Ortega Ramírez, por su orientación y apoyo durante todo el proceso de realización de este proyecto. Su guía no solo me ha permitido profundizar en un tema tan fascinante como la inteligencia artificial aplicada a problemas reales, sino que también ha sido fundamental para encontrar el enfoque adecuado para este trabajo.

También quiero agradecer a mi familia, por su incondicional apoyo y comprensión durante los momentos más intensos del estudio de este grado que me han permitido llegar hasta aquí. Su paciencia y motivación constante han sido una fuente de energía que me ha permitido seguir adelante y superar los desafíos que se presentaron en el camino.

A mi pareja, le agradezco su constante ánimo y por ser mi pilar en los momentos más difíciles. Su apoyo emocional ha sido clave para mantener el equilibrio entre el trabajo y la vida personal durante este proceso.

Finalmente, quiero agradecer a todos mis amigos y compañeros de carrera, quienes me han acompañado a lo largo de este viaje académico. Sus palabras de ánimo, colaboración y las conversaciones compartidas han sido invaluable. Este trabajo no habría sido posible sin la red de apoyo que me ha rodeado. Gracias a todos.

Resumen

La inteligencia artificial y, muy en concreto, los sistemas de recomendación están en todas partes: desde Netflix sugiriéndote series hasta Amazon empujándote ese gadget que “casualmente” estabas mirando. El problema es que tras esas recomendaciones suele esconderse una “caja negra”: un algoritmo al que nadie entiende del todo, ni siquiera sus creadores. Eso genera desconfianza: ¿por qué me recomienda esto y no lo otro? ¿Se basa en mis gustos reales o en simples patrones de comportamiento?

Este proyecto se desarrolló con el objetivo de diseñar e implementar una plataforma web con productos reales donde mostrar y testear los diferentes algoritmos de inteligencia artificial utilizados actualmente para crear sistemas avanzados de recomendación.

Durante la implementación del sistema web se afrontaron algunos retos importantes, como trabajar con grandes volúmenes de datos y entrenar algoritmos de inteligencia artificial en un ordenador convencional. Para superarlos, se elaboró un proceso ETL optimizado y eficiente, además del apoyo en funciones externas que permitían el consumo de datasets masivos mediante streaming y virtualización de datos. También se usaron modelos preentrenados para reducir el tiempo y la carga del sistema, logrando un modelo de recomendación funcional.

Para su desarrollo se ha utilizado como base *Django*, así como *Python* nativo acompañado de librerías específicas como *Pandas*, *NumPy* y *Hugging-faces* para el análisis y manipulación del catálogo de productos de *Amazon*, o *Scikit-learn* y *Surprise* para el desarrollo de los algoritmos de recomendación, basados principalmente en *KNN*, *SVD* y *Content-Based Filtering*. Se usó *Postgre* como base de datos.

Finalmente, se obtuvo una plataforma completa y robusta que permite explorar y entender cómo se comportan estos algoritmos en un entorno web con productos reales, descubriendo el potencial de estas técnicas para mejorar las ventas en eCommerce, personalizando y mejorando la experiencia del usuario.

Palabras clave: Inteligencia artificial, eCommerce, sistema de recomendación, Django, Python, aprendizaje automático

Abstract

Artificial intelligence—and more specifically, recommendation systems—is everywhere: from Netflix suggesting series to Amazon pushing that gadget you were “casually” browsing. The problem is that behind those recommendations there’s often a “black box”: an algorithm that nobody fully understands, not even its creators. That breeds distrust: why does it recommend this and not that? Does it actually reflect my real tastes or merely simple behavior patterns?

This project was developed with the goal of designing and implementing a web platform with real products on which to showcase and test the various artificial-intelligence algorithms currently used to build advanced recommendation systems.

During the implementation of the web system, several major challenges were faced, such as working with large volumes of data and training AI algorithms on a conventional computer. To overcome them, an optimized, efficient ETL process was devised, along with support from external functions that enabled consumption of massive datasets via streaming and data virtualization. Pre-trained models were also used to reduce both time and system load, resulting in a fully functional recommendation model.

For its development, Django was used as the foundation, together with native Python and specific libraries such as Pandas, NumPy and Hugging Face for analysis and manipulation of Amazon’s product catalog, and Scikit-learn and Surprise for developing the recommendation algorithms—primarily based on KNN, SVD and content-based filtering. PostgreSQL served as the database.

Finally, a complete, robust platform was delivered that allows one to explore and understand how these algorithms behave in a web environment with real products, uncovering the potential of these techniques to boost e-commerce sales by personalizing and improving the user experience.

Keywords: artificial intelligence, e-commerce, recommendation system, Django, Python, machine learning.

Índice general

Índice general	IV
Índice de figuras	VII
Índice de tablas	IX
1 Introducción	1
1.1. Contexto	1
1.2. Motivación	2
1.3. Objetivos del proyecto	2
1.4. Estructura de la memoria	4
2 Estudio previo	6
2.1. Introducción	6
2.2. Objetivos	6
2.3. Metodología	7
3 Estado del arte	11
3.1. Inteligencia artificial aplicada al eCommerce	11
3.2. Sistemas de recomendación en eCommerce	13
3.3. Casos de uso en grandes plataformas.	23
3.3.1. Amazon	23
3.3.2. Netflix	24
3.3.3. Spotify	24
3.3.4. Alibaba	25
4 Elicitación de requisitos	26
4.1. Participantes en el proyecto	27
4.2. Estado Actual	29
4.3. Objetivos del sistema	30
4.4. Catálogo de requisitos	35
4.4.1. Introducción a las historias de usuario	35
4.4.2. Roles de usuario	35
4.4.3. Historias épicas	37
4.4.4. Historias de usuario	39
4.5. Análisis de requisitos	44
4.5.1. Introducción	44
4.5.2. Catálogo de requisitos	45
4.5.3. Requisitos funcionales	45
4.5.4. Reglas de negocio	46
4.5.5. Requisitos no funcionales	48

4.6.	Matrices de trazabilidad	49
4.6.1.	Matriz de trazabilidad entre los objetivos y las historias de épicas de usuario	49
4.6.2.	Matriz de trazabilidad entre los objetivos y las historias de usuario	50
4.6.3.	Matriz de trazabilidad entre las historias de usuario épicas y las historias de usuario	50
4.6.4.	Matriz de trazabilidad entre objetivos y requisitos funcionales	51
4.6.5.	Matriz de trazabilidad entre objetivos y requisitos no funcionales	51
4.6.6.	Matriz de trazabilidad entre requisitos funcionales y requisitos de información	52
4.6.7.	Matriz de trazabilidad entre reglas de negocio y requisitos funcionales	52
5	Planificación	53
5.1.	Planificación temporal	53
5.2.	Planificación de costes	57
5.2.1.	Costes de personal	57
5.2.2.	Costes materiales	58
5.2.3.	Costes de software	58
5.2.4.	Costes de servicios	59
5.2.5.	Margen para imprevistos	59
5.2.6.	Costes totales	60
5.3.	Plan de gestión de riesgos	61
5.4.	Plan de gestión de la calidad	62
5.5.	Plan de gestión de las comunicaciones	62
5.6.	Justificación metodológica	63
5.7.	Sostenibilidad y optimización de recursos	63
6	Diseño de la solución	64
6.1.	Introducción	64
6.2.	Análisis y justificación del dataset: <i>Amazon Reviews 2023</i>	64
6.3.	Diseño de la arquitectura y sus componentes	66
6.3.1.	Arquitectura	67
6.3.2.	Componentes del sistema	69
6.3.3.	Modelo de datos y relaciones	71
6.4.	Funcionalidad del sistema	73
6.5.	Tecnologías utilizadas	76
6.6.	Consideraciones sobre calidad, mantenimiento y evolución futura . .	85
6.7.	Flujo de procesos	85
7	Ejecución	87
7.1.	Sprint 1: Proceso ETL	87
7.2.	Sprint 2 – Configuración del entorno y modelado de datos	91
7.3.	Sprint 3: Codificación del back-end e integración del módulo de recomendación	95

8 Cierre	106
8.1. Introducción	106
8.2. Conclusiones	106
.1. Anexo A: Elección del dataset	107
Bibliografía	109

Índice de figuras

2.1.	Descripción de la imagen	8
2.2.	Cuadro resumen de los beneficios clave de los algoritmos de recomendación personalizada	10
3.1.	Comparación entre filtrado basado en contenido y filtrado colaborativo	15
3.2.	Representación visual de la factorización de matrices en sistemas de recomendación	19
4.1.	Limitaciones comunes de los sistemas de recomendación tradicionales	29
4.2.	Formato de redacción de historias de usuario propuesto por Mike Cohn (2004).	35
5.1.	Diagrama de Gantt de la planificación inicial del proyecto	54
5.2.	Reparto porcentual de los costes totales del proyecto	60
6.1.	Esquema general de la arquitectura del sistema de recomendación. .	68
6.2.	Modelo Entidad-Relación (MER) del sistema recomendador. Se muestran las entidades principales (<i>User, Account, Product, Category, Review</i>) y las relaciones entre ellas, permitiendo una gestión eficiente y flexible de usuarios, productos, valoraciones y métricas.	71
6.3.	Arquitectura del flujo de datos en Django bajo el patrón MTV (Model-Template-View). Se ilustra cómo una petición del navegador atraviesa el mapeo de URL hacia una vista, la cual puede acceder al modelo y luego devolver una respuesta renderizada mediante una plantilla. Este flujo refleja la organización modular y coherente del framework.	78
6.4.	Flujo completo de procesos del sistema recomendador. El diagrama ilustra, de izquierda a derecha, todas las etapas del sistema: desde la ingestión y limpieza de datos externos, el almacenamiento y entrenamiento de modelos, hasta la entrega de recomendaciones al usuario final a través del backend y la interfaz web. Cada módulo está desacoplado para facilitar el mantenimiento, la trazabilidad y la escalabilidad futura.	86
7.1.	Resumen de características del dataset Amazon Reviews 2023, obtenido de su página oficial.	89
7.2.	Estructura general del proyecto	92
7.3.	Parámetros de conexión a la base de datos PostgreSQL en settings.py.	92
7.4.	Configuración de aplicaciones instaladas y middlewares en settings.py.	93
7.5.	Configuración regional, idioma y archivos estáticos en settings.py.	93
7.6.	Pantalla de bienvenida de Django tras el primer arranque del servidor.	95
7.7.	Flujo interno de procesamiento de peticiones en Django	101
7.8.	Captura de la ejecución del comando en el entorno de desarrollo . . .	102

7.9. Captura de pgAdmin4 donde se aprecian los productos cargados en la base de datos	103
---	-----

Índice de tablas

1.1. Objetivos específicos del proyecto y sus indicadores de validación . .	3
3.1. Comparativa de algoritmos de recomendación en función de características clave	23
4.1. Información del participante Antonio Silva Gordillo	27
4.2. Información del participante Juan Antonio Ortega Ramírez	27
4.3. Información de la organización: Departamento de Lenguajes y Sistemas Informáticos	28
4.4. Información de la organización: Universidad de Sevilla	28
4.5. Descripción del objetivo OBJ-001	30
4.6. Descripción del objetivo OBJ-002	31
4.7. Descripción del objetivo OBJ-003	32
4.8. Descripción del objetivo OBJ-004	33
4.9. Descripción del objetivo OBJ-005	34
4.10. Descripción del objetivo OBJ-006	34
4.11. Descripción del rol ROL-001 - Usuario registrado	36
4.12. Descripción del rol ROL-002 - Administrador	36
4.13. Historia de usuario épica 001 - Navegar y filtrar productos con recomendaciones personalizadas	37
4.14. Historia de usuario épica 002 - Gestionar catálogo de productos . . .	37
4.15. Historia de usuario épica 003 - Administrar algoritmos de recomendación	38
4.16. Historia de usuario épica 004 - Realizar el proceso ETL	38
4.17. Historia de usuario HUE-005 - Documentar y validar todas las fases del sistema	39
4.18. Historia de usuario HU-001 - Filtrar productos por categoría, precio y calificación	40
4.19. Historia de usuario HU-002 - Gestionar catálogo de productos	41
4.20. Historia de usuario HU-003 - Configurar algoritmos de recomendación	42
4.21. Historia de usuario HU-004 - Ejecutar proceso ETL para cargar datos de productos	43
4.22. Historia de usuario HU-005 - Crear documentación y guías del sistema	44
4.23. Requisitos de Información del Sistema	45
4.24. Requisitos funcionales del sistema	46
4.25. Reglas de negocio del sistema	47
4.26. Requisitos no funcionales del sistema	48
4.27. Matriz de trazabilidad entre los objetivos del sistema y las historias de usuario épicas	49
4.28. Matriz de trazabilidad entre los objetivos del sistema y las historias de usuario	50
4.29. Matriz de trazabilidad entre historias de usuario épicas y las historias de usuario	50

4.30. Matriz de trazabilidad entre objetivos del sistema y requisitos funcionales	51
4.31. Matriz de trazabilidad entre objetivos del sistema y requisitos no funcionales	51
4.32. Matriz de trazabilidad entre requisitos funcionales y requisitos de información	52
4.33. Matriz de trazabilidad entre reglas de negocio y requisitos funcionales	52
5.1. Hitos del proyecto y sus descripciones	53
5.2. Planificación temporal del proyecto.	54
5.3. Resumen cronológico de los hitos del proyecto con fechas y dependencias	55
5.4. Matriz de asignación de perfiles a los hitos del proyecto, con descripción	56
5.5. Sueldos medios anuales por rol en España (2025)	57
5.6. Coste por hora estimado incluyendo Seguridad Social	57
5.7. Costes de desarrollo del proyecto desglosado por perfil profesional y tipo de tarea	58
5.8. Coste total estimado del proyecto con 300 horas y entorno local . . .	60
5.9. Resumen de riesgos y planes de contingencia	61
6.1. Campos principales del dataset <i>Amazon Reviews 2023</i> , descargado desde HuggingFace el 3 de febrero de 2025.	65
7.1. Resumen de las fases del proceso ETL implementado	90

Índice de extractos de código

7.1. Comando para crear una nueva aplicación en Django	92
7.2. Comando para crear el usuario administrador en Django	94
7.3. Vista de la consola del backend al inciar el servidor	94
7.4. Modelo Category	97
7.5. Modelo Product	97
7.6. Modelo Account	98
7.7. Modelo Review	98
7.8. Rutas principales de la plataforma	99
7.9. Ejemplo de respuesta del endpoint de recomendaciones	104

1. Introducción

En este capítulo se presenta la temática en la que se basa el proyecto, así como el contexto, la motivación y los principales objetivos marcados en su desarrollo. Al final se presenta la estructura completa del mismo.

1.1. Contexto

En la actualidad, muchos negocios digitales donde destacan los *ecommerces*, utilizan sistemas de recomendación muy básicos. Estos sistemas, eligen las recomendaciones basados en reglas fijas, que no consideran como se comporta cada cliente de forma individual y que no aprovechan el gran potencial de los datos que obtiene al interactuar los potenciales clientes. Esto da como resultado, recomendaciones genéricas poco útiles, empeorando la experiencia de compra y afectando negativamente indicadores como el número de ventas, el valor del ticket medio y el ratio de fidelización de clientes. Para los pequeños y medianos negocios, implementar soluciones más sofisticadas suele ser costoso o técnicamente complicado, ya que a parte de la implementación requiere un mantenimiento y reajuste constante para seguir ofreciendo buenos resultados.

En este panorama, la inteligencia artificial emerge como una poderosa aliada capaz de abordar estos retos. En los últimos años, la IA ha revolucionado muchos sectores, y el comercio electrónico no se queda atrás. Las empresas del sector buscan constantemente formas innovadoras de mejorar sus ventas y hacer más atractiva la experiencia de los clientes.

Por eso he decidido desarrollar un sistema de recomendaciones como parte central de mi proyecto. Estos sistemas han demostrado su efectividad para impulsar los resultados comerciales cuando se combinan con estrategia de marketing efectivas, como ofrecer productos de mayor valor (*upselling*) o complementarios *cross-selling*. Con recomendaciones personalizadas, aumentan las posibilidades de venta mientras se mejora y personaliza la experiencia de cada cliente.

El desarrollo del sistema se ha realizado utilizando tecnologías modernas como *Django* y *Python* para la construcción del backend, y librerías especializadas en IA como *Scikit-Learn* o *Surprise* para el entrenamiento y despliegue de los modelos de recomendación. El frontend se implementará con *Bootstrap*, buscando ofrecer una interfaz intuitiva y eficiente que facilite la interacción del usuario con la plataforma.

1.2. Motivación

Mi motivación inicial para desarrollar este proyecto viene de mi interés por entender y aplicar de forma práctica la inteligencia artificial a un problema real con impacto tangible. Aunque la IA no es una tecnología reciente, su evolución en los últimos años ha sido tan significativa que ha pasado de ser un campo teórico a convertirse en una poderosa herramienta para la innovación en muchos campos.

No quería limitarme a estudiar algoritmos o conceptos puramente teóricos, sino enfrentarme al reto de diseñar, construir y desplegar una solución completa, utilizando tecnologías que, en muchos casos, no dominaba al iniciar el proyecto.

Impulsado por la propuesta de mi tutor Juan Antonio Ortega Ramírez de basarlo en *Python* y los algoritmos de aprendizaje automático, me decanté por los sistemas de recomendación ya que es una aplicación muy potente de esta tecnología con gran impacto y que es aplicable a muchos grandes sectores como el comercio electrónico, plataformas de streaming, entretenimiento, etc.

El resultado es este proyecto, que me ha permitido afianzar y poner en práctica los conocimientos y habilidades técnicas adquiridos durante el grado, generando una herramienta con impacto positivo en la sociedad, que mejora tanto las ventas como la retención y la experiencia de los usuarios de la plataforma donde se implemente.

1.3. Objetivos del proyecto

Este trabajo, a través de la creación de modelos de recomendación eficientes y accesibles, pretende demostrar que es posible construir una herramienta capaz de ofrecer recomendaciones realmente acertadas y personalizadas, incluso en entornos con recursos limitados. Representa una gran ayuda para pequeñas y medianas empresas que desean mejorar sus procesos de venta sin grandes inversiones en infraestructura o depender de costosos softwares de terceros.

El proyecto incluye tanto el desarrollo del sistema de recomendación como su integración en un prototipo funcional de plataforma de *ecommerce*. Esta plataforma, diseñada específicamente para este trabajo, servirá como entorno de prueba donde se evaluará el rendimiento del sistema, su capacidad de adaptación al comportamiento del usuario y su impacto en métricas clave.

Para entrenar el sistema y montar la plataforma se ha usado un *dataset* de más de 150 000 productos reales extraídos de Amazon, al cual se le ha aplicado un proceso *ETL* para poder cargarlos en el sistema de forma correcta.

Dichos datos se han extraído del *dataset* “Amazon-Reviews-2023” de McAuley Lab, alojado en HuggingFace. Incluye revisiones y metadatos de productos, especialmente en la categoría electrónica, abarcando un rango temporal desde mayo de 1996 hasta septiembre de 2023. Además, el procesamiento e importación de la

información de los productos hacia la base de datos, así como el entrenamiento de los modelos de recomendación, se realizó el 20 de marzo de 2025.

Objetivo del proyecto	Indicador de éxito
Diseñar e implementar un sistema de recomendación personalizado utilizando técnicas de IA	Funcionamiento correcto de modelos basados en K -NN, SVD y filtrado basado en contenido
Integrar el sistema de recomendación en un entorno funcional de ecommerce	Plataforma web operativa con recomendaciones visibles y dinámicas en la interfaz
Validar el rendimiento del sistema en condiciones realistas de datos y usuario	Métricas de evaluación positivas (por ejemplo, precisión $\geq 50\%$ en pruebas offline)
Automatizar el proceso de carga y transformación de datos a partir de un dataset real	Proceso ETL exitoso con más de 100 000 productos importados correctamente
Demostrar la viabilidad del proyecto en entornos con recursos computacionales limitados	Prueba de concepto ejecutada en máquina local (≤ 12 GB RAM, 4 hilos CPU)
Documentar y validar todas las fases del sistema	Manual de instalación y manual de usuario, guía de carga de datos y reportes de pruebas unitarias
Aplicar conocimientos adquiridos en el grado en un desarrollo completo y transversal	Cobertura de todas las fases: diseño, desarrollo, pruebas, validación y documentación

Tabla 1.1: Objetivos específicos del proyecto y sus indicadores de validación

1.4. Estructura de la memoria

Este documento recoge todo el desarrollo e implementación del proyecto desde el inicio hasta su finalización. El trabajo está compuesto por nueve partes generales que se detallan a continuación:

Parte 1: Introducción

En este primer bloque, se introduce el tema del proyecto y los objetivos principales.

Parte 2: Estudio previo

Se presenta un análisis general de los antecedentes y el marco teórico. Se detallan los fundamentos y conceptos clave en el proyecto, así como los objetivos específicos y la metodología elegida para el correcto desarrollo de este.

Parte 3: Estado del arte

Esta sección expone los enfoques y técnicas más representativas en el diseño de sistemas de recomendación, utilizados con el objetivo de predecir las preferencias de los usuarios a partir de datos.

Se detallan desde técnicas clásicas como la factorización de matrices hasta modelos probabilísticos y redes neuronales. También se analizan los principales desafíos y problemas asociados a su implementación, así como estrategias probadas para mitigarlos.

Finalmente, se presentan casos de uso en plataformas digitales reconocidas, que muestran la aplicación real y el impacto de estas tecnologías en el comercio electrónico y los servicios de contenido digital *mainstream*.

Parte 4: Elicitación de requisitos

En esta sección se documenta de forma estructurada el resultado del proceso de elicitar y organizar los requisitos del sistema.

Se definen inicialmente los objetivos principales del proyecto, seguidos de la elaboración de historias épicas y sus correspondientes historias de usuario, que describen de forma detallada las necesidades funcionales desde el punto de vista de los diferentes roles identificados.

Además, se presenta el catálogo completo de requisitos, clasificado en requisitos de información, requisitos funcionales, requisitos no funcionales y reglas de negocio, proporcionando una visión integral de las expectativas y restricciones del sistema.

Finalmente, se elaboran matrices de trazabilidad que relacionan los objetivos, requisitos y funcionalidades, garantizando la alineación de todos los elementos y facilitando el control y validación del sistema durante su desarrollo.

Parte 5: Planificación

Esta sección explica cómo se han organizado las tareas, recursos y tiempos para alcanzar los objetivos definidos y cumplir el cronograma, así como la estimación de

los costes del desarrollo de este proyecto.

Parte 6: Diseño

Esta sección aborda el diseño y la arquitectura técnica del sistema, detallando su estructura mediante diagramas y modelos de datos, así como la integración de las tecnologías utilizadas, con el objetivo de asegurar una solución escalable, eficiente y mantenible.

Parte 7: Implementación

Este apartado expone detalladamente el proceso de desarrollo e implementación del sistema, desde el preprocesamiento del conjunto de datos hasta la construcción del *pipeline* de datos y la aplicación web.

Se explican las distintas etapas del desarrollo, las decisiones técnicas tomadas y los principales retos enfrentados, incluyendo fragmentos de código y diagramas que muestran cómo se llevó a cabo el desarrollo de la plataforma.

Parte 8: Cierre

Aquí se exponen las conclusiones extraídas de este proyecto, con especial foco en los logros alcanzados en relación con los objetivos iniciales, así como en los problemas y desafíos que surgieron durante su desarrollo.

2. Estudio previo

2.1. Introducción

En este capítulo se presenta un análisis general de los antecedentes y la fundamentación teórica que sustentan el desarrollo del presente proyecto. Se examina cómo la inteligencia artificial (IA) ha sido aplicada en el ámbito del comercio electrónico (eCommerce), considerando los avances más recientes y las mejores prácticas en la implementación de sistemas de recomendación y estrategias de marketing automatizadas. Además, se definen los objetivos específicos del proyecto, se describe la metodología empleada y se sientan las bases conceptuales necesarias para comprender la relevancia de la solución propuesta.

2.2. Objetivos

El objetivo principal de este proyecto es diseñar e implementar un sistema de inteligencia artificial en un prototipo real de *eCommerce* con productos reales, con el propósito de poder testear y entender cómo se comportan, en base a ciertos valores, algoritmos recomendadores que pueden ser utilizados en un *eCommerce* real para mejorar las ventas y la experiencia de usuario, aplicándolos junto a técnicas de marketing como el *upselling* y el *cross-selling*.

Asimismo, este proyecto tiene como finalidad servirme a nivel personal para poner en práctica todos los conocimientos adquiridos durante el estudio del grado y enfrentarme a nuevos retos, tanto técnicos como de planificación y gestión.

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

- Investigar y analizar las tecnologías y algoritmos de inteligencia artificial más efectivos para su aplicación en el ámbito del eCommerce.
- Diseñar y desarrollar un sistema de recomendaciones personalizadas adaptado a las preferencias y comportamientos de los usuarios.
- Entender las bases de estrategias de marketing que respondan en tiempo real a las tendencias y dinámicas del mercado, para inspirarme en su funcionamiento y diseñar un sistema coherente con la realidad actual del comercio electrónico.
- Implementar y evaluar el sistema en un entorno de pruebas local pero usando tecnologías, arquitectura y productos propios de un sistema real .
- Validar la efectividad del sistema mediante pruebas y métricas.
- Afrontar el reto personal de desarrollar un proyecto completo que combine desarrollo web, algoritmia, matemáticas avanzadas e inteligencia artificial.

- Superar el desafío de lograrlo utilizando tecnologías y librerías modernas que no he estudiado previamente.
- Aplicar de forma efectiva metodologías de ingeniería de software, con el fin de consolidar las competencias adquiridas durante el grado y prepararme para el entorno profesional actual, siendo capaz de aportar valor a las empresas y soluciones en las que trabaje.

2.3. Metodología

Para este proyecto se ha elegido una metodología de desarrollo híbrida, que combina lo mejor de la clásica metodología en cascada, con enfoques más modernos como Scrum y Kanban propio de las metodologías ágiles. La planificación general y la ejecución del proyecto se han organizado de forma secuencial —siguiendo fases bien definidas como análisis, diseño, implementación, pruebas y documentación—, también se ha recurrido al uso de sprints, tableros Kanban y principios de Scrum para gestionar las tareas, los entregables parciales y el control del tiempo y el rendimiento.

Algunas de las ventajas de este enfoque híbrido son las siguientes:

- **Simplicidad y claridad:** Al ser lineal, facilita tanto la planificación como el desarrollo del proyecto, sobre todo para equipos pequeños donde es complicado paralelizar tareas.
- **Documentación consistente:** Cada fase genera documentación completa, lo que mejora la comprensión tanto de la fase concreta como del proyecto en general, tanto en la ejecución de este como en su posterior evaluación.
- **Facilidad de gestión:** Al tener etapas claramente definidas, es más fácil asignar tareas, establecer hitos y controlar tiempos y recursos.
- **Requisitos bien definidos:** Funciona bien cuando los requisitos están bien definidos desde el inicio y es poco probable que cambien a lo largo de su ejecución.
- **Entrega continua de valor:** La organización del trabajo en sprints permite obtener versiones funcionales parciales del sistema, lo que facilita la detección temprana de errores y la validación progresiva de los objetivos.
- **Flexibilidad ante cambios:** Aunque la planificación inicial es clara, el uso de principios ágiles permite adaptarse a imprevistos y realizar ajustes durante el desarrollo sin comprometer el avance global del proyecto.
- **Seguimiento visual y organizado:** El uso de tableros Kanban ha facilitado la visualización del estado de las tareas en todo momento, mejorando la organización, la priorización y la productividad.

Las fases metodológicas contempladas son las siguientes:

- **Investigación:** Revisión exhaustiva de la literatura relacionada con la aplicación de IA en eCommerce y análisis comparativo de tecnologías y algoritmos disponibles.
- **Diseño del sistema:** Definición de la arquitectura general del sistema y diseño detallado de los componentes y módulos clave.
- **Desarrollo e implementación:** Codificación e integración de los modelos y sistemas de recomendación en el prototipo de eCommerce, asegurando su funcionalidad y escalabilidad.
- **Pruebas y validación:** Realización de pruebas unitarias, pruebas de integración y validación del sistema en términos de usabilidad y rendimiento.
- **Documentación y análisis de resultados:** Recopilación de datos relevantes, análisis de resultados obtenidos y elaboración de la documentación final, incluyendo conclusiones y posibles mejoras que podrían desarrollarse en el futuro.

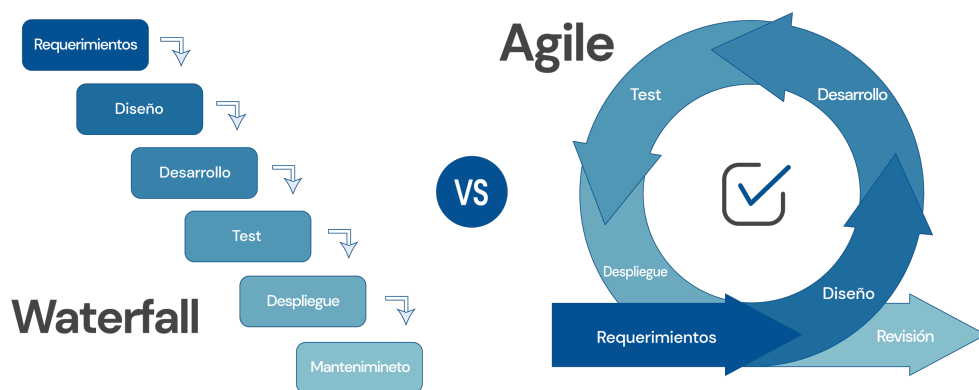


Figura 2.1: Descripción de la imagen

2.4. Fundamentación teórica

En los últimos años, la inteligencia artificial se ha posicionado como una herramienta clave para la transformación digital de numerosos sectores, destacando especialmente su impacto en el comercio electrónico. La capacidad de los sistemas basados en IA para procesar grandes volúmenes de datos, identificar patrones complejos y automatizar decisiones ha permitido a las empresas ofrecer experiencias más personalizadas y eficientes a sus clientes.

Uno de los campos donde la IA ha mostrado mayor relevancia es en el desarrollo de sistemas de recomendación. Estos sistemas tienen como finalidad predecir y sugerir productos o servicios que resulten de interés para los usuarios, tomando en cuenta información diversa, como el historial de comportamiento, preferencias declaradas o características de los productos. En el contexto del *eCommerce*, su implementación no solo mejora la experiencia del cliente, sino que también contribuye al aumento de las ventas, la fidelización y la competitividad empresarial.

A lo largo del tiempo, se han desarrollado diversas metodologías para la optimización de estos sistemas. Desde enfoques tradicionales, como el filtrado colaborativo y los sistemas basados en contenido, hasta modelos más sofisticados que integran métodos híbridos y técnicas de aprendizaje profundo, cada uno de ellos ofrece ventajas específicas y plantea desafíos técnicos. El avance en tecnologías de procesamiento de datos y el desarrollo de algoritmos de aprendizaje automático han sido determinantes para perfeccionar estas metodologías y adaptarlas a las exigencias cambiantes de los mercados digitales.

El análisis de casos prácticos de grandes plataformas líderes en el sector permite observar cómo la aplicación estratégica de la inteligencia artificial, en combinación con sistemas de recomendación y técnicas de marketing automatizado, ha sido clave para su éxito. Estas plataformas han logrado integrar soluciones avanzadas capaces de adaptarse en tiempo real a las preferencias y comportamientos de los usuarios, ofreciendo experiencias altamente personalizadas. Una síntesis de estos beneficios clave puede consultarse en el cuadro de la figura 2.2, donde se resumen los principales aportes de los algoritmos de recomendación personalizada en términos de valor comercial y experiencia del usuario.

Esta fundamentación teórica proporciona el marco conceptual necesario para comprender la importancia y las aplicaciones de la inteligencia artificial en el diseño e implementación de sistemas de recomendación dentro del comercio electrónico. Asimismo, establece las bases para el desarrollo del sistema propuesto en este proyecto, cuyo objetivo principal es aprovechar estas tecnologías emergentes para optimizar la experiencia del usuario y mejorar los procesos de venta.

Beneficios clave de los algoritmos de recomendación personalizada

01 Mayores ventas y tasas de conversión

Los sistemas de recomendación permiten sugerir productos relevantes que aumentan la probabilidad de compra, impulsando las ventas y mejorando el retorno de inversión.

02 Mayor participación del usuario

Al ofrecer recomendaciones personalizadas, se incrementa el tiempo de permanencia y la interacción de los usuarios con la plataforma.

03 Sobrecarga de información reducida

La personalización ayuda a filtrar la información irrelevante, facilitando la toma de decisiones y mejorando la experiencia del cliente.

04 Fidelización de clientes

Los usuarios perciben una experiencia más relevante y satisfactoria, lo que refuerza su vínculo con la marca y mejora la retención a largo plazo.

05 Adaptación en tiempo real

Gracias a los avances en inteligencia artificial, los sistemas pueden adaptarse dinámicamente al comportamiento del usuario en tiempo real, optimizando la recomendación al instante.

Figura 2.2: Cuadro resumen de los beneficios clave de los algoritmos de recomendación personalizada

3. Estado del arte

3.1. Inteligencia artificial aplicada al eCommerce

Definición y evolución de la IA en el comercio electrónico

La inteligencia artificial (IA) ha experimentado un crecimiento acelerado en el comercio electrónico, transformándose desde herramientas básicas de automatización hasta sistemas avanzados de personalización.

Inicialmente, la IA se aplicaba en la gestión de inventarios y en la automatización de tareas repetitivas, lo que mejoraba la eficiencia operativa. Con el desarrollo de algoritmos de aprendizaje profundo y el incremento de datos disponibles, se han implementado sistemas de recomendación, análisis predictivo, chatbots y asistentes virtuales.

Estos avances buscan ofrecer experiencias de usuario personalizadas y optimizar estrategias de marketing, lo que incrementa la competitividad en un mercado digital en constante cambio.

Aplicaciones clave de la IA en el eCommerce

- **Chatbots y asistentes virtuales:** Utilizando técnicas de Procesamiento de Lenguaje Natural (NLP), los chatbots responden de forma automatizada y personalizada a las consultas de los usuarios. A través del aprendizaje continuo de interacciones previas, estos sistemas mejoran la precisión de sus respuestas y predicen necesidades de los clientes. Por ejemplo, Luisa quiere comprar una blusa en H&M, pero tiene dudas sobre las tallas disponibles en su país y las políticas de devolución. En lugar de buscar entre las preguntas frecuentes o contactar con el equipo de soporte, seguramente después de un período de espera largo, inicia una conversación con el chatbot en la web. El asistente le responde de forma inmediata y amable, mostrándole una guía de tallas personalizada según su ubicación y explicándole el procedimiento para devolver productos. Gracias a esta interacción, Luisa se siente más segura y finaliza su compra con confianza. El asistente, al estar entrenado con los datos reales de la empresa y con problemas similares de otros clientes, es capaz de resolver la incidencia en menos tiempo, sin errores y está disponible las 24 horas del día en todos los idiomas.

Soluciones como *Tidio*, *Gorgias*, *Zendesk Answer Bot*, *Intercom* o *Drift* ofrecen asistentes virtuales que integran técnicas de NLP y machine learning. Estas plataformas permiten responder automáticamente a preguntas frecuentes, segmentar usuarios en tiempo real y escalar conversaciones a agentes humanos.

cuando es necesario. Algunas incluso incorporan generación de respuestas automáticas mediante modelos como *GPT* y sistemas de análisis de intención.

Estos asistentes pueden integrarse directamente con plataformas de comercio electrónico como *Shopify*, *WooCommerce* o *BigCommerce*, ofreciendo soporte multicanal (web, email, WhatsApp, Facebook Messenger) y recomendaciones contextuales dentro del flujo de compra. Este tipo de herramientas se ha demostrado que mejora la experiencia del usuario, reduce la tasa de abandono y optimiza los tiempos de atención al cliente a la vez que reduce los costos a la empresa si están bien optimizados.

- **Análisis de sentimientos:** Permite a las empresas analizar grandes volúmenes de opiniones y comentarios para identificar tendencias y posibles problemas emergentes. Mediante técnicas de NLP, se pueden analizar datos de reseñas y redes sociales, ajustando en tiempo real estrategias de marketing. Por ejemplo, eBay detecta un aumento en comentarios negativos en redes sociales relacionados con la batería de un dispositivo móvil lanzado recientemente. Utilizando el análisis de sentimientos, identifica rápidamente la causa del malestar y decide retirar temporalmente los anuncios de ese modelo, mientras lanza una campaña informativa para resolver dudas. Esta capacidad de reacción rápida minimiza el impacto negativo en la marca.

Herramientas como *MonkeyLearn*, *Lexalytics*, *Google Cloud Natural Language*, *IBM Watson NLU* o *MeaningCloud* ofrecen APIs que permiten integrar análisis de sentimientos en flujos de datos como reseñas de productos, encuestas, redes sociales o chats de atención al cliente. Estas soluciones aplican modelos de NLP y aprendizaje automático para clasificar los comentarios como positivos, negativos o neutros, e incluso identificar temas frecuentes o emociones específicas como frustración, entusiasmo o decepción.

Esto permite a los equipos de marketing y soporte tomar decisiones proactivas: lanzar campañas correctivas, ajustar descripciones de producto, cambiar precios o detectar puntos de mejora en la experiencia de compra. Integrado en dashboards de análisis como *Tableau*, *Power BI* o incluso dentro de CRMs como *Salesforce*, este tipo de sistemas mejora la capacidad de reacción de la empresa y reduce el impacto de posibles crisis de reputación.

- **Sistemas de recomendación:** Emplean algoritmos de aprendizaje profundo y filtrado colaborativo para personalizar la experiencia de compra sugiriendo productos relevantes. En plataformas como Amazon, los sistemas de recomendación analizan tanto el comportamiento de los usuarios como las características de los productos, incrementando la probabilidad de compra y mejorando la fidelización. Por ejemplo, Pedro está buscando unos auriculares en Amazon. Después de visitar un par de modelos, la plataforma le sugiere otros auriculares con mejores valoraciones, compatibles con su móvil y dentro del presupuesto que Pedro suele tener. Además, le muestra accesorios complementarios como fundas para el móvil o estuches para los nuevos auriculares. Pedro encuentra justo lo que buscaba sin necesidad de seguir buscando, y termina comprando más de lo que planeaba inicialmente gracias a las recomendaciones personalizadas. Pedro se va con buenas sensaciones porque

esos productos realmente eran necesarios para él, pero no había reparado en ello antes de que el sistema se lo recomendara.

Las grandes plataformas cuentan con sistemas y modelos privados que son considerados los más avanzados e innovadores del sector en estas herramientas. Aun así, cada vez son más las startups que ofrecen este tipo de soluciones en formato SAAS. Las más destacadas en el panorama actual son *Algolia Recommend*, *Luigi's Box*, *Salesforce Commerce Cloud Einstein*, *Nosto* y *Dynamic Yield*, las cuáles ofrecen sistemas de recomendación avanzados que pueden integrarse en tiendas online para proporcionar personalización en tiempo real, con segmentación dinámica y pruebas A/B automatizadas. Plataformas todo-en-uno para *ecommerce*, como *Shopify* con *Shopify Magic*, están ofreciendo recientemente nuevos planes donde la oferta de valor radica en el uso de IA tanto para la maquetación web como para servicios de recomendaciones de productos personalizados, predicciones del comportamiento del cliente y segmentación dinámica.

Combinar estos sistemas con estrategias de ventas online eficaces como los *bundles* (crear packs de 2 o más productos con un descuento si se compran en ese momento juntos), el *upselling* (ofrecer un producto de versión o categoría superior más caro al que está considerando) o el *cross-selling* (sugerir productos relacionados o complementarios en la misma franja de precios), como en el ejemplo anterior, permite maximizar el valor medio de cada pedido y mejorar la experiencia de compra al anticiparse a las necesidades del cliente.

3.2. Sistemas de recomendación en eCommerce

Los sistemas de recomendación son herramientas que ayudan a los usuarios a descubrir productos, servicios o información que puedan ser de su interés. Estos sistemas son ampliamente utilizados en plataformas como Netflix, Amazon, Spotify, entre otros, para personalizar la experiencia del usuario.

Filtrado colaborativo: Definición y concepto teórico

El filtrado colaborativo es una técnica de recomendación que se basa en las interacciones y preferencias de múltiples usuarios para predecir y recomendar elementos que podrían ser de interés para un usuario específico. A diferencia de otros enfoques, como el filtrado basado en contenido, el filtrado colaborativo no requiere información explícita sobre los atributos de los ítems, sino que se apoya en las relaciones entre usuarios e ítems.

El fundamento del filtrado colaborativo consiste en agrupar usuarios por intereses similares, de manera si un grupo de usuarios ha mostrado preferencias similares en el pasado, es probable que tengan preferencias similares en el futuro. De esta manera, las recomendaciones se generan a partir de las similitudes entre usuarios o entre ítems.

Tipos de filtrado colaborativo

Filtrado colaborativo basado en vecinos

Basado en usuarios (User-Based)

- Se centra en encontrar usuarios similares al usuario objetivo y recomendar ítems que estos usuarios hayan valorado positivamente.
- Utiliza métricas como la similitud del coseno, correlación de Pearson o distancia euclidiana para medir la similitud entre usuarios.
- Identifica los k usuarios más similares al usuario objetivo.
- Agrega las preferencias de los vecinos para recomendar ítems no vistos por el usuario objetivo.

Por ejemplo, si el usuario A y el usuario B han calificado positivamente una serie de películas similares, se podría recomendar al usuario A una película que el usuario B ha calificado altamente pero que el usuario A aún no ha visto.

Basado en ítems (Item-Based)

- En lugar de buscar usuarios similares, este método busca ítems similares a aquellos que el usuario ha mostrado interés.
- Se utilizan métricas similares a las del enfoque basado en usuarios para medir la similitud entre ítems, recomendando ítems que son similares a los que el usuario ha valorado positivamente.

Por ejemplo, si un usuario ha disfrutado de una película como “El señor de los anillos”, el sistema puede recomendar otras películas de ciencia ficción con temáticas similares como la serie “Juego de tronos”.

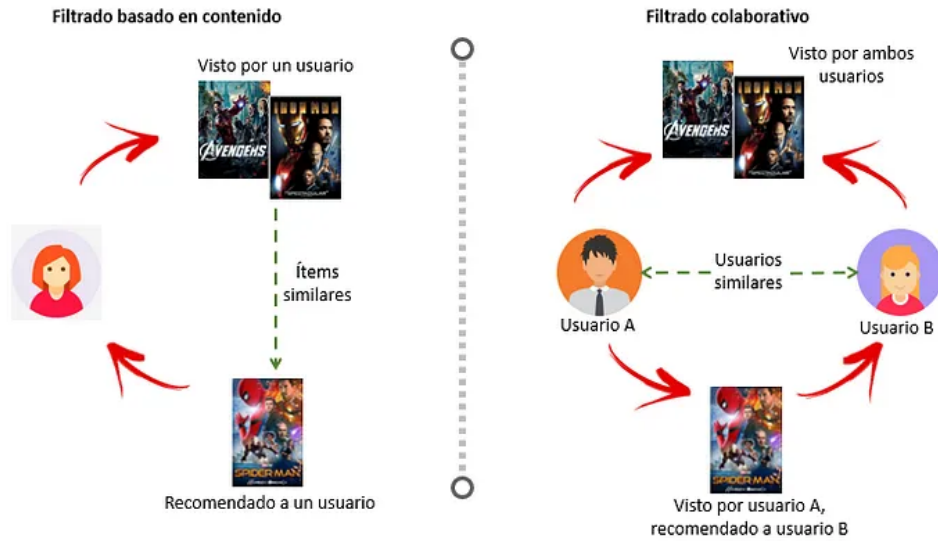


Figura 3.1: Comparación entre filtrado basado en contenido y filtrado colaborativo

Filtrado colaborativo basado en modelos

a. Factores de matriz (Matrix factorization)

La factorización de matrices se origina en los modelos de factores latentes, los cuales postulan la existencia de variables ocultas que caracterizan tanto a los usuarios como a los productos. Por ejemplo, es posible representar películas mediante dos factores numéricos: una dimensión que oscila entre comedia y seriedad, y otra entre realidad y fantasía.

En la primera dimensión, un valor elevado indica una película más seria, mientras que un valor bajo sugiere una mayor inclinación hacia la comedia. De manera análoga, en la segunda dimensión, un valor alto refleja una mayor tendencia hacia la fantasía, y un valor bajo denota un apego a la realidad. Estos factores latentes también se aplican a los usuarios; por ejemplo, un usuario con un valor alto en la primera dimensión preferirá películas serias, y uno con un valor alto en la segunda dimensión mostrará predilección por películas de fantasía.

Con esta representación, tanto usuarios como películas pueden modelarse mediante vectores. Un usuario que disfruta significativamente de películas serias y de fantasía podría ser representado por el vector:

$$\mathbf{u}_1 = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

Por otro lado, un usuario que prefiere comedias y es indiferente entre fantasía y realidad podría ser descrito por:

$$\mathbf{u}_2 = \begin{bmatrix} -2 \\ 0 \end{bmatrix}$$

Del mismo modo, podemos asignar vectores a dos películas específicas: una película seria y de fantasía como *El Señor de los Anillos*, y una comedia realista como *Mean Girls*. Sus vectores serían:

$$\mathbf{p}_1 = \begin{bmatrix} 2 \\ 4 \end{bmatrix}, \quad \mathbf{p}_2 = \begin{bmatrix} -3 \\ -3 \end{bmatrix}$$

Para modelar las calificaciones que cada usuario otorgaría a cada película, se utiliza la combinación lineal de los vectores de usuarios y películas. Por ejemplo, la calificación que el usuario $\mathbf{u}_1$ daría a la película $\mathbf{p}_1$ se calcula mediante el producto punto:

$$\mathbf{u}_1^T \mathbf{p}_1 = 3 \times 2 + 4 \times 4 = 6 + 16 = 22$$

Si se hace esto para todos los usuarios y películas, se tiene el siguiente sistema lineal:

$$\mathbf{R} = \mathbf{U}\mathbf{P}^T,$$

donde

$$\mathbf{U} = \begin{bmatrix} \mathbf{u}_1^T \\ \mathbf{u}_2^T \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ -2 & 0 \end{bmatrix}, \quad \mathbf{P} = \begin{bmatrix} \mathbf{p}_1^T \\ \mathbf{p}_2^T \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ -3 & -3 \end{bmatrix}$$

Y

$$\mathbf{R} = \begin{bmatrix} \mathbf{u}_1^T \mathbf{p}_1 & \mathbf{u}_1^T \mathbf{p}_2 \\ \mathbf{u}_2^T \mathbf{p}_1 & \mathbf{u}_2^T \mathbf{p}_2 \end{bmatrix} = \begin{bmatrix} 22 & -21 \\ -4 & 6 \end{bmatrix}$$

Este modelo refleja que el usuario $\mathbf{u}_1$, quien prefiere películas serias y de fantasía, otorga una calificación más alta a *El Señor de los Anillos* que a *Mean Girls*. Contrariamente, el usuario $\mathbf{u}_2$ aficionado a las comedias, muestra la preferencia opuesta.

En la práctica, los valores de las matrices $\mathbf{U}$ y $\mathbf{P}$ son desconocidos; únicamente disponemos de ciertas entradas de la matriz $\mathbf{R}$. El desafío consiste en estimar de manera precisa las matrices $\mathbf{U}$ y $\mathbf{P}$.

Este problema es una instancia de la descomposición en valores singulares (SVD), con la peculiaridad de que no se tienen todas las entradas de la matriz $\mathbf{R}$, sino solo una pequeña proporción de ellas.

El objetivo es, entonces, encontrar matrices $\mathbf{U}$ y $\mathbf{P}$ de rango K que satisfagan la aproximación $\mathbf{R} \approx \mathbf{U}\mathbf{P}^T$. De este modo, la calificación estimada que el usuario i asigna al artículo j se expresa como:

$$\hat{r}_{ij} = \mathbf{u}_i^T \mathbf{p}_j$$

Para evaluar la proximidad entre el producto de matrices y $\mathbf{R}$, se utilizan diferentes funciones de pérdida. Una métrica común es la raíz del error cuadrático medio (RMSE), calculada como:

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{A}|} \sum_{(i,j) \in \mathcal{A}} (r_{ij} - \mathbf{u}_i^T \mathbf{p}_j)^2}$$

donde r_{ij} es la calificación real del usuario i al artículo j , $\mathcal{A}$ es el conjunto de pares (i, j) para los cuales se conoce r_{ij} , y $|\mathcal{A}|$ es el número total de calificaciones.

En esencia, el RMSE representa la raíz cuadrada del promedio de los cuadrados de las diferencias entre las calificaciones estimadas y las reales.

Otra métrica frecuentemente utilizada es el error absoluto medio (MAE), definido por:

$$\text{MAE} = \frac{1}{|\mathcal{A}|} \sum_{(i,j) \in \mathcal{A}} |r_{ij} - \mathbf{u}_i^T \mathbf{p}_j|$$

El RMSE suele ser preferible en contextos donde los errores de predicción pequeños no son significativos, ya que penaliza de manera más severa los errores grandes en comparación con el MAE.

Para minimizar el RMSE, el problema se plantea como:

$$\min_{\mathbf{U}, \mathbf{P}} \sum_{(i,j) \in \mathcal{A}} \left[(r_{ij} - \mathbf{u}_i^T \mathbf{p}_j)^2 + \lambda (\|\mathbf{p}_j\|^2 + \|\mathbf{u}_i\|^2) \right]$$

donde λ es un parámetro de regularización que ayuda a prevenir el sobreajuste al penalizar la complejidad de los vectores de usuarios y productos.

b. Modelos probabilísticos

Los modelos probabilísticos en el filtrado colaborativo se basan en la teoría de la probabilidad para modelar las interacciones entre usuarios e ítems. Estos modelos asumen que las interacciones observadas (como calificaciones o clics) son el resultado de procesos estocásticos subyacentes que pueden ser capturados mediante distribuciones de probabilidad. La ventaja principal de este enfoque es su capacidad para manejar la incertidumbre y proporcionar estimaciones probabilísticas de las preferencias de los usuarios, lo que puede mejorar la robustez y la interpretabilidad de las recomendaciones.

1. Modelos de mezcla y factorización de matrices bayesianas

Uno de los enfoques más destacados dentro de los modelos probabilísticos es la factorización de matrices bayesianas. Estos modelos extienden las técnicas de factorización de matrices tradicionales (como Singular Value Decomposition, SVD) incorporando priors bayesianos sobre los factores

latentes, lo que permite una inferencia más flexible y una mejor generalización en escenarios con datos escasos.

II. Factorización de matrices bayesianas

En la factorización de matrices bayesianas, se modela la matriz de interacciones R (donde $R_{u,i}$ representa la interacción del usuario u con el ítem i) como el producto de dos matrices latentes U y V , donde cada entrada de U y V está distribuida de acuerdo con una distribución previa. Formalmente, se puede expresar como:

$$R_{u,i} \sim \mathcal{N}(U_u V_i^T, \sigma^2)$$

donde U_u y V_i son los vectores latentes para el usuario u y el ítem i , respectivamente, y $\mathcal{N}$ denota una distribución normal con media $U_u V_i^T$ y varianza σ^2 . Además, se definen priors sobre U y V , típicamente gaussianos:

$$U_u \sim \mathcal{N}(0, \lambda_U^{-1} I)$$

$$V_i \sim \mathcal{N}(0, \lambda_V^{-1} I)$$

Este enfoque permite la incorporación de conocimiento previo y la regularización automática, ayudando a prevenir el sobreajuste y mejorando la capacidad del modelo para generalizar a nuevos datos.

III. Inferencia y aprendizaje:

La inferencia en modelos de factorización de matrices bayesianas generalmente se realiza mediante métodos de inferencia variacional o Markov Chain Monte Carlo (MCMC), que buscan aproximar la distribución posterior de los factores latentes U y V dado los datos observados. El objetivo es maximizar la probabilidad posterior de los parámetros latentes, lo que se logra iterativamente ajustando las estimaciones de U y V para minimizar la divergencia de Kullback-Leibler entre la distribución aproximada y la distribución posterior real.

Ventajas:

- **Manejo de incertidumbre:** Proporciona estimaciones probabilísticas que capturan la incertidumbre en las predicciones.
- **Regularización automática:** Los priors bayesianos ayudan a evitar el sobreajuste, especialmente en escenarios con datos escasos.
- **Flexibilidad:** Permite la incorporación de diferentes tipos de datos y priors personalizados según el dominio de aplicación.

Desventajas:

- **Complejidad computacional:** Los métodos de inferencia probabilística pueden ser computacionalmente costosos, especialmente para grandes conjuntos de datos.
- **Escalabilidad:** A medida que aumenta la dimensión de la matriz de interacciones, el costo de la inferencia puede volverse prohibitivo.

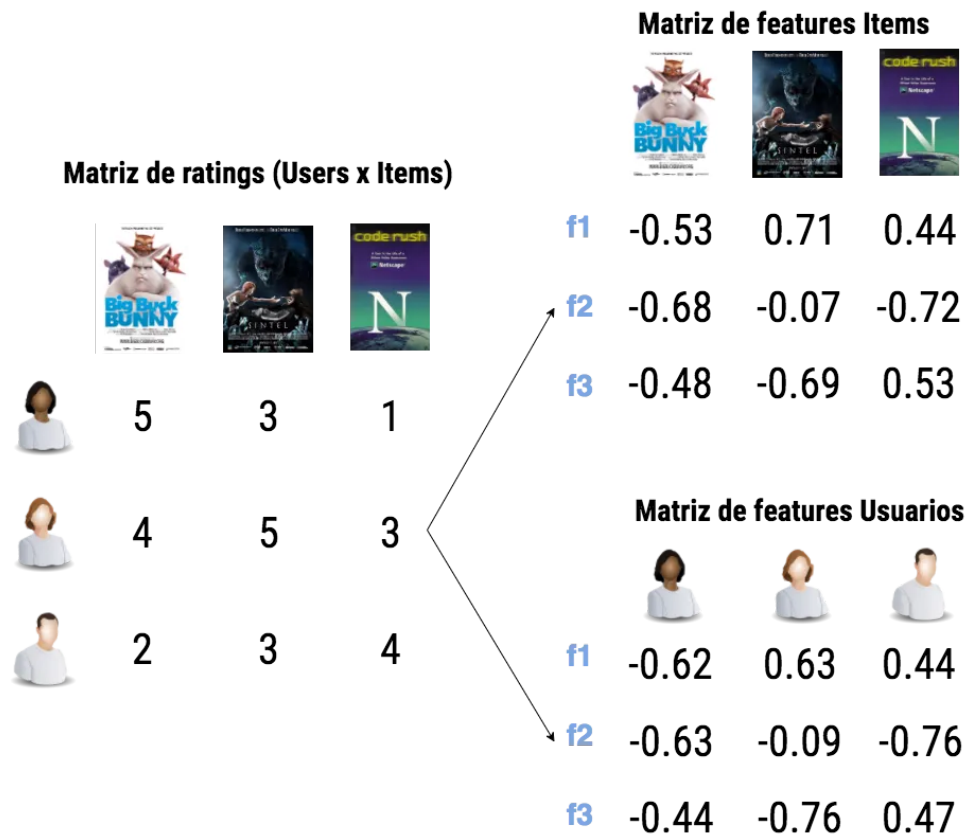


Figura 3.2: Representación visual de la factorización de matrices en sistemas de recomendación

c. Redes neuronales

Las redes neuronales han emergido como una herramienta poderosa en el filtrado colaborativo, gracias a su capacidad para modelar relaciones no lineales y capturar patrones complejos en los datos de interacción entre usuarios e ítems. Las arquitecturas de redes neuronales profundas permiten aprender representaciones ricas y abstractas de los usuarios e ítems, mejorando significativamente la calidad de las recomendaciones.

- I. **Autoencoders para recomendaciones:** Los autoencoders son una clase de redes neuronales utilizadas para aprender representaciones compactas de los datos mediante un proceso de codificación y decodificación. En el contexto de sistemas de recomendación, los autoencoders se utilizan para aprender representaciones latentes de los usuarios e ítems a partir de la matriz de interacciones R .

II. **Arquitectura de autoencoders:** Un autoencoder típico consta de dos partes principales:

- **Encoder:** Mapea la entrada (por ejemplo, las interacciones de un usuario) a un espacio latente de menor dimensión.
- **Decoder:** Reconstruye la salida a partir de la representación latente, intentando replicar la entrada original.

En sistemas de recomendación, el encoder puede tomar las interacciones de un usuario con diferentes ítems y mapearlas a una representación latente que capture las preferencias subyacentes del usuario. El decoder, a su vez, intenta predecir las interacciones faltantes o futuras del usuario a partir de esta representación latente.

Ventajas de los autoencoders:

- **Captura de relaciones complejas:** Pueden modelar interacciones no lineales y relaciones complejas entre usuarios e ítems.
- **Reducción de dimensionalidad:** Aprenden representaciones latentes compactas que facilitan el procesamiento y la recomendación.
- **Flexibilidad:** Pueden integrarse con otros tipos de datos y características adicionales para mejorar las recomendaciones.

III. **Redes neuronales convolucionales (CNNs) para recomendaciones:** Las CNNs, conocidas por su éxito en el procesamiento de datos espaciales como imágenes, también se han adaptado para sistemas de recomendación. En este contexto, las CNNs pueden capturar patrones locales y relaciones espaciales en las interacciones usuario-ítem, mejorando la precisión de las recomendaciones.

Aplicación de CNNs en recomendaciones:

- **Captura de patrones espaciales:** Las CNNs pueden identificar patrones locales en la matriz de interacciones que podrían pasar desapercibidos para otros modelos.
- **Integración con datos de contenido:** Las CNNs pueden combinar datos de interacción con datos de contenido (como imágenes de productos) para generar recomendaciones más ricas y contextuales.

IV. **Ejemplos de redes neuronales en recomendaciones:**

- **DeepCoNN:** Una arquitectura que utiliza dos redes neuronales convolucionales para modelar las interacciones de usuarios e ítems por separado antes de combinarlas para predecir calificaciones.
- **Neural Collaborative Filtering (NCF):** Combina técnicas de filtrado colaborativo con redes neuronales profundas para aprender interacciones no lineales entre usuarios e ítems.

V. **Ventajas de las redes neuronales:**

- **Capacidad de aprender representaciones complejas:** Pueden capturar interacciones no lineales y relaciones de alta dimensión entre usuarios e ítems.
- **Escalabilidad y flexibilidad:** Las arquitecturas pueden adaptarse y escalarse según la complejidad de los datos y las necesidades del sistema.
- **Integración de múltiples fuentes de información:** Permiten la incorporación de datos adicionales (como contenido multimedia) para enriquecer las recomendaciones.

VI. Desventajas:

- **Requerimientos computacionales:** Las redes neuronales profundas requieren recursos computacionales significativos para el entrenamiento y la inferencia.
- **Complejidad de la implementación:** La configuración y optimización de arquitecturas de redes neuronales puede ser compleja y requiere experiencia técnica avanzada.
- **Interpretabilidad:** Los modelos basados en redes neuronales pueden ser menos interpretables en comparación con enfoques más simples como la factorización de matrices.

d. Filtrado colaborativo basado en sistemas híbridos

Combina múltiples enfoques de recomendación para aprovechar las ventajas de cada uno y mitigar sus limitaciones. Por ejemplo, combinar filtrado colaborativo con filtrado basado en contenido o integrar diferentes técnicas dentro del filtrado colaborativo.

Métricas de similitud

Para evaluar la similitud entre usuarios o ítems, se emplean diversas métricas:

Similitud del coseno Mide el coseno del ángulo entre dos vectores de calificación.

$$\text{Sim}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

Correlación de Pearson Evalúa la correlación lineal entre las calificaciones de dos usuarios.

$$\text{Sim}(A, B) = \frac{\sum (A_i - \bar{A})(B_i - \bar{B})}{\sqrt{\sum (A_i - \bar{A})^2} \sqrt{\sum (B_i - \bar{B})^2}}$$

Distancia euclidiana Calcula la distancia directa entre dos vectores de calificación.

$$\text{Dist}(A, B) = \sqrt{\sum (A_i - B_i)^2}$$

Desafíos del filtrado colaborativo

A pesar de su efectividad, el filtrado colaborativo enfrenta varios desafíos técnicos:

I. Escasez de datos (Sparsity)

En sistemas con una gran cantidad de ítems y usuarios, la matriz de interacción suele ser muy dispersa, lo que dificulta la identificación de patrones de similitud.

Soluciones:

- Utilizar técnicas de factorización de matrices para inferir valores faltantes.
- Implementar métodos de imputación de datos.

II. Arranque en frío (Cold Start)

Dificulta la recomendación para nuevos usuarios o ítems que no tienen suficientes interacciones registradas.

Soluciones:

- Incorporar información adicional mediante filtrado basado en contenido.
- Emplear sistemas híbridos que combinen múltiples fuentes de información.

III. Escalabilidad

El aumento en la cantidad de usuarios e ítems puede impactar negativamente el rendimiento de los algoritmos basados en vecinos.

Soluciones:

- Optimizar cálculos mediante estructuras de datos eficientes.
- Utilizar algoritmos de aproximación y técnicas de reducción de dimensionalidad.

IV. Sincronización de preferencias

Las preferencias de los usuarios pueden cambiar con el tiempo, lo que requiere que los sistemas actualicen continuamente sus modelos.

Soluciones:

- Implementar modelos dinámicos que consideren la evolución temporal de las preferencias.
- Utilizar ventanas de tiempo para ponderar más las interacciones recientes.

Algoritmo	Ventajas	Desventajas	Ejemplos reales	Complej.
KNN	Fácil de implementar, interpretable y útil con pocos datos	Escala mal con grandes volúmenes, requiere datos densos	Amazon (básico), sistemas escolares	Baja
SVD	Buena precisión, capta relaciones latentes entre usuarios e ítems	Requiere entrenamiento, alta demanda computacional	Netflix, Spotify, MovieLens	Media-Alta
Content-Based	Personalización individual sin necesidad de otros usuarios	Limitado por metadatos, no descubre preferencias nuevas	Artículos, e-commerce con metadatos	Media
Híbridos	Combina lo mejor de varios enfoques, mayor precisión	Difícil de implementar, ajuste complejo	YouTube, LinkedIn, Netflix (avanzado)	Alta

Tabla 3.1: Comparativa de algoritmos de recomendación en función de características clave

3.3. Casos de uso en grandes plataformas.

El estado actual de la inteligencia artificial en el eCommerce muestra una evolución continua hacia sistemas más avanzados y personalizados. Las aplicaciones de IA en este ámbito, como los chatbots, el análisis de sentimientos y los sistemas de recomendación, han optimizado significativamente la experiencia del usuario y los procesos de venta. Sin embargo, aún existen desafíos, como la escalabilidad, la escasez de datos y los sesgos éticos.

Las grandes plataformas, como Amazon y Netflix, han demostrado el éxito de la integración de enfoques híbridos, ofreciendo importantes lecciones sobre adaptabilidad e innovación para futuras investigaciones en este campo.

3.3.1. Amazon

Amazon utiliza una combinación avanzada de algoritmos de recomendación y análisis de datos para personalizar la experiencia de compra de sus usuarios. Entre sus tecnologías y características principales se destacan:

- **Sistemas de recomendación:** Mediante el uso de IA, Amazon sugiere productos basándose en el historial de compras y navegación del usuario. Las recomendaciones abarcan categorías como “productos relacionados”, “los clientes que compraron esto también compraron” y “recomendaciones personalizadas”.
- **Análisis predictivo:** Amazon predice la demanda de productos utilizando análisis predictivo para optimizar su inventario, lo que permite una gestión eficiente de stock y tiempos de entrega.
- **Chatbots y asistentes virtuales:** Alexa, el asistente de voz de Amazon, permite a los usuarios realizar compras mediante comandos de voz, mejorando la

accesibilidad y facilidad de compra.

- **Optimización de precios:** Amazon ajusta sus precios en tiempo real a través de algoritmos de IA que maximizan la competitividad y las ventas, adaptándose a las fluctuaciones del mercado.

Gracias a estas tecnologías, Amazon ofrece una experiencia personalizada, permitiendo a los usuarios encontrar productos de interés con mayor rapidez, lo cual incrementa la probabilidad de compra y la satisfacción del cliente. Las estrategias de recomendación y optimización de precios han demostrado ser efectivas en el aumento de las ventas y en la retención de clientes.

3.3.2. Netflix

Netflix implementa sistemas de recomendación avanzados que identifican y sugieren contenido relevante para cada usuario. Este sistema se basa en:

- **Algoritmos de recomendación:** Netflix utiliza algoritmos de aprendizaje automático para sugerir películas y series, tomando en cuenta el historial de visualización y preferencias previas de cada usuario.
- **Perfilado de usuarios:** La plataforma construye perfiles detallados que permiten entender mejor los gustos de los usuarios y adaptar las recomendaciones en consecuencia.
- **Pruebas A/B:** Netflix realiza constantes pruebas A/B para evaluar y optimizar tanto la interfaz de usuario como las recomendaciones, asegurando que la experiencia del usuario sea intuitiva y atractiva.

Las recomendaciones personalizadas de Netflix ayudan a retener a los usuarios en la plataforma, promoviendo el consumo de contenido y aumentando el tiempo de visualización. Este sistema mejora notablemente la satisfacción del usuario al proporcionar contenido relevante y atractivo.

3.3.3. Spotify

Spotify utiliza inteligencia artificial para generar recomendaciones musicales personalizadas, facilitando el descubrimiento de nueva música:

- **Algoritmos de recomendación:** Las playlists personalizadas como “Discover Weekly” y “Daily Mix” son producto de algoritmos que analizan el historial de escucha del usuario y adaptan las recomendaciones en función de sus preferencias ([Spotify's Recommendation System](#)).
- **Análisis de datos:** Spotify analiza grandes cantidades de datos de escucha para identificar patrones y tendencias musicales, mejorando la relevancia de sus recomendaciones .

- **Filtrado colaborativo:** Utilizando técnicas de filtrado colaborativo, Spotify recomienda canciones que otros usuarios con gustos similares han escuchado, fomentando la exploración musical.

Las recomendaciones personalizadas en Spotify mejoran la experiencia de los usuarios, quienes descubren música nueva acorde a sus gustos, lo cual incrementa su fidelidad a la plataforma. Este sistema también aumenta el tiempo de permanencia de los usuarios en la plataforma.

3.3.4. Alibaba

Alibaba emplea inteligencia artificial para optimizar la experiencia de compra y potenciar las ventas mediante:

- **Recomendaciones personalizadas:** Alibaba utiliza IA para recomendar productos a sus usuarios, basándose en el comportamiento de compra y navegación, lo que mejora la relevancia de las sugerencias.
- **Asistentes virtuales:** Tmall Genie, un asistente virtual de Alibaba, ayuda a los usuarios en sus compras, proporcionando una experiencia interactiva y eficiente.
- **Análisis de sentimiento:** Alibaba analiza reseñas y opiniones para comprender mejor la percepción de los usuarios sobre los productos, lo cual permite ajustes en la oferta y la mejora de servicios.

Las recomendaciones personalizadas de Alibaba han demostrado incrementar las ventas de la plataforma y mejorar la satisfacción del cliente. El uso de asistentes virtuales y análisis de sentimiento optimiza la calidad del servicio, ofreciendo una experiencia de compra más satisfactoria y enfocada en las necesidades de los clientes.

4. Elicitación de requisitos

La correcta elicitación de requisitos constituye una de las fases más críticas en el desarrollo de cualquier sistema software. En el contexto del presente proyecto, orientado a la construcción de un sistema de recomendación inteligente para plataformas de comercio electrónico, la obtención precisa de requisitos funcionales y no funcionales resulta indispensable para garantizar el cumplimiento de los objetivos de negocio, la calidad del producto final y la satisfacción de los usuarios.

Para ello, se han seguido las directrices aprendidas en asignaturas como *Introducción a la Ingeniería del Software* y *Planificación y Gestión de Proyectos Informáticos*, donde se enfatiza la necesidad de transformar las necesidades iniciales de los interesados en requisitos documentados, priorizados y técnicamente viables.

El primer paso en el proceso de elicitación de requisitos consiste en identificar las necesidades de los interesados, también denominadas necesidades de negocio o necesidades de cliente. En el contexto de este proyecto, estas necesidades se centran en aumentar el valor medio de compra en un entorno de comercio electrónico, mejorando la rentabilidad de las ventas mediante la aplicación de estrategias de recomendación personalizadas basadas en datos reales de comportamiento de los usuarios.

Estas necesidades, inicialmente formuladas de manera general a partir de los objetivos del proyecto, deben ser transformadas en requisitos claros, documentados y priorizados que permitan guiar el desarrollo de la solución tecnológica de forma efectiva. Esta transformación da lugar a los denominados requisitos del cliente o requisitos de usuario, que representan una descripción estructurada de las expectativas y objetivos del sistema utilizando un lenguaje accesible y orientado a negocio.

En este proyecto, los principales interesados son el alumno desarrollador y el tutor académico, quienes han definido conjuntamente las metas del sistema y supervisan su correcta concreción. Para facilitar la planificación del trabajo y la organización de las etapas de implementación, los requisitos de cliente han sido priorizados y, en su mayoría, expresados mediante historias de usuario que reflejan las necesidades funcionales de la plataforma de recomendaciones.

Finalmente, a partir de los requisitos de cliente se derivan los requisitos de producto, los cuales describen con mayor nivel de detalle las características técnicas específicas que el sistema debe cumplir para satisfacer las necesidades planteadas. Estos requisitos de producto se han estructurado en distintas categorías, incluyendo tanto requisitos funcionales como no funcionales, y han sido documentados de manera precisa para asegurar su correcta implementación y validación durante el desarrollo del proyecto.

A continuación, se describen los participantes clave en el proyecto, el estado actual del dominio de aplicación, los objetivos perseguidos, el catálogo de

requisitos, los criterios de aceptación establecidos y las matrices de trazabilidad elaboradas para asegurar la correcta correspondencia entre los objetivos y los requisitos definidos.

4.1. Participantes en el proyecto

Campo	Descripción
Participante	Antonio Silva Gordillo
Organización	Universidad de Sevilla
Rol	Desarrollador
¿Es desarrollador?	Sí
¿Es cliente?	No
¿Es usuario?	No
Comentarios	Alumno del grado de Ingeniería Informática - Tecnologías Informáticas en la Escuela Técnica Superior de Ingeniería Informática de la Universidad de Sevilla que realizará por completo el desarrollo de la aplicación web.

Tabla 4.1: Información del participante Antonio Silva Gordillo

Campo	Descripción
Participante	Juan Antonio Ortega Ramírez
Organización	Departamento de Lenguajes y Sistemas Informáticos
Rol	Tutor
¿Es desarrollador?	No
¿Es cliente?	No
¿Es usuario?	No
Comentarios	Profesor titular perteneciente al Departamento de Lenguajes y Sistemas Informáticos de la Universidad de Sevilla, encargado de supervisar y guiar el trabajo del primer participante.

Tabla 4.2: Información del participante Juan Antonio Ortega Ramírez

Campo	Descripción
Organización	Departamento de Lenguajes y Sistemas Informáticos
Dirección	Escuela Técnica Superior de Ingeniería Informática. Avda. Reina Mercedes s/n. C.P. 41012 Sevilla, España.
Teléfono	(+34) 954555964
Fax	(+34) 954557139
Email	buzon@lsi.us.es
Web	http://www.lsi.us.es
Comentarios	Departamento adscrito a la Universidad de Sevilla que se encuentra ubicado en el edificio de la Escuela Técnica Superior de Ingeniería Informática de dicha universidad, en el campus de Reina Mercedes.

Tabla 4.3: Información de la organización: Departamento de Lenguajes y Sistemas Informáticos

Campo	Descripción
Organización	Universidad de Sevilla
Dirección	C/ S. Fernando, 4, C.P. 41004 Sevilla, España.
Teléfono	Centralita: (+34) 954551000
Fax	(+34) 954211294
Email	Servicio de Asuntos Generales: negasuntosg@us.es - serviasuntosg@us.es
Web	http://www.us.es
Comentarios	La Universidad de Sevilla es una institución que presta un servicio público de educación superior mediante el estudio, la docencia y la investigación, así como la generación, desarrollo y difusión del conocimiento al servicio de la Sociedad y de la Ciudadanía.

Tabla 4.4: Información de la organización: Universidad de Sevilla

4.2. Estado Actual

Como se ha visto en la introducción de este documento, los sistemas tradicionales de recomendación en plataformas de comercio electrónico suelen basarse en reglas estáticas o filtros manuales, lo que limita su capacidad para adaptarse al comportamiento real y cambiante del mercado. En consecuencia, reducen la efectividad de las estrategias de venta como el *cross-selling* o el *upselling*.

Además, para las pequeñas y medianas empresas, la adopción de sistemas de recomendación avanzados representa un desafío técnico y económico considerable, dado que las soluciones existentes en el mercado suelen requerir altos niveles de inversión en infraestructura, mantenimiento continuo de modelos y personal especializado en inteligencia artificial.

Estas limitaciones clave de los sistemas tradicionales se resumen visualmente en la Figura 4.1, donde se destacan los principales obstáculos que enfrentan al plantearse la implementación de este tipo de herramientas.



Figura 4.1: Limitaciones comunes de los sistemas de recomendación tradicionales

4.3. Objetivos del sistema

El objetivo de este proyecto es desarrollar un módulo de recomendación basado en técnicas de aprendizaje automático que permita a las pymes ofrecer recomendaciones de calidad adaptadas al comportamiento de sus clientes, sin necesidad de grandes inversiones iniciales. Para validar la efectividad de este módulo, se ha desarrollado en paralelo una plataforma web que simula un entorno de comercio electrónico real, construida utilizando el framework **Django**, que permite cargar catálogos de productos reales y medir el rendimiento de los diferentes algoritmos de recomendación implementados.

Este planteamiento no sólo permite evaluar de manera objetiva la precisión y la utilidad de las recomendaciones generadas, sino que también proporciona un banco de pruebas controlado que facilita el ajuste, la comparación y la evolución de los modelos en condiciones similares a las de un entorno comercial real.

Campo	Descripción
Código	OBJ-001
Título	Integrar un módulo de recomendación en la plataforma de Ecommerce
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	El sistema deberá ofrecer recomendaciones de productos personalizadas basados en datos de los productos, con el objetivo de poner en práctica el potencial de estos algoritmos en un entorno real parametrizable.
Importancia	Alta
Urgencia	Alta
Estado	Validado
Comentarios	El motor de recomendaciones deberá integrarse de forma transparente en la experiencia de usuario del ecommerce, respetando el diseño y la navegación existente.

Tabla 4.5: Descripción del objetivo OBJ-001

Campo	Descripción
Código	OBJ-002
Título	Desarrollar una plataforma funcional inspirada en Ecommerce real como entorno de validación
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	Se deberá construir una plataforma web de Ecommerce utilizando Django que permita cargar catálogos de productos reales junto con metainformación que alimente el sistema de recomendación como reviews, categorías, descripciones, etc.
Importancia	Alta
Urgencia	Alta
Estado	Validado
Comentarios	La plataforma deberá incluir funcionalidades básicas de navegación, fichas de producto, perfil de usuario, detalle del producto y sus recomendaciones.

Tabla 4.6: Descripción del objetivo OBJ-002

Campo	Descripción
Código	OBJ-003
Título	Implementar múltiples algoritmos de recomendación para su evaluación comparativa
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	El sistema deberá incluir diferentes enfoques de recomendación, como filtrado colaborativo, sistemas basados en contenido y modelos híbridos, permitiendo su selección y comparación a través de métricas específicas.
Importancia	Alta
Urgencia	Media
Estado	Validado
Comentarios	Los algoritmos implementados deberán ser configurables y evaluables de forma independiente, con el objetivo de alcanzar al menos un 50 % de precisión en los test de las métricas seleccionadas.

Tabla 4.7: Descripción del objetivo OBJ-003

Campo	Descripción
Código	OBJ-004
Título	Realizar un proceso ETL para preparar los datos de productos del dataset de Amazon
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	El proyecto deberá incluir un proceso ETL que permita extraer datos relevantes del dataset público <i>Amazon-Reviews-2023</i> disponible en HuggingFace, transformarlos al formato adecuado, limpiarlos y almacenarlos en un archivo CSV preparado para su posterior carga automática en los modelos de la plataforma Django.
Importancia	Alta
Urgencia	Alta
Estado	Validado
Comentarios	El proceso ETL debe garantizar el preprocesado e importación correctamente un mínimo de 100.000 registros del dataset 'Amazon-Reviews-2023', la integridad y consistencia de los datos, filtrando productos incompletos o irrelevantes, normalizando los campos necesarios y adaptando la estructura para facilitar su inserción eficiente en la base de datos de pruebas.

Tabla 4.8: Descripción del objetivo OBJ-004

Campo	Descripción
Código	OBJ-005
Título	Garantizar viabilidad en entornos limitados
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	El sistema (ETL + modelos + servidor Django) debe poder ejecutarse en local con ≤ 12 GB de RAM y una CPU de 4 hilos, demostrando su viabilidad sin depender de infraestructura de alto rendimiento.
Importancia	Alta
Urgencia	Media
Estado	Validado
Comentarios	Prueba de concepto incluyendo preprocesado, importación, entrenamiento y ejecución del servidor en máquina local.

Tabla 4.9: Descripción del objetivo OBJ-005

Campo	Descripción
Código	OBJ-006
Título	Documentar y validar todas las fases del sistema
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	Incluir manual de instalación, manual de usuario, guía de carga de datos y reportes detallados de pruebas unitarias e integración para cada componente.
Importancia	Media
Urgencia	Media
Estado	Validado
Comentarios	Debe cubrir diseño, desarrollo, pruebas, validación y despliegue, garantizando trazabilidad y facilidad de uso.

Tabla 4.10: Descripción del objetivo OBJ-006

4.4. Catálogo de requisitos

4.4.1. Introducción a las historias de usuario

Los requisitos de cliente de este proyecto están redactados mediante historias de usuario, acercándose de este modo al desarrollo en metodologías ágiles. El formato de dichas historias es el propuesto por Mike Cohn (2004), siendo este el que se muestra en la Figura 4.2.

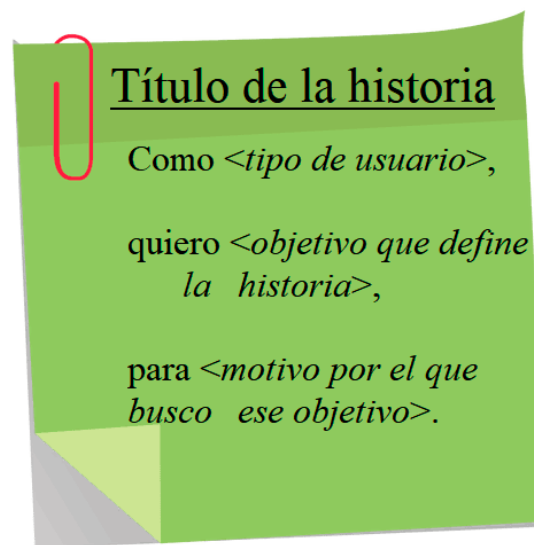


Figura 4.2: Formato de redacción de historias de usuario propuesto por Mike Cohn (2004).

Además del título y la descripción, cada historia de usuario está compuesta por unos criterios de aceptación o condiciones de satisfacción, que son unas pruebas de aceptación de alto nivel que se cumplen cuando la historia de usuario está realizada correctamente. Estos criterios ayudan a detallar la historia de usuario, mejorando la explicación para su posterior desarrollo.

4.4.2. Roles de usuario

Para la elaboración de las historias de usuario, se ha seguido el proceso de ejemplo propuesto por Mike Cohn en su libro *User Stories Applied for Agile Software Development* (2004). En primer lugar, es necesario identificar los roles de los usuarios del sistema. Para la aplicación desarrollada en este proyecto, se consideran inicialmente dos roles principales: usuarios registrados y usuarios administradores.

Por lo tanto, los dos roles de usuario considerados en el sistema, junto con los detalles que Mike Cohn aconseja definir para cada uno de ellos, son los siguientes (véanse las tablas de las Figuras 4.11 a 4.11).

Campo	Descripción
Código	ROL-001
Título	Usuario registrado
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	Cualquier persona con un usuario registrado en la plataforma. Puede explorar el catálogos de productos y recibir recomendaciones generales basadas en popularidad y también recibir recomendaciones personalizadas basadas en su historial de navegación e interacciones. Puede ver su perfil y cambiar la contraseña
Frecuencia de uso	Media
Experiencia	Media
Manejo con software	Medio
Objetivo	Visualizar y filtrar el catalogo de productos e interactuar con los algoritmos de recomendación.

Tabla 4.11: Descripción del rol ROL-001 - Usuario registrado

Campo	Descripción
Código	ROL-002
Título	Administrador
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	Usuario con permisos especiales para gestionar el catálogo de productos y sua propiedades asociadas y usuarios garantizando el correcto funcionamiento del sistema.
Frecuencia de uso	Alta
Experiencia	Alta
Manejo con software	Alto
Objetivo	Administrar la plataforma, actualizar productos y sus propiedades, perfiles de usuario, monitorizar el impacto de las recomendaciones y asegurar una experiencia de usuario óptima.

Tabla 4.12: Descripción del rol ROL-002 - Administrador

4.4.3. Historias épicas

Tras la definición de los objetivos y roles de usuario, comienza el proceso de redacción de las historias de usuario. Para facilitar esta tarea, se ha elaborado un mapa de historias de usuario, en el cual, mediante agrupaciones temáticas, se han recogido todas las funcionalidades que los actores del sistema desean realizar.

Este mapa de historias de usuario está dividido en dos niveles: las primeras filas están compuestas por historias épicas u objetivos generales (de alto nivel y bajo nivel de detalle), mientras que las filas siguientes estarán formadas por historias de usuario más concretas, derivadas de estas épicas. A continuación, se detallan las principales historias épicas identificadas para el desarrollo de la plataforma.

Campo	Descripción
ID	HUE-001
Título	Navegar y filtrar productos con recomendaciones personalizadas
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	Como usuario con cualquier rol, quiero navegar por el catálogo de productos y poder filtrarlos por diferentes criterios, para encontrar más fácilmente artículos con unas características determinadas y así evaluar como trabaja cada algoritmo en diferentes tipos de productos

Tabla 4.13: Historia de usuario épica 001 - Navegar y filtrar productos con recomendaciones personalizadas

Campo	Descripción
ID	HUE-002
Título	Gestionar catálogo de productos
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	Como administrador, quiero gestionar los productos disponibles en la tienda, para mantener actualizado el catálogo sobre el que funcionan las recomendaciones.

Tabla 4.14: Historia de usuario épica 002 - Gestionar catálogo de productos

Campo	Descripción
ID	HUE-003
Título	Administrar algoritmos de recomendación
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	Como usuario con cualquier rol, quiero seleccionar y configurar los algoritmos de recomendación activos, para evaluar su impacto.

Tabla 4.15: Historia de usuario épica 003 - Administrar algoritmos de recomendación

Campo	Descripción
ID	HUE-004
Título	Realizar el proceso ETL para cargar datos de productos
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	Como desarrollador, quiero realizar el proceso ETL que permita transformar el dataset de Amazon en datos válidos para los modelos de Django, para disponer de un catálogo realista en la plataforma de prueba.

Tabla 4.16: Historia de usuario épica 004 - Realizar el proceso ETL

Campo	Descripción
ID	HUE-005
Título	Documentar y validar todas las fases del sistema
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	Como responsable de producto, quiero disponer de toda la documentación necesaria (manual de instalación, guía de usuario, pautas de despliegue y reportes de pruebas) para garantizar la mantenibilidad y trazabilidad de cada componente del sistema.
Historia épica	HUE-005
Criterio de aceptación	– Manual de instalación completo que cubra requisitos previos y pasos de despliegue. – Guía de usuario con ejemplos de uso de la plataforma y del módulo de recomendaciones. – Reportes de pruebas unitarias.
Importancia	Media
Estado	Validado

Tabla 4.17: Historia de usuario HUE-005 - Documentar y validar todas las fases del sistema

4.4.4. Historias de usuario

Cada historia de usuario está ligada a una historia épica a través del campo *Historia épica*, de esta manera las historias de usuario añaden información y detalle a los objetivos generales del trabajo.

De igual manera, cada historia de usuario posee un campo denominado *criterio de aceptación*, donde se especifican las condiciones necesarias para que dicha historia se considere completa y satisfactoria, como expone Mike Cohn en sus recomendaciones. Este campo ayuda a definir de manera precisa qué debe lograrse para dar por concluida cada funcionalidad.

La representación de historias de usuario se ha documentado en formato digital a través de tablas estructuradas, en vez de en tarjetas físicas como es habitual en metodologías ágiles. Esto nos ha permitido reflejar de manera clara y estructurada las funcionalidades que los distintos roles del sistema deben poder ejecutar dentro de la plataforma y el funcionamiento del módulo de recomendación.

Se destaca que las historias de usuario recogidas son aproximaciones documentales, sirviendo principalmente como guía de trabajo. La verdadera

concreción y validación de los requisitos ha surgido de las conversaciones y acuerdos alcanzados entre el alumno como desarrollador y el tutor de la escuela, las cuales permitieron encauzar correctamente el desarrollo de este proyecto.

Campo	Descripción
ID	HU-001
Título	Filtrar productos por categoría, precio y calificación
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Historia asociada	épica HUE-001
Descripción	Como usuario registrado, quiero aplicar filtros de búsqueda por categoría, rango de precio y calificación mínima en el listado de productos, para encontrar más fácilmente artículos de mi interés.
Criterios de aceptación	– El usuario podrá buscar productos mediante un formulario visible. – Se podrán combinar múltiples filtros (categoría + precio + calificación). – El sistema actualizará los resultados aplicando los filtros seleccionados. – Se permitirá ordenar los resultados por precio ascendente, descendente o mejor valoración.
Importancia	Alta
Estado	Validado

Tabla 4.18: Historia de usuario HU-001 - Filtrar productos por categoría, precio y calificación

Campo	Descripción
ID	HU-002
Título	Gestionar catálogo de productos
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	Como administrador, quiero poder crear, editar y eliminar productos en el catálogo, para mantener actualizada la oferta de artículos disponibles para los usuarios y alimentar correctamente el motor de recomendaciones.
Historia épica	HUE-002
Criterio de aceptación	– El sistema permitirá añadir nuevos productos incluyendo campos como título, precio, descripción, categoría y valoración inicial. – El administrador podrá editar los datos de productos existentes. – Será posible eliminar productos del catálogo de forma controlada. – Los cambios en el catálogo impactarán en las recomendaciones generadas.
Importancia	Alta
Estado	Validado

Tabla 4.19: Historia de usuario HU-002 - Gestionar catálogo de productos

Campo	Descripción
ID	HU-003
Título	Configurar y administrar algoritmos de recomendación
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	Como usuario, quiero poder seleccionar y cambiar el algoritmo de recomendación activo en la recomendación de un producto concreto, para poder analizar el comportamiento y comparar el impacto de diferentes estrategias sobre las métricas.
Historia épica	HUE-003
Criterio de aceptación	– El usuario podrá seleccionar el algoritmo de recomendación activo desde un panel de configuración. – Será posible alternar entre diferentes algoritmos implementados (filtrado colaborativo, basado en contenido, híbrido). – Los cambios deben aplicarse sin afectar la integridad de los datos de los usuarios o productos.
Importancia	Alta
Estado	Validado

Tabla 4.20: Historia de usuario HU-003 - Configurar algoritmos de recomendación

Campo	Descripción
ID	HU-004
Título	Ejecutar proceso ETL para cargar datos de productos
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	Como desarrollador, quiero realizar un proceso de extracción, transformación y carga (ETL) del dataset de Amazon Reviews 2023, para generar un archivo CSV limpio que permita cargar los productos en la base de datos de la plataforma Django.
Historia épica	HUE-004
Criterio de aceptación	– El proceso debe extraer solo los campos relevantes del dataset original. – Se deben eliminar registros incompletos o con valores incoherentes. – El archivo generado debe estar en formato CSV y seguir el esquema de los modelos Django de la plataforma. – El proceso debe asegurar la compatibilidad de tipos de datos y formatos (textos, números, fechas, etc.).
Importancia	Alta
Estado	Validado

Tabla 4.21: Historia de usuario HU-004 - Ejecutar proceso ETL para cargar datos de productos

Campo	Descripción
ID	HU-005
Título	Crear documentación y guías del sistema
Épica	HUE-005
Versión	1.0 (26/04/2025)
Autores	Antonio Silva, Juan Antonio Ortega Ramírez
Fuentes	Antonio Silva
Descripción	Como usuario final y desarrollador, quiero acceder a manuales y guías que me expliquen cómo instalar, configurar, usar y probar el sistema de recomendaciones y la plataforma <i>eCommerce</i> , de modo que pueda operar y mantener el sistema de manera autónoma.
Criterio de aceptación	– Disponibilidad de un documento de instalación que detalle instalación de dependencias y despliegue. – Guía de usuario con capturas de pantalla y casos de uso (registro, navegación, configuración de algoritmos). – Documento de carga de datos que explique el formato CSV y uso del comando de importación. – Reporte de pruebas con instrucciones para ejecutar test unitarios y métricas de cobertura.
Importancia	Media
Estado	Validado

Tabla 4.22: Historia de usuario HU-005 - Crear documentación y guías del sistema

4.5. Análisis de requisitos

4.5.1. Introducción

En esta sección se realiza un análisis detallado del sistema a desarrollar, partiendo de los requisitos previamente elicitados. A diferencia de enfoques basados en diagramas de casos de uso clásicos, este análisis se estructura en torno a la clasificación de los requisitos en cuatro categorías principales: requisitos de información, requisitos funcionales, requisitos no funcionales y reglas de negocio.

Cada apartado recoge de manera sistemática los diferentes tipos de necesidades que el sistema debe cubrir, permitiendo trazar un mapa completo de las expectativas, limitaciones y directrices que guiarán el diseño, la implementación y la validación del sistema de recomendación desarrollado.

4.5.2. Catálogo de requisitos

Requisitos de Información

Los requisitos de información definen los conjuntos de datos que el sistema debe gestionar, almacenar y procesar para ofrecer sus funcionalidades principales. En este proyecto, enfocado en la simulación de un ecommerce con un sistema de recomendaciones inteligente, resulta fundamental manejar de forma adecuada tanto la información de productos como los datos de usuarios y sus interacciones.

ID	Título	Descripción
INF-001	Productos del catálogo	Información básica de productos disponibles: nombre, precio, categorías, imagen, descripción breve y valoración media.
INF-002	Filtros aplicados en búsqueda	Datos de filtros utilizados por los usuarios: categoría, rango de precios, valoración mínima, ordenación.
INF-003	Información de usuarios registrados	Datos personales de los usuarios: nombre, correo electrónico, teléfono, dirección.
INF-004	Elementos de navegación (Navbar)	Información dinámica de los enlaces de navegación visibles según estado del usuario (login, logout, perfil).
INF-005	Detalle de producto	Información extendida de un producto individual: imagen, descripción larga, categorías, precio y valoración media.

Tabla 4.23: Requisitos de Información del Sistema

4.5.3. Requisitos funcionales

Los requisitos funcionales describen las funcionalidades específicas que el sistema debe ofrecer para cumplir las necesidades de los usuarios y del negocio. Definen las acciones que la aplicación debe ser capaz de ejecutar y las interacciones principales entre usuarios y sistema.

ID	Nombre	Descripción
RF-001	Visualizar productos	El sistema debe permitir a los usuarios navegar y visualizar productos disponibles con sus principales atributos.
RF-002	Filtrar productos	Permitir aplicar filtros de búsqueda avanzados sobre el catálogo de productos.
RF-003	Registro de usuarios	Permitir el registro de nuevos usuarios en la plataforma mediante un formulario seguro.
RF-004	Inicio de sesión y cierre de sesión	Permitir autenticarse y cerrar sesión de forma segura en la plataforma.
RF-005	Editar información de usuario	Permitir a los usuarios registrados actualizar sus datos personales.
RF-006	Configurar algoritmo de recomendación	Permitir a los usuarios registrados seleccionar el algoritmo activo desde el dashboard para aplicarlo al producto seleccionado.
RF-007	Visualizar métricas de rendimiento	Mostrar estadísticas de rendimiento de los algoritmos activos en el dashboard.
RF-008	Navegación adaptativa (Navbar)	Mostrar un menú dinámico adaptado al estado de autenticación del usuario.
RF-009	Visualizar detalle del producto con recomendaciones	Mostrar detalle completo de producto y sugerencias personalizadas.
RF-010	Importación masiva de productos desde CSV	El sistema debe ser capaz de importar productos de forma masiva a partir de un archivo CSV respetando un formato concreto definido, mediante un comando de sistema.

Tabla 4.24: Requisitos funcionales del sistema

4.5.4. Reglas de negocio

Las reglas de negocio definen políticas y restricciones internas que el sistema debe respetar para mantener su coherencia operativa y funcional.

ID	Regla	Descripción
RN-001	Autenticación requerida para administrar	Solo usuarios autenticados podrán acceder al perfil y al panel de administración.
RN-002	Algoritmo único activo	Solo un algoritmo de recomendación puede estar activo simultáneamente en una relación usuario-producto-sesión-algoritmo.
RN-003	Integridad del catálogo	Los productos deben tener nombre, precio y categoría para ser publicados en el catálogo.
RN-004	Protección de formularios	Todos los formularios deben estar protegidos frente a ataques CSRF y validar la entrada de datos.
RN-005	Navbar dinámico según estado	El menú de navegación debe adaptarse en tiempo real al estado de login/logout del usuario.

Tabla 4.25: Reglas de negocio del sistema

4.5.5. Requisitos no funcionales

Los requisitos no funcionales describen atributos de calidad del sistema, como el rendimiento, la seguridad, la usabilidad o la escalabilidad, que afectan a su comportamiento general.

ID	Requisito	Descripción
RNF-001	Tiempo de respuesta	El sistema debe responder a interacciones de usuarios en menos de 2 segundos en el 95 % de los casos.
RNF-002	Escalabilidad	Soportar hasta 10.000 productos en catálogo sin pérdida de rendimiento significativa.
RNF-003	Seguridad de datos	Proteger datos personales y de navegación mediante HTTPS, validaciones y buenas prácticas.
RNF-004	Accesibilidad y usabilidad	Garantizar un diseño responsive, accesible y fácil de usar en diferentes dispositivos.

Tabla 4.26: Requisitos no funcionales del sistema

4.6. Matrices de trazabilidad

En esta sección se presentan las matrices de trazabilidad que muestran las relaciones entre los objetivos del sistema, definidos en la Sección 4.3, y los requisitos expresados en forma de historias de usuario, detallados en la Sección 4.5.2. Primero se presentan las relaciones entre los objetivos y las historias de usuario épicas, proporcionando una visión global de las dependencias principales. A continuación, se expone una matriz más detallada que relaciona los objetivos del sistema con las historias de usuario específicas. Finalmente, se muestra la relación jerárquica entre las historias de usuario épicas y las historias de usuario que las desarrollan en detalle.

4.6.1. Matriz de trazabilidad entre los objetivos y las historias de épicas de usuario

Esta matriz de trazabilidad relaciona los objetivos principales del sistema con las historias de usuario épicas, permitiendo identificar qué historias de alto nivel contribuyen a la consecución de cada objetivo.

Historia Épica	OBJ-001	OBJ-002	OBJ-003	OBJ-004	OBJ-005	OBJ-006
HUE-001	X	X				
HUE-002		X	X			
HUE-003			X	X		
HUE-004				X	X	
HUE-005						X

Tabla 4.27: Matriz de trazabilidad entre los objetivos del sistema y las historias de usuario épicas

4.6.2. Matriz de trazabilidad entre los objetivos y las historias de usuario

Esta matriz detalla la relación entre los objetivos del sistema y las historias de usuario específicas, proporcionando una vista más granular de las funcionalidades que soportan cada objetivo.

Historia de Usuario	OBJ-001	OBJ-002	OBJ-003	OBJ-004	OBJ-005	OBJ-006
HU-001	X					
HU-002		X				
HU-003			X			
HU-004				X	X	
HU-005						X

Tabla 4.28: Matriz de trazabilidad entre los objetivos del sistema y las historias de usuario

4.6.3. Matriz de trazabilidad entre las historias de usuario épicas y las historias de usuario

Esta matriz muestra la relación entre las historias de usuario épicas y las historias de usuario específicas que desarrollan cada épica en mayor detalle.

Historia de Usuario	HUE-001	HUE-002	HUE-003	HUE-004
HU-001	X			
HU-002		X		
HU-003			X	
HU-004				X

Tabla 4.29: Matriz de trazabilidad entre historias de usuario épicas y las historias de usuario

4.6.4. Matriz de trazabilidad entre objetivos y requisitos funcionales

Esta matriz refleja cómo cada requisito funcional contribuye a cumplir uno o varios de los objetivos principales del proyecto.

Objetivo \ RF	RF-001	RF-002	RF-003	RF-004	RF-005	RF-006	RF-007	RF-008	RF-009	RF-010
OBJ-001 – Módulo recomendación	X	X	X	X						
OBJ-002 – Plataforma <i>eCommerce</i>	X	X	X	X	X		X	X	X	
OBJ-003 – Implementar algoritmos y métricas							X			
OBJ-004 – Proceso ETL										X
OBJ-005 – Viabilidad local										X
OBJ-006 – Documentación							X			

Tabla 4.30: Matriz de trazabilidad entre objetivos del sistema y requisitos funcionales

4.6.5. Matriz de trazabilidad entre objetivos y requisitos no funcionales

Esta matriz muestra cómo los requisitos no funcionales soportan la calidad esperada para alcanzar los objetivos.

Objetivo \ RNF	RNF-001	RNF-002	RNF-003	RNF-004
OBJ-001 – Módulo recomendación	X	X	X	X
OBJ-002 – Plataforma <i>eCommerce</i>	X	X	X	X
OBJ-003 – Implementar algoritmos y métricas				X
OBJ-004 – Proceso ETL				
OBJ-005 – Viabilidad local		X		
OBJ-006 – Documentación				X

Tabla 4.31: Matriz de trazabilidad entre objetivos del sistema y requisitos no funcionales

4.6.6. Matriz de trazabilidad entre requisitos funcionales y requisitos de información

Esta matriz muestra qué requisitos funcionales utilizan o dependen de qué requisitos de información.

RF \ INF	INF-001	INF-002	INF-003	INF-004	INF-005
RF-001 - Visualizar productos	X				X
RF-002 - Filtrar productos	X	X			
RF-003 - Mostrar recomendaciones	X				
RF-004 - Registro de usuarios			X		
RF-005 - Inicio/cierre sesión			X	X	
RF-006 - Editar perfil			X		
RF-007 - Configurar algoritmo					
RF-008 - Visualizar métricas					
RF-009 - Navbar adaptativo				X	
RF-010 - Ver detalle producto	X				X

Tabla 4.32: Matriz de trazabilidad entre requisitos funcionales y requisitos de información

4.6.7. Matriz de trazabilidad entre reglas de negocio y requisitos funcionales

Esta matriz refleja cómo cada requisito funcional contribuye a cumplir uno o varios de los objetivos principales del proyecto.

RN \ RF	RF-001	RF-002	RF-003	RF-004	RF-005	RF-006	RF-007	RF-008	RF-009	RF-010
RN-001 - Autenticación requerida				X	X	X	X	X	X	
RN-002 - Algoritmo único activo							X			
RN-003 - Integridad del catálogo	X	X								X
RN-004 - Protección de formularios				X	X	X				
RN-005 - Navbar dinámico					X				X	

Tabla 4.33: Matriz de trazabilidad entre reglas de negocio y requisitos funcionales

5. Planificación

En este capítulo se expone el **plan de desarrollo del proyecto**, centrado en definir las tareas a ejecutar, los hitos principales, los roles y recursos humanos involucrados, así como la distribución temporal (cronograma) y la estimación de costes. Además, se incluyen los planes de gestión complementarios (riesgos, calidad y comunicaciones), alineados con las buenas prácticas de la ingeniería del software.

5.1. Planificación temporal

La planificación temporal del proyecto deberá cumplimentar aproximadamente 300 horas, que son las requeridas por el plan de estudios. Se dividirá el proyecto en hitos, los cuales corresponderán a un conjunto de tareas en específico, con una secuencia lógica adecuada.

Hito	Descripción
01	Definición del problema, estudio teórico, establecimiento de objetivos generales.
02	Planificación del proyecto: elaboración del EDT, diccionario de tareas, cronograma y estimación de costes y recursos.
03	Análisis del dataset de Amazon Reviews 2023, diseño e implementación del proceso ETL para limpieza, normalización y generación de CSV de productos listo para carga.
04	Elicitación de requisitos, selección de tecnologías, diseño de la arquitectura y componentes del sistema. Modelado de datos. Puesta en marcha del entorno de desarrollo.
05	Codificación e implementación de la plataforma web, integración del módulo de recomendación y desarrollo del backend en Django.
06	Integración del sistema, validación y desarrollo de un frontend funcional.
07	Pruebas de integración, validación de algoritmos de recomendación y correcciones.
08	Elaboración de manuales de usuario e instalación. Conclusiones y cierre del proyecto.

Tabla 5.1: Hitos del proyecto y sus descripciones

Para la fase de planificación inicial y la estimación de costes se han asignado un total de diez horas. La ejecución de los cuatro sprints que conforman la solución de

software requerida implica un esfuerzo de 230 horas. Finalmente se, se dedicarán 15 horas a la elaboración de los manuales de usuario e instalación y otras 15 horas a la preparación de la defensa. De este modo, la carga de trabajo total da como resultado 300 horas, satisfaciendo los requisitos establecidos en el plan de estudios

Hito	Fecha de Inicio	Fecha de Fin	Horas
01	20/02/2025	28/02/2025	30
02	01/03/2025	07/03/2025	10
03	08/03/2025	20/03/2025	50
04	21/03/2025	10/04/2025	50
05	11/04/2025	02/05/2025	100
06	03/05/2025	10/05/2025	30
07	11/05/2025	18/05/2025	15
08	19/05/2025	26/05/2025	15
Total de Horas			300

Tabla 5.2: Planificación temporal del proyecto.

A continuación se presenta un diagrama de Gantt de los hitos, junto con sus fechas de inicio y fin, así como las dependencias directas entre ellos.

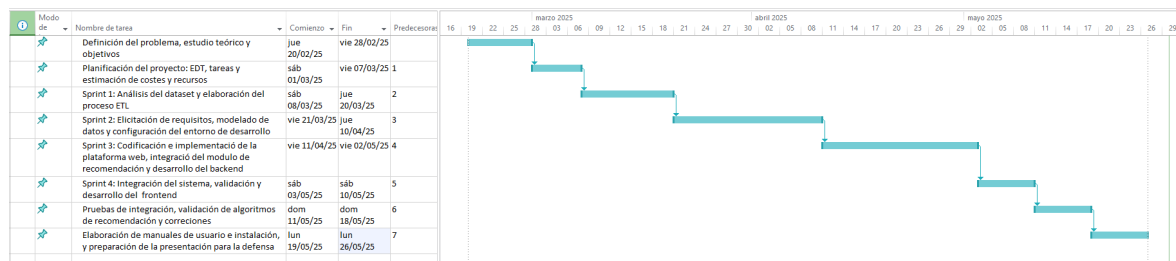


Figura 5.1: Diagrama de Gantt de la planificación inicial del proyecto

A continuación se presenta la tabla 5.3 donde se aprecia de forma más clara los hitos, las fechas y las dependencias.

ID	Hito	Inicio	Fin	Predecesora
01	Definición del problema, estudio teórico y establecimiento de objetivos generales	20/02/2025	28/02/2025	—
02	Planificación del proyecto: elaboración del EDT, diccionario de tareas, cronograma y estimación de costes y recursos	01/03/2025	07/03/2025	01
03	Análisis del dataset de Amazon Reviews 2023, diseño e implementación del proceso ETL para limpieza, normalización y generación de CSV de productos listo para carga	08/03/2025	20/03/2025	02
04	Elicitación de requisitos, diseño de la arquitectura y modelo de datos, instalación y configuración del entorno de desarrollo	21/03/2025	03/04/2025	03
05	Codificación e implementación de la plataforma web, integración del módulo de recomendación y desarrollo del backend en Django	04/04/2025	13/04/2025	04
06	Integración del sistema, validación y desarrollo del frontend	14/04/2025	28/04/2025	05
07	Pruebas de integración, validación de algoritmos de recomendación y correcciones	29/04/2025	07/05/2025	06
08	Elaboración de manuales de usuario e instalación. Conclusiones y cierre del proyecto	15/05/2025	26/05/2025	07

Tabla 5.3: Resumen cronológico de los hitos del proyecto con fechas y dependencias

Se han asignado los diferentes perfiles profesionales del equipo a lo largo del proyecto de la manera que muestra la tabla 5.4 garantizando un desarrollo optimizado con una carga de trabajo adecuada cada miembro según su grado de responsabilidad y sus características técnicas.

Hito	Descripción	Jefe de Proyecto	Analista	Desarrollador	Tester
01	Definición del problema, estudio teórico y objetivos generales	X	X		
02	Planificación del proyecto: EDT, cronograma y recursos	X	X		
03	Análisis del dataset y diseño del proceso ETL		X	X	
04	Requisitos, diseño de la arquitectura y modelo de datos. Configuración del entorno		X	X	
05	Desarrollo del backend, frontend y recomendador			X	
06	Integración del sistema y validación inicial	X		X	X
07	Pruebas finales y validación de resultados				X
08	Manuales, instalación y preparación de defensa	X	X	X	X

Tabla 5.4: Matriz de asignación de perfiles a los hitos del proyecto, con descripción

5.2. Planificación de costes

A partir de la planificación temporal estimada anteriormente, se detallan los costes asociados para la realización del proyecto. El trabajo se estima que se llevará a cabo entre el 20 de febrero y el 26 de mayo de 2025, con una duración de 13 semanas y un total de 300 horas efectivas de trabajo.

El equipo está compuesto por cuatro perfiles profesionales comunes en entornos de desarrollo tecnológico: jefe de proyecto, analista, desarrollador y tester (QA). Cada miembro ha trabajado de forma local utilizando un equipo personal, específicamente, un portátil Acer Aspire 3 con 12GB de RAM. Esta configuración permite reducir notablemente los costes materiales, cumpliendo con uno de los objetivos clave del proyecto: la optimización de recursos y la capacidad de desarrollar el proyecto de manera accesible a PYMES.

5.2.1. Costes de personal

Los sueldos medios anuales actualizados a 2025 para cada perfil se indican en el Cuadro 5.5.

Rol	Salario Anual (€)	Fuente
Jefe de Proyecto	47.000	Glassdoor
Analista	34.900	Jobted
Desarrollador	30.000	Talent.com
Tester (QA)	25.000	Glassdoor

Tabla 5.5: Sueldos medios anuales por rol en España (2025)

Considerando una jornada laboral anual de 1.760 horas, se obtiene el coste por hora base. A estos valores se les aplica un incremento del 34,8 % correspondiente a las cargas de la Seguridad Social:

Rol	Salario/h (€)	Coste con SS (34,8 %) (€)	Coste redondeado (€)
Jefe de Proyecto	26,70	35,98	36,00
Analista	19,83	26,72	26,70
Desarrollador	17,05	22,98	23,00
Tester (QA)	14,20	19,14	19,10

Tabla 5.6: Coste por hora estimado incluyendo Seguridad Social

La distribución de las 300 horas entre los perfiles se realiza en proporción a las responsabilidades de cada rol. El cuadro siguiente muestra el número de horas asignadas y el coste total por perfil (Cuadro 5.7).

Perfil	Tareas del Proyecto	Horas	Sueldo por Hora (€)	Coste (€)
Jefe de Proyecto	Planificación y coordinación	16	36.00	576.00
	Seguimiento y control	16	36.00	576.00
	Cierre y documentación	8	36.00	288.00
Analista	Definición de requisitos	20	26.70	534.00
	Diseño funcional y técnico	18	26.70	480.60
	Análisis de datos y modelado	17	26.70	453.90
Desarrollador	Implementación backend	38	23.00	874.00
	Desarrollo frontend	32	23.00	736.00
	Integración del sistema	25	23.00	575.00
	Algoritmos de recomendación y entrenamiento	25	23.00	575.00
Tester	Diseño de pruebas	25	19.10	477.50
	Ejecución y validación	40	19.10	764.00
	Informe de errores	20	19.10	382.00
Total	—	300	—	7.285,00

Tabla 5.7: Costes de desarrollo del proyecto desglosado por perfil profesional y tipo de tarea

5.2.2. Costes materiales

Cada miembro del equipo ha utilizado un equipo propio: un Acer Aspire 3 con 12GB de RAM. El coste estimado por unidad es de 550€, lo que supone un total de:

- 4 portátiles Acer Aspire 3 (12GB RAM): $4 \times 550€ = 2.200€$

Total hardware: 2.200€

5.2.3. Costes de software

- Dado que Django es un proyecto open source y el resto de herramientas y librerías son gratuitas en un entorno local el entorno de desarrollo no tiene costes asociados.

- Sistemas operativos: 600€

Total software: 600€

5.2.4. Costes de servicios

Los costes asociados a electricidad, conexión a internet y otros gastos del lugar de trabajo se estiman en:

Total servicios: 1.000€

5.2.5. Margen para imprevistos

A partir del desglose de costes reales del proyecto, se considera el siguiente escenario:

- **Coste de personal:** 7.285,00 €
- **Hardware (4 portátiles Acer Aspire 3, 12GB RAM):** 2.200,00 €
- **Software (sistemas operativos):** 600,00 €
- **Servicios (electricidad, internet, suministros):** 1.000,00 €
- **Total base:** 11.085,00 €

Imprevistos (15 %): 1.662,75 €

5.2.6. Costes totales

Subtotal con imprevistos = 11,085,00 + 1,662,75 = 12,747,75 €

Margen de beneficio (20 %): 2.549,55 €

Coste total estimado: 15.297,30 €

Tipo de gasto	Coste (€)
Hardware (4 portátiles Acer Aspire 3)	2.200,00
Software (sistemas operativos)	600,00
Personal	7.285,00
Servicios	1.000,00
Imprevistos (15 %)	1.662,75
Margen de beneficio (20 %)	2.549,55
Total estimado	15.297,30

Tabla 5.8: Coste total estimado del proyecto con 300 horas y entorno local

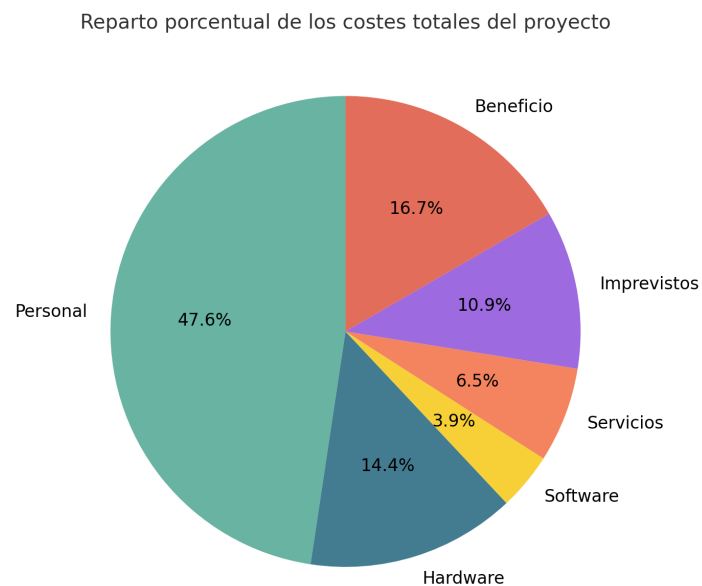


Figura 5.2: Reparto porcentual de los costes totales del proyecto

5.3. Plan de gestión de riesgos

La correcta gestión de riesgos es esencial para minimizar posibles incidencias durante el desarrollo y garantizar la finalización exitosa del proyecto. A continuación, se identifican los principales riesgos, su probabilidad e impacto estimados, y las medidas preventivas o de contingencia planificadas:

Riesgo	Probabilidad	Impacto	Medida de mitigación
Pérdida de datos	Baja	Alta	Copias de seguridad automáticas semanales y almacenamiento en la nube.
Retrasos por problemas técnicos	Media	Media	Sprints cortos, revisiones frecuentes y resolución temprana de bloqueos.
Indisponibilidad de hardware	Baja	Media	Uso de hardware propio con soporte inmediato y disponibilidad de equipos alternativos.
Cambios en requisitos	Media	Alta	Documentación continua y validación frecuente con el tutor o cliente.
Dependencia de fuentes de datos externas	Baja	Media	Descarga anticipada de datasets y almacenamiento local de versiones críticas.
Baja del responsable o miembros clave	Baja	Alta	Documentación exhaustiva y avance incremental para facilitar sustituciones.

Tabla 5.9: Resumen de riesgos y planes de contingencia

5.4. Plan de gestión de la calidad

La calidad del producto final se garantiza mediante la aplicación de buenas prácticas en todas las fases del desarrollo. Las estrategias de control de calidad previstas incluyen:

- **Revisiones de código:** inspecciones periódicas del código fuente para garantizar claridad, mantenibilidad y ausencia de errores críticos.
- **Testing automatizado:** implementación de pruebas unitarias y funcionales que validan el comportamiento correcto de los módulos clave.
- **Checklist de entregables:** verificación sistemática de cada entregable según requisitos funcionales y estándares definidos.
- **Pruebas con usuarios:** validación de la usabilidad y robustez del sistema mediante interacción directa o simulada con usuarios finales.

Los criterios de aceptación que se aplican para cada tarea o hito son los siguientes:

- Cumplimiento de los requisitos funcionales y técnicos definidos.
- Superación satisfactoria de las pruebas previstas en el plan de calidad.
- Disponibilidad de documentación técnica y de usuario, actualizada y verificada.
- Revisión y validación final del entregable por parte de los responsables correspondientes.

5.5. Plan de gestión de las comunicaciones

La comunicación del proyecto se gestiona de forma estructurada, con el objetivo de mantener una coordinación fluida entre todos los actores implicados. El plan contempla:

- **Reuniones de seguimiento semanales:** revisión del progreso, resolución de bloqueos y planificación de próximas tareas. En proyectos individuales, estas pueden documentarse como simulaciones con el tutor académico.
- **Herramientas colaborativas:** uso de Trello para la gestión de tareas, Google Drive para compartir documentación y GitHub como repositorio de control de versiones y revisión de código.
- **Informes de avance:** generación de informes periódicos que reflejan el estado del proyecto y sirven como soporte de comunicación formal con el tutor o cliente simulado.

5.6. Justificación metodológica

El proyecto ha seguido una **metodología incremental**, organizada en fases secuenciales con entregas parciales. Este enfoque permite:

- Validar los avances de manera iterativa.
- Detectar y corregir desviaciones tempranamente.
- Adaptar los requisitos a partir del feedback recibido.

La integración continua y las revisiones periódicas han facilitado una evolución controlada y una mejora constante de la calidad del producto.

5.7. Sostenibilidad y optimización de recursos

Desde su concepción, el proyecto ha priorizado la sostenibilidad y la eficiencia. Se han adoptado decisiones clave como:

- **Uso de hardware local:** aprovechamiento de equipos existentes, reduciendo el consumo de recursos adicionales.
- **Tecnologías open source:** empleo de soluciones libres como Django y herramientas colaborativas en la nube, lo que permite minimizar costes y barreras de acceso.
- **Replicabilidad:** el enfoque adoptado resulta fácilmente reproducible por pequeñas y medianas empresas, favoreciendo su implantación en entornos reales.

6. Diseño de la solución

6.1. Introducción

En este capítulo se describe la arquitectura completa de la plataforma web de sistemas de recomendación, destacando los componentes clave y las razones que han guiado cada decisión de diseño. También se incluye un análisis general del dataset empleado, ya que muchas decisiones de diseño se han tomado acorde a él. SE ha obtenido así una integración perfecta en el sistema desarrollado. El principal objetivo es lograr un entorno de experimentación de los diferentes algoritmos de recomendación, usando productos y metadatos reales en un entorno estable y flexible.

6.2. Análisis y justificación del dataset: *Amazon Reviews 2023*

Para el desarrollo y evaluación del sistema recomendador se ha seleccionado el conjunto de datos **Amazon Reviews 2023**, descargado el **3 de febrero de 2025** desde [HuggingFace Datasets](https://huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023) (<https://huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023>). Esta elección se fundamenta en un análisis comparativo con otras alternativas como Retailrocket y el estudio publicado en arXiv (*referencia 2307.09688*), y queda documentada en el anexo correspondiente (véase *Dataset contemplados*).

Motivación y ventajas:

- **Cobertura temporal y temática:** *Amazon Reviews 2023* recoge valoraciones de productos reales desde 1996 hasta 2023, abarcando más de 570 millones de reseñas y 48 millones de productos distribuidos en 33 categorías distintas.
- **Calidad y herramientas asociadas:** El repositorio incluye scripts para transformación, preprocesado y generación de conjuntos de entrenamiento, validación y prueba, así como modelos preentrenados como BLaIR y datasets complementarios (por ejemplo, Amazon-C4) útiles para tareas de recuperación semántica avanzada.
- **Actualización y representatividad:** La magnitud y frecuencia de actualización del dataset garantizan la validez de los patrones de comportamiento, proporcionando un entorno realista y exigente para evaluar algoritmos de recomendación.

Trazabilidad y reproducibilidad: Con el fin de garantizar la replicabilidad del estudio, se indica la URL exacta del dataset empleado (<https://huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023>) y la fecha

concreta de descarga. Todos los scripts de preprocesado, filtrado y selección de muestras se encuentran disponibles públicamente en el repositorio oficial del trabajo.¹.

Estructura y campos principales del dataset:

Campo	Tipo	Descripción
reviewerID	String	Identificador anónimo del usuario que realiza la valoración. Permite mapear usuarios y analizar comportamiento de compra.
asin	String	Identificador único del producto (ASIN). Clave primaria para trazabilidad y consolidación de valoraciones.
overall	Integer	Valoración numérica (de 1 a 5 estrellas). Núcleo para algoritmos colaborativos y métricas de rendimiento.
reviewText	String	Texto libre redactado por el usuario. Base para análisis de sentimiento y modelos de contenido.
summary	String	Resumen breve de la valoración, útil para la extracción rápida de opiniones clave.
category	String	Categoría del producto, esencial para filtrado, segmentación y análisis por sector.
brand	String	Marca asociada al producto, permite estudiar la fidelidad y popularidad de marcas.
unixReviewTime	Integer	Fecha de la review en formato UNIX timestamp, útil para análisis temporales.
imageURL	String	URLs de imágenes del producto. Potencial para enriquecer la visualización en la interfaz.
verified_purchase	Boolean	Indica si la compra fue verificada. Útil para filtrar valoraciones poco fiables.
vote	Integer	Número de votos de utilidad recibidos por la review, indicador de su relevancia.

Tabla 6.1: Campos principales del dataset *Amazon Reviews 2023*, descargado desde HuggingFace el 3 de febrero de 2025.

Limitaciones y consideraciones: A pesar de su amplitud, el dataset presenta sesgos habituales en plataformas comerciales: sobrerrepresentación de productos populares, usuarios hiperactivos y posible ruido o spam en los textos. Para mitigar estos efectos, se aplicaron filtros de limpieza, muestreos y segmentaciones específicas durante la fase de preprocesado. Esta estrategia permitió centrarse en un subconjunto de datos manejable y representativo, ajustado a los recursos disponibles en entornos académicos. Como se detalla en la sección [Recursos](#)

¹https://github.com/antoniosilva096/tfg_recommender_system

utilizados, la viabilidad técnica del sistema se ha demostrado incluso en equipos personales con especificaciones modestas.

6.3. Diseño de la arquitectura y sus componentes

Con el objetivo de facilitar la carga de productos reales y ofrecer una buena experiencia de uso, se ha desarrollado una arquitectura basada en módulos que, aunque interactúan entre sí, tienen un gran grado de independencia, consiguiendo así un sistema robusto y escalable. Dicho sistema integra algunos componentes clásicos de un sistema web, orientado a un catálogo de producto y registro de usuarios, así como módulos más avanzados de *machine learning* para generar las recomendaciones. Esta arquitectura permite almacenar, procesar y analizar grandes volúmenes de información de usuarios, productos y valoraciones, generando recomendaciones precisas y adaptadas al usuario y el producto seleccionado.

La arquitectura del sistema se estructura en torno a cuatro tecnologías clave:

- **Django**
- **PostgreSQL**
- **Django REST Framework**
- **Librerías de *machine learning* en Python** (scikit-learn, surprise)

Django actúa como columna vertebral de la aplicación, orquestando la lógica web, la exposición de APIs REST y la administración del modelo de datos. PostgreSQL se encarga del almacenamiento persistente de toda la información relevante, incluyendo usuarios, productos, categorías, valoraciones y métricas de recomendación. El módulo de recomendaciones emplea modelos de filtrado colaborativo, *content-based* y algoritmos híbridos, entrenados y evaluados con las librerías seleccionadas.

Los modelos se entrenan *offline* a partir de datasets masivos, se serializan en disco y se cargan dinámicamente en el backend para ofrecer recomendaciones personalizadas a la plataforma.

Cada componente ha sido seleccionado y configurado para garantizar la escalabilidad, la facilidad de mantenimiento y la posibilidad de futuras ampliaciones (por ejemplo, integración de nuevos algoritmos o conexión con fuentes externas de datos).

En la Figura 6.1 se puede observar un esquema general de la arquitectura.

A lo largo de esta sección se detallará el papel de cada uno de los componentes, así como han sido integrados y su rol respecto al funcionamiento global de la solución propuesta.

6.3.1. Arquitectura

La arquitectura del sistema recomendador está diseñada bajo el principio de separación de responsabilidades, articulando cuatro capas especializadas: *presentación*, *lógica de negocio*, *persistencia de datos* y *procesamiento inteligente* basado en *machine learning*. Esta estructura modular no solo facilita el desarrollo, mantenimiento y escalado del sistema, sino que también permite una integración sencilla de nuevos algoritmos, fuentes de datos o interfaces en el futuro.

La interacción entre capas sigue un flujo perfectamente orquestado y desacoplado, donde cada bloque cumple una función crítica dentro de la solución. (siguiente página)

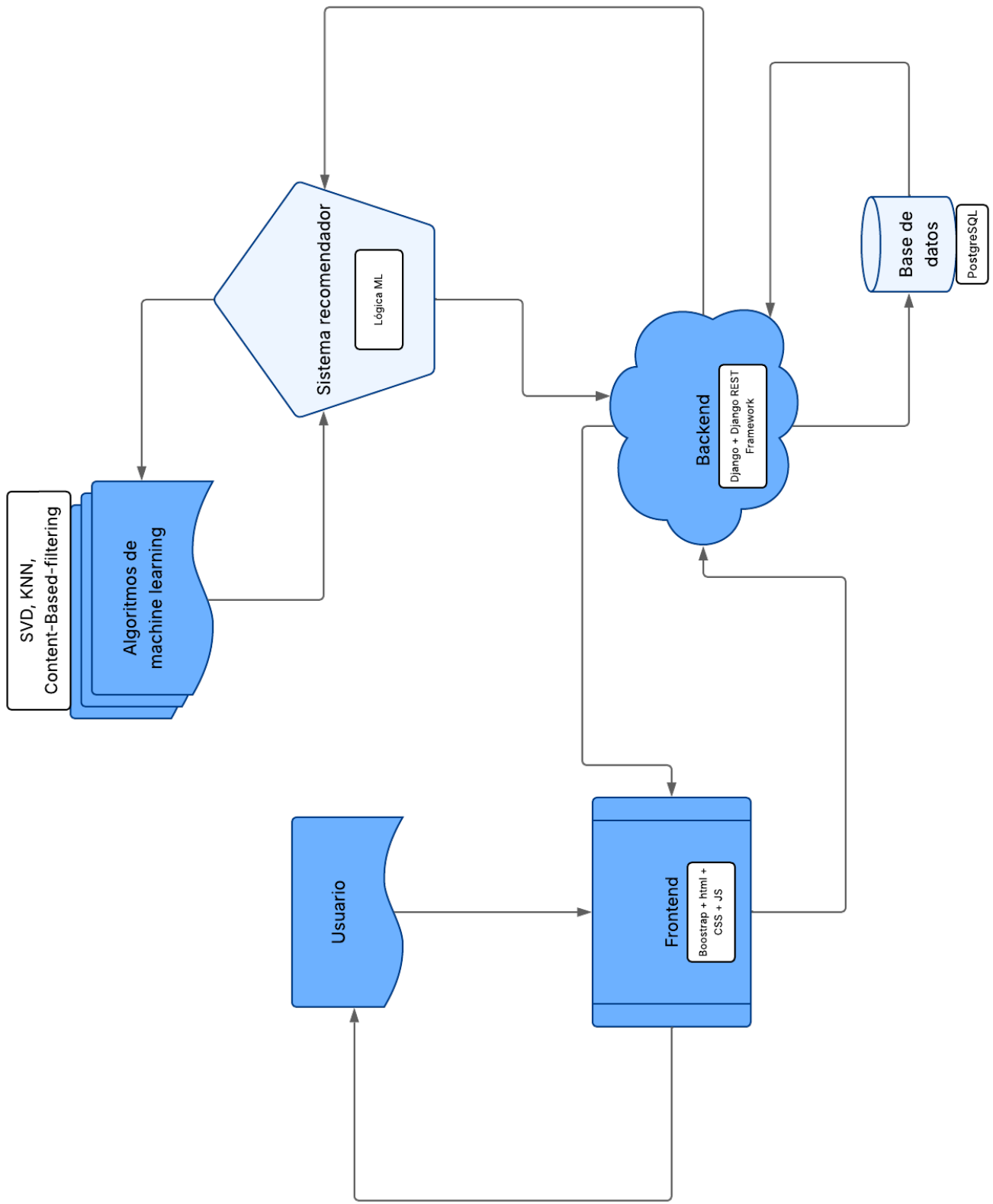


Figura 6.1: Esquema general de la arquitectura del sistema de recomendación.

6.3.2. Componentes del sistema

Frontend web

- Implementado mediante Django Templates y Bootstrap, junto con JavaScript para mejorar la interacción dinámica con el usuario.
- Permite iniciar sesión, cambiar la contraseña, ver el perfil, visualizar el catálogo de productos, filtrarlo, lanzar solicitudes de recomendación y consultar el dashboard con las diferentes recomendaciones de cada algoritmo usado junto con sus variables y métricas.

Backend

- Desarrollado íntegramente en Django 5.2 LTS, estructurado en *apps* especializadas: core, accounts, products, reviews y recommendations.
- Centraliza la lógica de negocio, expone servicios y APIs RESTful, y gestiona el ciclo de vida de los datos y la autenticación.
- Incorpora mecanismos de seguridad, validación y gestión de permisos, asegurando la integridad y privacidad de la información.

APIs REST

- Proporciona endpoints documentados como `/api/reviews/`, `/recommendations/recommend/` y otros para métricas.
- Facilita las integraciones con herramientas de terceros y la automatización mediante scripts.
- Permite ampliar la funcionalidad del sistema sin acoplar la lógica de presentación y datos.

Base de datos (BBDD)

- Sistema PostgreSQL seleccionado por su robustez y eficiencia en consultas relacionales complejas.
- Almacena entidades clave: usuarios, productos, categorías, valoraciones, registros de evaluación y parámetros de los algoritmos recomendadores.
- Optimizada mediante índices y relaciones para operaciones CRUD, análisis y consulta masiva de datos.

Módulos de Machine Learning y evaluación

- Scripts en Python para preprocesado, limpieza, entrenamiento y evaluación de modelos (scikit-learn, Surprise, pandas).
- Soporta múltiples enfoques: filtrado colaborativo, content-based, KNN e híbrido.
- El entrenamiento se realiza *offline*; los modelos se serializan (pickle) y se integran dinámicamente en el backend.

Sistema de administración y gestión

- Panel basado en Django Admin para gestión de datos, productos, usuarios y reviews.
- Scripts auxiliares para carga masiva, limpieza, mantenimiento y pruebas automatizadas.

Justificación de la arquitectura

- **Separación estricta de capas:** mantenibilidad, escalabilidad y testabilidad.
- **Django como framework central:** desarrollo ágil, excelente documentación, recursos y comunidad tanto en español como en inglés. Ofrece un ecosistema sólido para aplicaciones de inteligencia artificial y *machine learning* al estar basado en Python.
- **PostgreSQL:** adecuado para relaciones complejas y análisis intensivo de datos. Permite un despliegue rápido en local sin coste, ideal para entornos de bajo presupuesto.
- **REST APIs:** facilita la integración multiplataforma. Esta arquitectura podría implementarse en un *eCommerce* real con ajustes mínimos y es compatible con el crecimiento futuro del sistema.
- **ML desacoplado y evaluable:** permite y facilita la mejora continua sin afectar negativamente la experiencia del usuario.
- **Automatización y experimentación:** scripts de apoyo que permiten ciclos rápidos de validación y ajuste de algoritmos.

6.3.3. Modelo de datos y relaciones

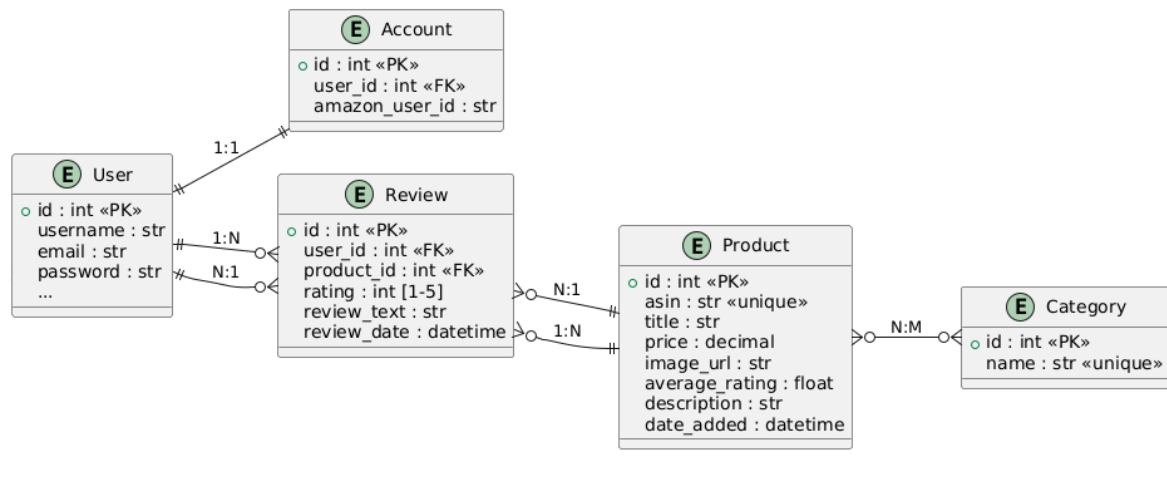


Figura 6.2: Modelo Entidad-Relación (MER) del sistema recomendador. Se muestran las entidades principales (*User*, *Account*, *Product*, *Category*, *Review*) y las relaciones entre ellas, permitiendo una gestión eficiente y flexible de usuarios, productos, valoraciones y métricas.

Como interpretar las relaciones

- Las líneas indican las relaciones entre entidades: 1:1 (uno a uno), 1:N (uno a muchos), N:M (muchos a muchos).
- Los campos marcados como «PK» son claves primarias, y «FK» indican claves foráneas.

Tablas principales

- **User:** basado en el modelo de Django y extendido para almacenar el identificador de usuario de *Amazon* mediante el campo `amazon_user_id`, esta extensión permite mapear y vincular usuarios externos (provenientes en este caso del datasets con ids reales de amazon) con los usuarios internos del sistema. Esto posibilita la importación masiva de valoraciones reales, la trazabilidad de datos a lo largo de todo el pipeline y abre la puerta para en un futuro permitir a usuarios reales de amazon iniciar sesión con su cuenta y sincronizar su historial y perfil con nuestra plataforma de forma sencilla.
- **Product:** cada producto se identifica de forma única mediante el código ASIN (Amazon Standard Identification Number), permitiendo la trazabilidad con el dataset y la posible integración y sincronización con sistemas externos en el futuro. Se almacenan metadatos enriquecidos como el título, la descripción completa, el precio actualizado, la URL de la imagen asociada y la media de valoraciones agregadas (`average_rating`), todo ello orientado a ofrecer una experiencia de usuario informativa y a facilitar la alimentación de los algoritmos

de recomendación. La relación ManyToMany con Category permite una categorización flexible, soportando tanto taxonomías jerárquicas como etiquetas múltiples por producto.

- **Category:** las categorías actúan como agrupadores lógicos y temáticos de los productos, permitiendo filtrar, organizar y presentar el catálogo de forma estructurada respetando la organización del dataset. Cada categoría está identificada por un nombre único y puede asociarse a múltiples productos, habilitando una navegación transversal y una mejor experiencia de búsqueda de los productos del catálogo.
- **Review:** la valoración es el eje de la interacción usuario-producto y constituye la principal fuente de datos para los algoritmos de recomendación. Cada review está asociada de manera única a un par (user, product), garantizada mediante la restricción `unique_together`. Incluye campos para puntuar el (rating, de 1 a 5 estrellas), un comentario en forma de texto (review_text) y la fecha de creación (review_date). El modelo está optimizado para consultas agregadas y permite el cálculo eficiente de estadísticas como medias, desviaciones y rankings por producto o usuario.

Motivación del diseño de datos

- La relación N:M entre productos y categorías ofrece gran flexibilidad.
- La entidad Review es fundamental para alimentar algoritmos colaborativos.
- El modelo es extensible para nuevos tipos de usuario, sincronización con plataformas externas, sobre todo Amazon cuyo diseño se ha tomado de referencia para desarrollar nuestro sistema. Permite fácilmente mejoras en el sistema de recomendación sin afectar ni al uso ni a otros componentes, permite la integración de nuevos algoritmos de machine learning y ampliación del catálogo de productos.

6.4. Funcionalidad del sistema

1. Autenticación y gestión de usuarios

El módulo de autenticación y gestión de usuarios es la puerta de entrada al sistema, garantizando la seguridad y la personalización de la experiencia.

Principales funcionalidades:

- **Registro de usuarios:** Implementado mediante formularios personalizados (`register.html` y lógica en `forms.py`, dentro de la app `accounts`), incorpora validación estricta de campos, control de unicidad de usuario/email y almacenamiento seguro de contraseñas gracias al sistema de *hashing* nativo de Django.
- **Login y logout:** El acceso se realiza con vistas propias y formularios que aprovechan el sistema de sesiones y autenticación robusto de Django, incluyendo protección ante ataques de fuerza bruta y control seguro de sesiones.
- **Recuperación de contraseña:** Se implementa el flujo estándar de Django (con plantillas `password_reset.html`, `password_reset_done.html`, etc.), validando *tokens* de reseteo y mostrando mensajes visuales para guiar al usuario durante el proceso.
- **Gestión de perfil:** Cada usuario puede acceder y modificar sus datos desde la vista `perfil.html`. El modelo `Account` amplía el modelo estándar `User` mediante una relación 1:1, permitiendo almacenar datos extra (por ejemplo, el `amazon_user_id`) de forma compatible y extensible.
- **Gestión de permisos y seguridad:** Todas las rutas sensibles están protegidas mediante decoradores `@login_required`. Se separan de forma clara los usuarios autenticados y anónimos, validando permisos tanto del lado del servidor como del cliente.

Detalles técnicos:

- Extensión de usuario con `Account` (relación 1:1).
- Uso de messages para *feedback* visual y comunicación de errores/éxitos.
- Plantillas específicas para cada flujo, facilitando una experiencia de usuario consistente y amigable.

2. Catálogo de productos

El catálogo de productos es el núcleo sobre el que se articulan las funcionalidades principales del sistema, permitiendo la consulta, filtrado y análisis de productos reales.

Principales funcionalidades:

- **Listado y filtrado de productos:** Mediante vistas basadas en clase y plantillas (`product_list.html`), el usuario puede explorar el catálogo con filtros avanzados (categoría, precio, valoración media). El *backend* optimiza las consultas usando `select_related` y `prefetch_related` para minimizar latencia.
- **Búsqueda textual:** Se ofrece búsqueda eficiente sobre el título y la descripción, con validación en el *backend* para evitar abusos o sobrecarga.
- **Detalle enriquecido:** Cada producto tiene una ficha detallada (`product_detail.html`) que incluye imagen, descripción, precio, valoración media, historial de *reviews* y recomendaciones relacionadas (generadas dinámicamente por el motor de ML).
- **Gestión y navegación por categorías:** Relación N:M entre productos y categorías, gestionada tanto en Django Admin como en el *frontend*. Cada categoría tiene su propia vista de listado.
- **Interfaz moderna y modular:** Componentes reutilizables (*navbar*, *loader*, tarjetas de producto) y paginación automática.

Detalles técnicos:

- Formularios de filtrado robustos y validados.
- Django Admin personalizado para gestión eficiente de productos/categorías.
- Gráficos analíticos (e.g., *rating* a lo largo del tiempo) integrados con librerías JavaScript como `Chart.js`.

3. Sistema de valoraciones

La gestión de valoraciones es el punto de unión entre usuarios y productos, y la base para el sistema de recomendaciones.

Principales funcionalidades:

- **Creación y edición de reviews:** Cada usuario autenticado puede valorar cada producto una única vez (`unique_together (user, product)` en el modelo). El formulario de review soporta edición y borrado, con validaciones *server-side* para integridad y rango de *rating*.
- **Visualización y gestión:** Listado de *reviews* en el detalle de producto, mostrando usuario, fecha, puntuación y comentarios. Acceso rápido al historial desde el perfil del usuario.

- **API REST de valoraciones:** Exposición de endpoints CRUD (/api/reviews/), implementados con Django REST Framework, protegidos por permisos y validados con serializers avanzados.
- **Optimización:** Consultas mediante `select_related` y `prefetch_related` para cargas masivas de datos.

Detalles técnicos:

- Modelo Review con campos: `user`, `product`, `rating`, `review_text`, `review_date`.
- Validación avanzada en serializers para evitar duplicidades y garantizar integridad.
- API documentada y con control de permisos fino.

4. Módulo de recomendaciones

El módulo de recomendaciones es el corazón de la inteligencia artificial aplicada al sistema, permitiendo sugerencias personalizadas y análisis avanzados.

Principales funcionalidades:

- **Selección y configuración de algoritmos:** El usuario puede escoger el tipo de recomendación (colaborativo, contenido, KNN, SVD, híbrido) y ajustar parámetros como k (vecinos) o α (peso híbrido) desde un formulario intuitivo.
- **Algoritmos integrados:** El motor utiliza scripts dedicados (`collaborative.py`, `content.py`, `knn.py`, `svd.py`, `hybrid.py`), desacoplados de la lógica de vistas. Modelos complejos (KNN, SVD) entrenados *offline* y serializados con `pickle`.
- **Presentación de resultados:** Las recomendaciones se muestran en tarjetas de producto, con *feedback* visual sobre el algoritmo y parámetros usados.
- **Entrenamiento, actualización y extensibilidad:** Modelos entrenados *offline* (`train_models.py`, `train_svd_model.py`), fácilmente actualizables y extensibles para añadir nuevos algoritmos o métricas en el futuro.
- **Evaluación y trazabilidad:** Incluye soporte para análisis experimental, trazabilidad de las recomendaciones y logging detallado.

Detalles técnicos:

- Integración completa con la base de datos para extracción/actualización de valoraciones.
- Uso de librerías líderes (`scikit-learn`, `surprise`, `pandas`) y validación rigurosa de parámetros de entrada.
- Modularidad que facilita el testeo y mejora iterativa del sistema.

6.5. Tecnologías utilizadas



PYTHON 3.13 LTS

Lenguaje principal

Se ha elegido para el proyecto como lenguaje principal **Python 3.13 LTS**, la versión más estable y optimizada de uno de los lenguajes de programación más extendidos en el ámbito del desarrollo web, la ciencia de datos y la inteligencia artificial. Python se caracteriza por su sintaxis limpia y legible, su gran comunidad, la disponibilidad de librerías especializadas y el soporte continuado para paradigmas tanto imperativos como orientados a objetos y funcionales. También ha sido uno de los lenguajes principales estudiados durante el grado, siendo ampliamente usado en asignaturas como FP o IA.

- **Compatibilidad y ecosistema:** Python es el lenguaje de referencia en machine learning y data science, lo que facilita la integración con librerías líderes como scikit-learn, pandas, surprise y herramientas de manipulación y persistencia de datos (pickle, numpy, etc.).
- **Desarrollo rápido y mantenible:** la simplicidad y expresividad de Python permiten una velocidad de desarrollo superior, minimizando la aparición de errores y reduciendo la curva de aprendizaje para nuevos colaboradores o futuros mantenimientos del sistema.
- **Características de Python 3.13 LTS:** esta versión introduce mejoras en la gestión de excepciones, un mayor rendimiento en equipos con recursos limitados y nuevas funcionalidades del intérprete que benefician tanto a la ejecución de scripts ETL como a la eficiencia de los procesos backend y de entrenamiento de modelos. Al ser de soporte a largo plazo (en inglés, Long Term Support, abreviadamente, LTS) está diseñado para tener soporte durante un período más largo que el normal, ofreciendo un equilibrio entre rendimiento y estabilidad superior a una versión *current*, que suelen incorporar cambios y nuevas características con mayor frecuencia y no están tan testeadas pudiendo tener más vulnerabilidades o errores que afecten la estabilidad del sistema.
- **Interoperabilidad:** Python ofrece una integración directa con sistemas externos (APIs REST, bases de datos relacionales y no relacionales, servicios de contenedores, etc.), lo que simplifica la futura ampliación del sistema y su despliegue en entornos de producción.



DJANGO 5.2 LTS

Framework principal

El sistema se apoya en **Django 5.2 LTS**, el framework de desarrollo web más robusto y completo en el ecosistema Python. Django proporciona una arquitectura modular que favorece la escalabilidad, el desarrollo ágil y la seguridad, integrando de forma nativa una amplia gama de funcionalidades esenciales para proyectos complejos como los sistemas recomendadores.

- **Estructura modular y reutilización:** Django permite organizar el código en aplicaciones independientes (django-apps), lo que facilita la separación de responsabilidades, el mantenimiento y la futura ampliación del sistema. De esta forma nos facilita el diseño del sistema siguiendo los principios **SOLID**, fundamentales en la ingeniería de software orientada a objetos:
 - **S — Single Responsibility Principle (SRP):** cada módulo o clase debe tener una única responsabilidad bien definida.
 - **O — Open/Closed Principle (OCP):** el software debe estar abierto para su extensión pero cerrado para su modificación.
 - **L — Liskov Substitution Principle (LSP):** los objetos de una clase derivada deben poder sustituir a los de su clase base sin alterar el correcto funcionamiento del programa.
 - **I — Interface Segregation Principle (ISP):** es preferible tener interfaces específicas y pequeñas en lugar de una interfaz general y extensa.
 - **D — Dependency Inversion Principle (DIP):** los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones.
- **ORM avanzado:** La capa de acceso a datos (Object-Relational Mapping) simplifica la gestión y consulta de la base de datos, garantizando integridad referencial, migraciones automáticas y eficiencia en las consultas complejas necesarias para el motor de recomendaciones.
- **Sistema de autenticación y permisos integrado:** Django proporciona herramientas robustas para la gestión de usuarios, autenticación, sesiones y permisos, minimizando el riesgo de vulnerabilidades y facilitando la implementación de flujos seguros.
- **Administración avanzada:** El panel de administración de Django permite gestionar de manera visual y eficiente los modelos principales del sistema

(usuarios, productos, reviews, categorías), acelerando la puesta en marcha y las pruebas.

- **Desarrollo ágil y testeo:** Django incorpora utilidades para el desarrollo rápido (plantillas, formularios, validadores), así como soporte integrado para testing automatizado y despliegue seguro.
- **Django REST Framework (DRF):** El sistema integra Django REST Framework para la exposición de APIs RESTful, posibilitando la integración con clientes externos, automatización de pruebas y ampliaciones de la funcionalidad añadiendo más módulos o microservicios.

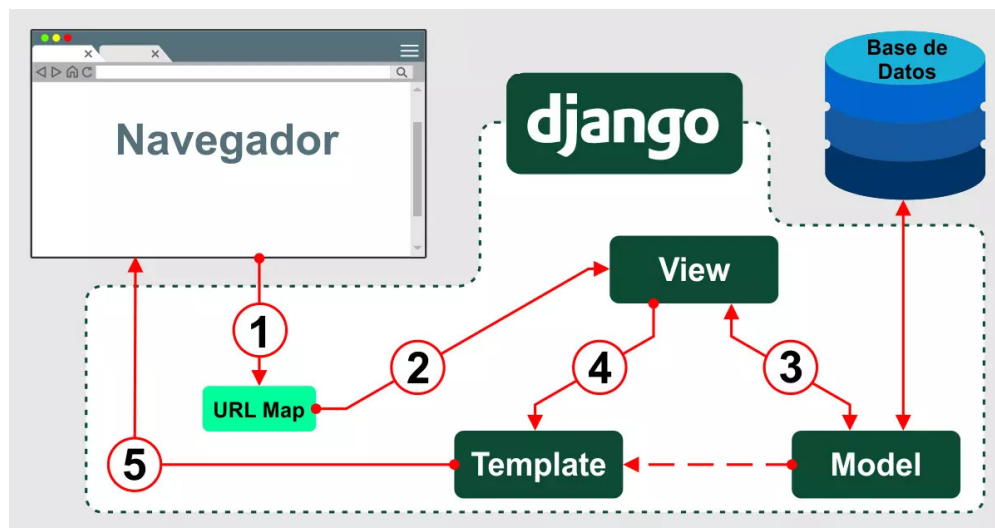


Figura 6.3: Arquitectura del flujo de datos en Django bajo el patrón MTV (Model-Template-View). Se ilustra cómo una petición del navegador atraviesa el mapeo de URL hacia una vista, la cual puede acceder al modelo y luego devolver una respuesta renderizada mediante una plantilla. Este flujo refleja la organización modular y coherente del framework.

El esquema anterior resume gráficamente el funcionamiento interno de Django al gestionar una petición web. El proceso comienza con el navegador solicitando una URL que es redirigida mediante el URL Map (1) hacia una View (2), la cual puede interactuar con los Models (3) para consultar o modificar datos en la base de datos. Luego, la vista prepara el contexto necesario y selecciona un Template (4) para construir la respuesta HTML. Finalmente, esta respuesta es enviada de vuelta al navegador (5). Este ciclo demuestra la integración eficiente de las distintas capas del framework y su alineación con el paradigma MTV, propio de django que no es más que una adaptación del clásico MVC (Modelo-Vista-Controlador).



BOOTSTRAP 5.3

Framework de desarrollo front-end

Para el desarrollo de la interfaz de usuario de la plataforma se ha optado por el framework **Bootstrap**, una de las herramientas más consolidadas y populares en el diseño web moderno. Bootstrap permite crear interfaces atractivas y funcionales utilizando tecnologías estándar como HTML, CSS y JavaScript, facilitando la construcción de sitios web que se adaptan de manera automática a cualquier tipo de dispositivo o tamaño de pantalla (diseño *responsive*).

- **Facilidad de uso y rápida integración:** Bootstrap destaca por su estructura intuitiva y la disponibilidad de componentes predefinidos, como menús, tarjetas, formularios y tablas, que han permitido acelerar el desarrollo de la interfaz y mantener un diseño coherente en todas las vistas de la plataforma. La curva de aprendizaje es baja comparado con otros frameworks de desarrollo front-end como **React** (Meta) o **Angular** (Alphabet). Personalmente cuento con la ventaja de haber trabajado bastante con el framework en asignaturas como *IISSI*², *IPO*³, o *SOS*⁴ por lo que ha sido una de las ventajas claras para decantarme por este framework en este proyecto.
- **Compatibilidad total entre navegadores:** Garantizar la accesibilidad y la correcta visualización del sistema en los principales navegadores (Chrome, Firefox, Edge, Safari, Opera, etc.) es crucial en un proyecto realista. Bootstrap se ha encargado de resolver los posibles problemas de compatibilidad, facilitando al desarrollador de ajustes manuales y pruebas repetitivas y poder centrarse en el desarrollo propiamente dicho.
- **Diseño responsive:** Facilita que la web se adapte automáticamente a a varios tamaños de pantalla, garantizando una experiencia fluida y uniforme al usuario.
- **Amplia comunidad y documentación:** Bootstrap dispone de una comunidad muy activa y una abundancia de recursos, tutoriales y ejemplos prácticos que facilitan la adopción de buenas prácticas de diseño web. Gracias a ello, ha resultado sencillo resolver cualquier duda o inconveniente durante el desarrollo, optimizando el tiempo dedicado a la resolución de cuestiones estéticas o de usabilidad. Cabe destacar que Bootstrap fue desarrollado originalmente por

²IISSI: Introducción a la Ingeniería del Software y los Sistemas de la Información

³IPO: Interacción Persona-Ordenador

⁴SOS: Sistemas Orientados a Servicios

Twitter, aunque en la actualidad se mantiene como un proyecto completamente *open source*, a diferencia de las otras alternativas, que siguen gestionadas directamente por sus creadores originales.

- **Rapidez de desarrollo:** El uso de componentes y utilidades preconfiguradas ha permitido centrar el esfuerzo en la lógica y funcionalidades clave del sistema, delegando en Bootstrap la maquetación y el aspecto visual. Esto ha reducido de forma significativa los plazos de desarrollo y ha incrementado la productividad global del proyecto.
- **Estandarización y mantenibilidad:** Apostar por un framework tan consolidado como Bootstrap garantiza la mantenibilidad a largo plazo de la interfaz. El código resultante es limpio, ordenado y fácil de escalar o modificar en futuras iteraciones del sistema. No impone patrones de lógica o arquitectura y se adapta al proyecto mediante clases y componentes fáciles de integrar.



Visual Estudio Code

Entorno de desarrollo integrado (IDE)

Para la escritura, edición y gestión del código fuente se ha optado por **VSCode**, un entorno de desarrollo integrado (IDE) ampliamente adoptado en los ámbitos profesional y académico en todo el mundo. VSCode destaca por su ligereza, su rápido tiempo de arranque y su compatibilidad con una gran variedad de lenguajes y tecnologías, lo que lo convierte en una opción especialmente adecuada para proyectos multidisciplinarios como el presente.

Entre sus principales ventajas se encuentran la extensa colección de extensiones disponibles, que han permitido integrar herramientas de lindeo, formateo automático, control de versiones (Git), gestión de entornos virtuales y depuración de código Python y Django de forma fluida y centralizada. Además, su interfaz intuitiva y personalizable, junto con la funcionalidad de IntelliSense y la integración con terminales y sistemas de control de versiones, han contribuido significativamente a la productividad y la calidad del desarrollo.



PostgreSQL

Base de datos

El sistema emplea **PostgreSQL** como motor de base de datos relacional, el cual ofrece características imprescindibles para este proyecto como son la integridad, eficiencia y capacidad de crecimiento ya que vamos a manejar un volumen muy alto de datos y realizaremos muchas transacciones para generar las recomendaciones.

- **Modelo relacional robusto:** PostgreSQL facilita el diseño de un esquema relacional coherente, estructurando entidades complejas (usuarios, productos, reviews, categorías) y gestionando eficientemente las relaciones N:M y restricciones de unicidad, fundamentales para la lógica de negocio del sistema.
- **Soporte avanzado para consultas:** Permite ejecutar consultas complejas y agregadas (por ejemplo, cálculo de medias, rankings, filtrado avanzado de productos) con un alto rendimiento, optimizando la respuesta en vistas como el catálogo o las recomendaciones personalizadas.
- **Integridad y transacciones:** Proporciona un sistema de transacciones completo y soporte de constraints avanzadas (primary key, foreign key, unique, check), garantizando la integridad referencial y previniendo inconsistencias durante la carga o edición masiva de datos.
- **Escalabilidad y extensibilidad:** PostgreSQL está preparado para escalar tanto en vertical como en horizontal y soporta ampliaciones mediante extensiones (arrays, funciones agregadas, JSON, índices avanzados) que pueden aprovecharse en futuras mejoras del sistema.
- **Compatibilidad y portabilidad:** Su integración nativa con el ORM de Django simplifica la gestión de migraciones, la portabilidad entre entornos (desarrollo, pruebas, producción) y la automatización de tareas administrativas y de backup.
- **Rendimiento probado en producción:** Es una de las opciones más fiables para proyectos que requieren manejar grandes volúmenes de datos históricos (como los datasets de reviews y productos de Amazon), permitiendo futuras integraciones con herramientas de análisis o sistemas de recomendación distribuidos.



MACHINE LEARNING

Librerías: scikit-learn, surprise, pandas, pickle

El sistema de recomendaciones se apoya en un conjunto de librerías líderes en el ecosistema Python para implementar, entrenar y validar los distintos algoritmos de machine learning y facilitar el procesamiento eficiente de grandes volúmenes de datos.

Se han elegido estas librerías por la necesidad de alinearse con el objetivo (Ver *Objetivos del proyecto*) de construir un sistema estable, eficiente y escalable, pero que a su vez pueda desarrollarse y ejecutarse en entornos locales de bajos recursos computacionales, como los equipos personales disponibles como se menciona en la sección *Costes materiales*. Desarrollar desde cero algoritmos complejos de recomendación y factorización de matrices supondría un coste en tiempo, memoria y procesamiento prohibitivo y dificultaría la evolución ágil del proyecto, especialmente a la hora de iterar, depurar y comparar resultados.

- **scikit-learn:** Librería de referencia para machine learning clásico en Python. Se utiliza para la implementación de modelos de filtrado basado en contenido (content-based), K-Nearest Neighbors (KNN) y para operaciones de procesamiento de características y cálculo de similitud (TF-IDF, coseno, normalización, etc.). Proporciona utilidades robustas para validación cruzada, selección de parámetros y preprocesamiento, todo ello con una gestión de memoria optimizada y una curva de aprendizaje baja.
- **surprise:** Librería especializada en sistemas de recomendación colaborativos. Permite construir y entrenar modelos avanzados como SVD (Singular Value Decomposition), gestionar evaluaciones offline y comparar el rendimiento de diferentes enfoques. Su alto grado de optimización hace viable el trabajo experimental con datasets reales en equipos modestos.
- **pandas:** Herramienta esencial para la manipulación y análisis de datos tabulares. Utilizada en todo el pipeline ETL para cargar, limpiar, filtrar y transformar los datasets (por ejemplo, selección de top usuarios/productos, muestreo, eliminación de duplicados), así como en la evaluación y análisis estadístico de los resultados.

- **pickle**: Módulo estándar de Python para la serialización y deserialización de objetos complejos. Permite guardar los modelos entrenados de machine learning en disco y cargarlos en memoria en tiempo real, optimizando el rendimiento del sistema y reduciendo la latencia de las predicciones en producción.



Control del tiempo

El control y seguimiento del tiempo es una práctica fundamental en la ingeniería del software, ya que permite cuantificar de manera objetiva el esfuerzo dedicado a cada tarea y facilita la gestión de la planificación global del proyecto. Inicialmente, se realiza una estimación de la duración de todas las tareas en la fase de planificación, lo que permite establecer una fecha objetivo de finalización. Sin embargo, los retrasos en la ejecución de tareas individuales pueden repercutir directamente en el cumplimiento de los plazos globales, por lo que resulta esencial registrar tanto la duración estimada como el tiempo real invertido en cada actividad.

Entre sus funcionalidades más destacadas, cabe resaltar la posibilidad de utilizar extensiones para navegador, lo que agiliza la monitorización del tiempo en entornos de desarrollo variados y fomenta una gestión eficiente de los recursos. En este proyecto, Clockify ha permitido identificar desviaciones sobre la planificación inicial y ha aportado datos objetivos para la mejora de la estimación y la gestión del tiempo .



Control de versiones

Para la gestión eficiente del código fuente, la colaboración y el seguimiento de la evolución del proyecto, se ha utilizado el sistema de control de versiones **Git**, junto con un repositorio remoto abierto en **GitHub**: https://github.com/antoniosilva096/tfg_recommender_system

- **Histórico y trazabilidad:** Git permite mantener un registro detallado de todos los cambios realizados en el código, facilitando la identificación, reversión y análisis de cualquier modificación, así como la restauración de versiones estables en caso de errores.
- **Colaboración y control de calidad:** GitHub aporta una interfaz visual para la gestión de ramas, revisiones de código (pull requests) y resolución de conflictos, así como integración con herramientas de automatización (CI/CD), seguimiento de incidencias y documentación colaborativa.
- **Automatización y backup:** La integración con GitHub garantiza la existencia de copias de seguridad remotas y permite automatizar tareas como la generación de documentación, la ejecución de tests o el despliegue en entornos controlados.

***Nota sobre el repositorio:** El repositorio actual es una reconstrucción limpia del proyecto original. Durante el desarrollo inicial, no se aislaron correctamente mediante `.gitignore` los archivos de entrenamiento de modelos, que son especialmente pesados y llegaron a corromper el historial, dificultando la sincronización y comprometiendo la seguridad del progreso local. Por este motivo, se optó por migrar todo el código y la estructura a un nuevo repositorio, asegurando una correcta gestión de archivos grandes y una mayor estabilidad para futuras evoluciones.

6.6. Consideraciones sobre calidad, mantenimiento y evolución futura

Durante el proceso de diseño se ha priorizado no solo la funcionalidad, sino la estabilidad, rendimiento y la mantenibilidad a largo plazo. El sistema ha sido estructurado de forma modular, aplicando buenas prácticas de desarrollo como la separación de responsabilidades, el uso de patrones de diseños apoyados en las facilidades que nos presenta Django para ello.

Por último, la arquitectura propuesta favorece la escalabilidad y la incorporación de nuevas funcionalidades o algoritmos, permitiendo que el sistema se adapte a futuras necesidades o a cambios en los requisitos, alineándose con los principios de ingeniería de software estudiados durante el grado.

6.7. Flujo de procesos

La Figura 6.4 muestra el flujo de procesos completo del sistema recomendador, desde la ingestión de datos hasta la presentación de recomendaciones personalizadas en la interfaz web.

En primer lugar, el sistema parte de la **extracción de datos** masivos desde fuentes externas (como los datasets públicos de valoraciones de Amazon). Estos datos son procesados mediante **scripts ETL** en Python, donde se filtran, limpian y transforman para ajustarse al modelo de datos definido en el sistema. Una vez validados, los datos son cargados en la base de datos PostgreSQL, quedando disponibles para todas las operaciones del sistema.

Posteriormente, los distintos módulos de **machine learning** entrenan los modelos de recomendación (colaborativo, contenido, híbrido, etc.) utilizando los datos almacenados. Los modelos se serializan y se mantienen listos para ser utilizados en producción, optimizando el tiempo de respuesta y el uso de recursos.

Cuando un usuario accede a la aplicación web, el **backend Django** gestiona la autenticación y la navegación por el catálogo de productos, permitiendo la valoración y consulta de información enriquecida. Cuando el usuario solicita recomendaciones, el sistema selecciona el algoritmo configurado y recupera (o calcula en tiempo real) las sugerencias personalizadas utilizando tanto los modelos entrenados como los datos frescos de la base de datos.

Finalmente, las recomendaciones generadas se presentan en la interfaz de usuario de forma clara, permitiendo al usuario explorar e interactuar los modelos.

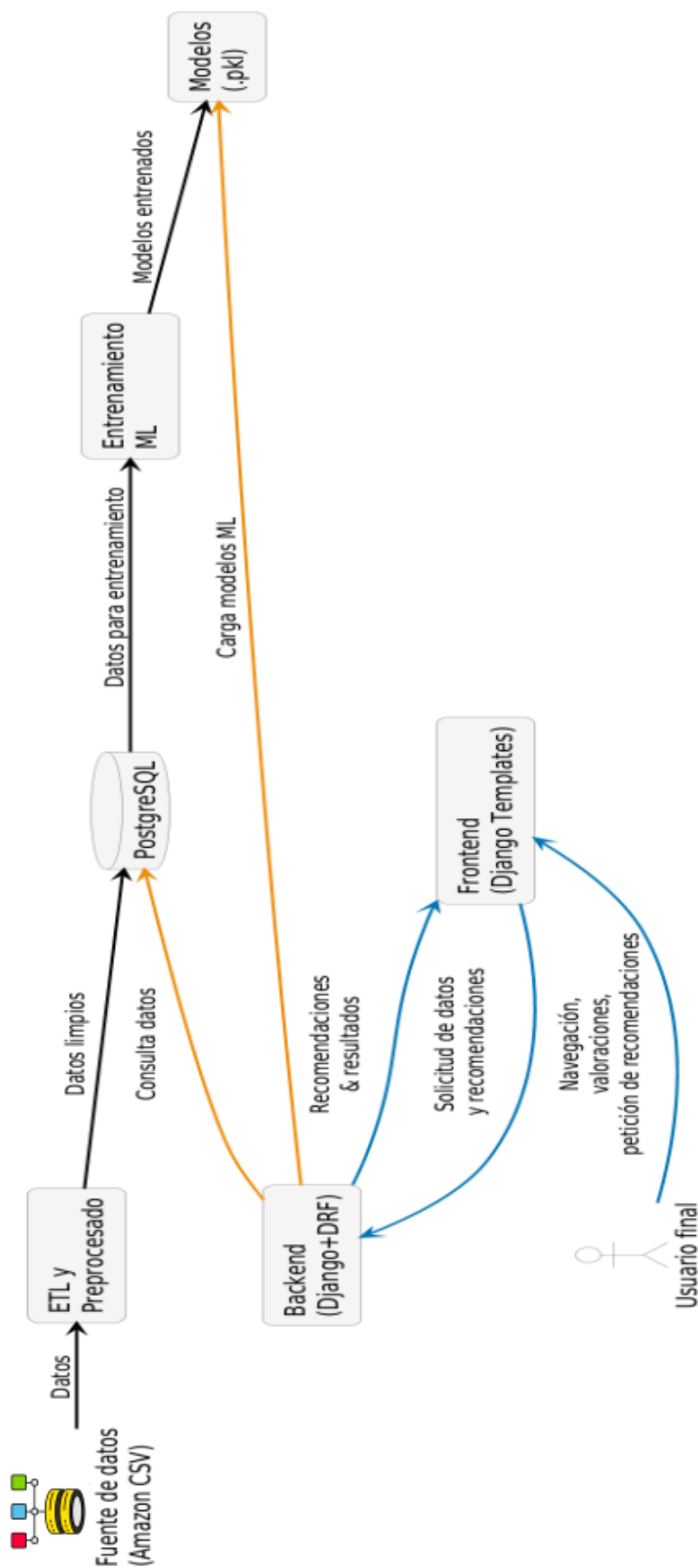


Figura 6.4: Flujo completo de procesos del sistema recomendador. El diagrama ilustra, de izquierda a derecha, todas las etapas del sistema: desde la ingestión y limpieza y limpieza de datos externos, el almacenamiento y entrenamiento de modelos, hasta la entrega de recomendaciones al usuario final a través del backend y la interfaz web. Cada módulo está desacoplado para facilitar el mantenimiento, la trazabilidad y la escalabilidad futura.

7. Ejecución

Este capítulo detalla exhaustivamente el proceso de desarrollo de un sistema recomendador integrado en una plataforma web. Dicha plataforma actúa como un entorno de pruebas (*playground*), permitiendo experimentar y evaluar distintos algoritmos de recomendación aplicados a un catálogo real de productos de Amazon. La interfaz simula las funcionalidades propias de un comercio electrónico, incorporando un catálogo navegable y filtros avanzados, lo que permite validar el sistema en un contexto similar al real.

A lo largo del capítulo se presentan las distintas fases del desarrollo, las tecnologías empleadas en cada etapa y las medidas adoptadas para garantizar la calidad y coherencia del sistema, siempre alineadas con los objetivos y requisitos previamente establecidos.

El desarrollo se ha organizado en cinco *sprints*, siguiendo una metodología ágil de tipo Scrum. Esta estrategia ha facilitado una evolución iterativa del sistema, permitiendo validaciones y ajustes incrementales en función de los resultados obtenidos. Cada *sprint* aborda un conjunto específico de funcionalidades y culmina con la entrega de componentes verificables.

Para una mejor comprensión, el capítulo está estructurado según los distintos *sprints*, reflejando el flujo de trabajo real seguido durante la ejecución del proyecto.

7.1. Sprint 1: Proceso ETL

En este proyecto y, en general, en cualquier solución basada en ciencia de datos, el proceso **ETL** (siglas en inglés de *Extract, Transform, Load*) constituye una etapa fundamental para garantizar la calidad y la fiabilidad de los resultados obtenidos.

El proceso ETL abarca tres fases principales:

- **Extracción (Extract):** consiste en la obtención de datos brutos desde diversas fuentes externas, que pueden ser ficheros CSV, bases de datos, APIs públicas o repositorios online.
- **Transformación (Transform):** en esta etapa, los datos extraídos se limpian, filtran, normalizan y estructuran para adaptarse al modelo lógico y funcional del sistema. Aquí se resuelven inconsistencias, se eliminan duplicados y se ajustan formatos, garantizando así la coherencia y utilidad de la información.
- **Carga (Load):** finalmente, los datos transformados se cargan en la base de datos o sistema de almacenamiento principal, quedando listos para su explotación por los algoritmos de recomendación y las distintas funcionalidades de la plataforma.

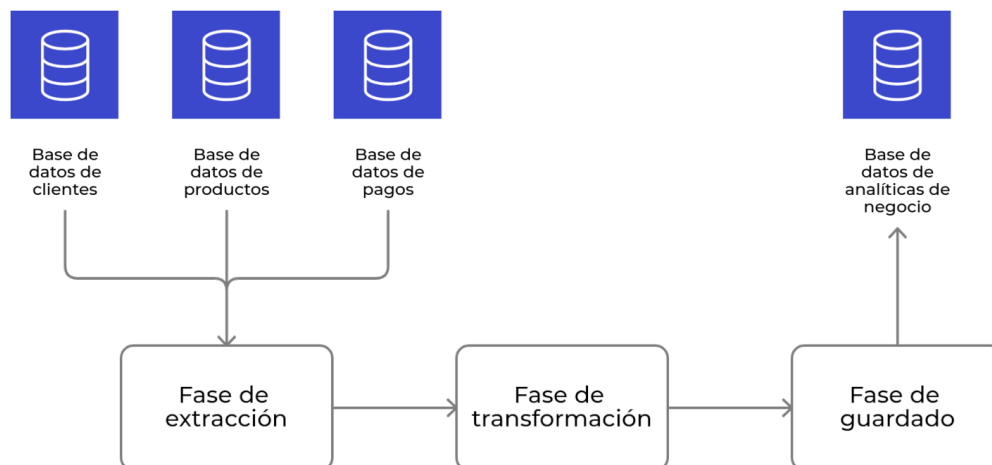


Diagrama de un proceso ETL

La correcta ejecución del pipeline ETL es esencial para evitar problemas derivados de datos erróneos, incompletos o mal estructurados, que pueden conducir a recomendaciones de baja calidad o incluso a errores críticos en el sistema. Un buen proceso ETL no solo mejora la precisión de los algoritmos, sino que también facilita la reproducibilidad, la trazabilidad y la escalabilidad de la solución desarrollada.

1. Objetivos:

El objetivo principal de este primer sprint fue la preparación del entorno de desarrollo y la implementación de un flujo ETL eficiente, capaz de transformar los datos crudos del dataset Amazon Reviews 2023 (Electronics) en un conjunto limpio y estructurado, listo para ser integrado en el sistema.

2. Dataset utilizado:

El sistema se nutre de datos provenientes del dataset *Amazon Reviews 2023 – Electronics*, alojado en la plataforma Hugging Face y publicado por McAuley-Lab. Este dataset contiene metainformación sobre productos, reseñas, valoraciones, precios e imágenes asociadas.

Con el fin de reducir la complejidad del conjunto de datos y eliminar inconsistencias, se diseñó e implementó un script en Python que automatiza el proceso de obtención de los datos y preprocesado. Este script hace uso de bibliotecas como *datasets*, *pandas* y *nlTK*, y realiza un procesamiento en modo *streaming* para evitar desbordamientos de memoria. Se tomó la decisión de trabajar exclusivamente con productos pertenecientes a la *main category* Electronics. Esta elección se justifica por la necesidad de homogeneidad en los datos, esencial tanto para alimentar el sistema como para entrenar los modelos. Al centrarse en una

categoría específica, se consigue un equilibrio entre el volumen de datos y la manejabilidad, asegurando una base suficientemente amplia pero coherente para el procesamiento y análisis posterior.

En la Figura 7.1 se muestra un resumen de las principales características del dataset, extraído directamente de su página oficial¹. Esta información proporciona una visión global del volumen y la estructura de los datos empleados en el desarrollo del sistema recomendador.

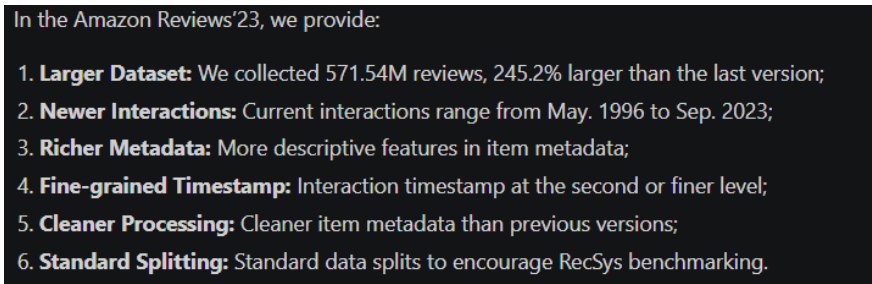


Figura 7.1: Resumen de características del dataset Amazon Reviews 2023, obtenido de su página oficial.

3. Implementación del pipeline ETL

Se desarrolló un script en Python que automatiza las tres fases del ETL:

- Extracción

Los datos se descargan directamente desde HuggingFace mediante

```
1 datasets.load_dataset(streaming=True)
```

. Esto permite procesarlos por lotes sin cargarlos completamente en memoria.

- Transformación

Se aplicaron varias operaciones para limpiar y normalizar los datos:

- Eliminación de registros incompletos (sin título, categoría, precio o rating).
- Limpieza textual usando expresiones regulares y `nltk`.
- Reestructuración de categorías jerárquicas como cadenas delimitadas.
- Conversión de campos numéricos a tipo `float`.
- Selección de la mejor URL de imagen disponible.
- Asignación de identificador único (`parent_asin` o `asin`).

¹<https://huggingface.co/datasets/McAuley-Lab/Amazon-Reviews-2023>

- Carga (persistencia intermedia)

Como paso previo a la carga final en Django, los registros procesados se exportaron a un archivo CSV (`products_clean.csv`) con un máximo de 150 000 entradas válidas. Este archivo actúa como forma de persistencia intermedia entre el preprocesado y la integración posterior en los modelos del sistema.

Ventajas de este enfoque

Este enfoque modular y ligero permite:

- Ejecutar el preprocesado incluso en equipos con recursos limitados.
- Dividir la carga de trabajo del sistema, facilitando el desarrollo posterior.
- Restablecer el catálogo con un comando de administración reutilizando el CSV.
- Reproducir el experimento con datos estables e inmutables.

Exportación del dataset procesado

El resultado final se exportó como un archivo `products_clean.csv` alojado en el directorio Data del proyecto con un máximo de 150.000 registros válidos. Este archivo fue posteriormente utilizado como fuente para la carga en los modelos Django del sistema.

```
1 asin,title,categories,price,average_rating,image_url
2 B00XYZ123,Logitech M185,Electronics > Mice,13.99,4.3,http://...
3 ...
```

Recorrido de las etapas de extracción y transformación de los datos

Fase	Módulo/Función	Descripción
Extracción	<code>datasets.load_dataset</code> + <code>streaming=True</code>	Lee datos raw desde la nube sin cargar todo en RAM
Transform.	<code>clean_text</code> , <code>get_main_image_url</code> , <code>transform</code>	Limpia texto, filtra <i>stopwords</i> , convierte tipos, empaqueta registro listo para CSV
Persistencia	<code>csv.DictWriter</code>	Persiste registros en disco hasta límite configurado

Tabla 7.1: Resumen de las fases del proceso ETL implementado

7.2. Sprint 2 – Configuración del entorno y modelado de datos

1. Objetivos:

- Instalar y configurar el entorno de desarrollo web.
- Definir la arquitectura inicial del proyecto en Django.
- Modelar los módulos principales según los requisitos extraídos.

2. Configuración del entorno de desarrollo:

El desarrollo se realizó en un equipo con Windows 11. Se instalaron todas las dependencias necesarias (Python, Django, PostgreSQL, etc.), creando un entorno virtual con venv para aislar el proyecto. Se configuró la conexión con la base de datos PostgreSQL en local y se habilitó el manejo de archivos estáticos, rutas, recursos multimedia y la configuración regional (idioma español y zona horaria de Madrid). El entorno se ejecutó en modo desarrollo con soporte para depuración y recarga automática de cambios.

3. Definición de la arquitectura y estructura modular:

Se definieron las siguientes aplicaciones dentro del proyecto con la configuración mínima.

Las responsabilidades principales de cada módulo en el sistema diseñado son:

- core: navegación general, base de plantillas y menú dinámico.
- accounts: autenticación, registro y gestión de perfiles.
- products: gestión de productos, categorías y vistas asociadas.
- recommendations: lógica de recomendación y entrenamiento de modelos.
- reviews: almacenamiento de valoraciones de usuarios.

Cada aplicación fue registrada en settings.py y configurada con sus propios modelos, rutas, vistas y plantillas.

Nombre	Fecha de modificación	Tipo	Tamaño
.git	17/05/2025 9:05	Carpeta de archivos	
accounts	17/05/2025 9:05	Carpeta de archivos	
BD	16/05/2025 9:00	Carpeta de archivos	
core	17/05/2025 9:05	Carpeta de archivos	
data	26/05/2025 20:40	Carpeta de archivos	
ecommerce_rs	17/05/2025 10:38	Carpeta de archivos	
env	17/05/2025 10:33	Carpeta de archivos	
logs	17/05/2025 11:24	Carpeta de archivos	
products	17/05/2025 12:09	Carpeta de archivos	
recommendations	17/05/2025 9:05	Carpeta de archivos	
reviews	17/05/2025 9:05	Carpeta de archivos	
venv	25/05/2025 13:11	Carpeta de archivos	
.gitignore	16/05/2025 8:59	Archivo de origen ...	1 KB
exclude.txt	16/05/2025 8:58	Archivo TXT	1 KB
manage.py	01/02/2025 19:21	Python File	1 KB
requirements.txt	17/05/2025 9:06	Archivo TXT	1 KB
svd_model.pkl	15/05/2025 15:44	Archivo PKL	83.273 KB
tfg_ecommerce_rs.code-workspace	28/02/2025 21:12	Archivo de origen ...	1 KB

Figura 7.2: Estructura general del proyecto

```
1 python manage.py startapp nombre_app
```

Extracto de código 7.1: Comando para crear una nueva aplicación en Django

```
WSGI_APPLICATION = 'ecommerce_rs.wsgi.application'

# Database
# https://docs.djangoproject.com/en/5.1/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'ecommerce_db',
        'USER': 'postgres',
        'PASSWORD': 'postgres',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

Figura 7.3: Parámetros de conexión a la base de datos PostgreSQL en settings.py.

```

DEBUG = True

ALLOWED_HOSTS = []

# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # Apps del proyecto
    'core',
    'accounts',
    'products',
    'reviews',
    'recommendations',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'ecommerce_rs.urls'

```

Figura 7.4: Configuración de aplicaciones instaladas y middlewares en `settings.py`.

```

# Internationalization
# https://docs.djangoproject.com/en/5.1/topics/i18n/

# Configurar el idioma en español de España
LANGUAGE_CODE = 'es-es'

# Configurar la zona horaria de España (Madrid)
TIME_ZONE = 'Europe/Madrid'

# Habilitar internacionalización
USE_I18N = True

# Habilitar uso de zona horaria
USE_TZ = True

# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/5.1/howto/static-files/

STATIC_URL = '/static/'

STATICFILES_DIRS = [
    os.path.join(BASE_DIR, 'core', 'static'),
]
MEDIA_URL = '/media/'
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')

```

Figura 7.5: Configuración regional, idioma y archivos estáticos en `settings.py`.

4. Creación del usuario administrador:

Para gestionar el sistema y acceder al panel de administración de Django, se creó un usuario administrador específico. El nombre de usuario asignado se ha basado en el usuario virtual de la U.S. (uvus) **antsilgor**, facilitando su trazabilidad y asociación directa. La contraseña utilizada durante la fase de desarrollo fue 123456etsii, como estamos en un entorno de desarrollo es más ágil una contraseña que nos permita testear longitudes y caracteres reales que pueda contener una clave, pero que sea sencilla de recordar y rápida de escribir para que no interrumpa el flujo de desarrollo.

El usuario administrador se creó utilizando el siguiente comando en la terminal del proyecto:

```
1 python manage.py createsuperuser --username antsilgor --email
  antsilgor@alum.us.es
```

Extracto de código 7.2: Comando para crear el usuario administrador en Django

Durante el proceso interactivo, se estableció la contraseña mencionada para este usuario. Posteriormente, este acceso permitió la configuración inicial del sistema, la gestión avanzada de usuarios, productos y valoraciones, así como la supervisión y administración de la plataforma a través del panel de Django Admin.

5. Levantamiento del servidor de desarrollo y pruebas:

Una vez completada la configuración básica del proyecto y las aplicaciones, se procedió al primer arranque del servidor de desarrollo mediante el comando `python manage.py runserver`. Como se muestra en el siguiente fragmento, el entorno respondió correctamente, indicando la ausencia de errores y la disponibilidad del servicio en la URL por defecto.

Al acceder a la dirección local en el navegador, Django presenta su pantalla de bienvenida (Figura 7.6), confirmando que el proyecto está correctamente configurado, que la conexión con la base de datos es correcta y que está preparado para comenzar a desarrollar el back-end.

```
1 python manage.py runserver
2 Watching for file changes with StatReloader
3 Performing system checks...
4
5 System check identified no issues (0 silenced).
6 March 28, 2025 - 18:17:23
7 Django version 5.2, using settings 'ecommerce_rs.settings'
8 Starting development server at http://127.0.0.1:8000/
9 Quit the server with CONTROL-C.
```

Extracto de código 7.3: Vista de la consola del backend al iniciar el servidor

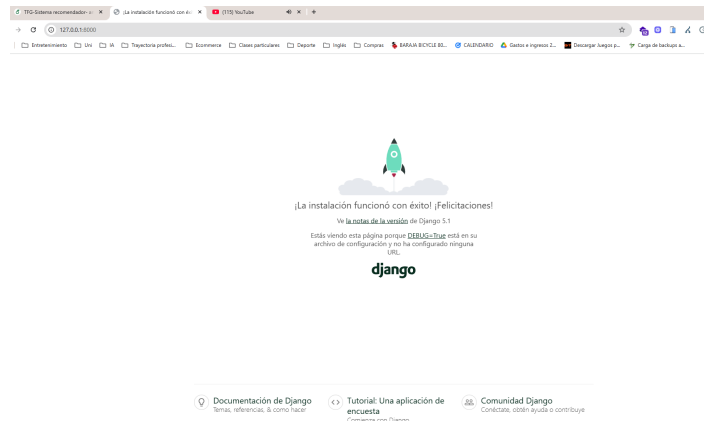


Figura 7.6: Pantalla de bienvenida de Django tras el primer arranque del servidor.

7.3. Sprint 3: Codificación del back-end e integración del módulo de recomendación

El tercer sprint representa la fase central de desarrollo, en la que se realiza la implementación del **back-end** de la plataforma y la integración del módulo de recomendaciones. Ha requerido el diseño, codificación y puesta a punto de los componentes lógicos que gestionan tanto la lógica de negocio del sistema como la interacción con los algoritmos de *machine learning* previamente entrenados y serializados también durante esta etapa.

1. Objetivos:

- Desarrollar la estructura y lógica principal del back-end sobre el framework Django, asegurando la modularidad y mantenibilidad del código.
- Implementar los modelos de datos principales (productos, usuarios, reviews).
- Integrar los pipelines de carga masiva y validación de datos reales.
- Automatizar la importación de reseñas y garantizar la coherencia e integridad de las valoraciones.
- Exponer la API RESTful para la gestión y consulta de productos, usuarios, reviews y recomendaciones.
- Realizar pruebas funcionales de los endpoints y procesos principales del sistema.

2. Desarrollo e integración del back-end

Durante este sprint se abordó la implementación de la lógica de negocio del sistema, distribuyéndola en aplicaciones específicas (core, products, accounts, reviews, recommendations), cada una orientada a una funcionalidad concreta. El

uso de Django permitió definir modelos robustos, gestionar la autenticación y autorización, y configurar los permisos y validaciones necesarias tanto en el back-end como en los formularios del front-end.

La integración del motor de recomendaciones fue especialmente relevante. Para ello, se diseñó un módulo dedicado que encapsula la lógica de selección y ejecución de los distintos algoritmos (colaborativo, basado en contenido, KNN, SVD e híbrido). Este diseño desacoplado permite cargar los modelos entrenados bajo demanda mediante pickle, optimizando el rendimiento y facilitando la extensibilidad futura.

Para garantizar la interoperabilidad y facilitar futuras integraciones, se expusieron endpoints RESTful usando Django REST Framework. Esta API cubre todas las operaciones CRUD sobre productos, usuarios, valoraciones y obtención de recomendaciones, y está documentada para su consumo por clientes externos o aplicaciones móviles, así como para la automatización de pruebas.

2.1 Flujo de trabajo y estructura de desarrollo en Django

El desarrollo siguió las mejores prácticas de Django, empleando un flujo estructurado que facilita la escalabilidad y el mantenimiento:

- **Modelos y ORM:** Los modelos de Django representan las entidades y relaciones principales del sistema, implementados como clases Python sobre el ORM nativo. Esto permite gestionar operaciones de creación, actualización, consulta y borrado de registros (CRUD) de forma eficiente y segura, evitando la necesidad de escribir SQL manualmente.

Modelos principales. A continuación, se presentan los modelos principales implementados en Django, base de la estructura de datos y del sistema de recomendación.

Category. Define las categorías a las que pertenece cada producto. El campo name es único, lo que evita duplicados y facilita la clasificación y el filtrado de productos.

```
1 class Category(models.Model):
2     name = models.CharField(max_length=255, unique=True)
3
4     def __str__(self):
5         return self.name
```

Extracto de código 7.4: Modelo Category

Cada categoría puede estar asociada a múltiples productos.

Product. Modelo central que representa un producto real del catálogo, identificable por su ASIN (Amazon Standard Identification Number). Incluye referencias a una o varias categorías y atributos clave como el título, precio, rating promedio e imagen.

```
1 class Product(models.Model):
2     asin = models.CharField(max_length=50, unique=True)
3     title = models.TextField()
4     categories = models.ManyToManyField(Category, related_name="products"
5     )
6     price = models.FloatField()
7     average_rating = models.FloatField(null=True, blank=True)
8     image_url = models.URLField(blank=True, null=True)
9
10    def __str__(self):
11        return self.title
```

Extracto de código 7.5: Modelo Product

El uso de una relación ManyToMany permite que un producto pertenezca a varias categorías, reflejando la complejidad real de los catálogos comerciales.

Account. Extiende el modelo de usuario de Django con un identificador externo (por ejemplo, el `amazon_user_id` del dataset). Esto permite importar usuarios masivamente y asociar cuentas a cada *review* real.

```
1 class Account(models.Model):
2     user = models.OneToOneField(User, on_delete=models.CASCADE)
3     amazon_user_id = models.CharField(max_length=255, unique=True, null=
4     True, blank=True)
5
6     def __str__(self):
7         return self.user.username
```

Extracto de código 7.6: Modelo Account

La relación `OneToOne` garantiza una correspondencia exacta entre el usuario del sistema y el usuario original de Amazon.

Review. Representa la valoración de un producto por parte de un usuario. Incluye calificación, texto opcional y fecha. La restricción `unique_together` asegura que cada usuario solo pueda dejar una valoración por producto, evitando duplicados y manteniendo la integridad de la base de datos.

```
1 class Review(models.Model):
2     user = models.ForeignKey(
3         User,
4         on_delete=models.CASCADE,
5         related_name='reviews'
6     )
7     product = models.ForeignKey(
8         Product,
9         on_delete=models.CASCADE,
10        related_name='reviews'
11    )
12    rating = models.PositiveSmallIntegerField(help_text="Calificacion de
13    1 a 5")
14    review_text = models.TextField(blank=True, null=True)
15    review_date = models.DateTimeField(auto_now_add=True)
16
17    class Meta:
18        ordering = ['-review_date']
19        unique_together = ('user', 'product')
20
21    def __str__(self):
22        return f"{self.user.username} - {self.product.name} ({self.rating
23        })"
```

Extracto de código 7.7: Modelo Review

El uso de claves foráneas y restricciones de unicidad es fundamental para el correcto funcionamiento de los algoritmos de recomendación y el mantenimiento de la base de datos.

- **Vistas basadas en funciones y en clases:** La lógica de negocio y el procesamiento de peticiones se implementan a través de **vistas**, que pueden ser:

- **Vistas basadas en funciones (FBV, Function-Based Views):** son funciones Python que reciben una petición (request) y devuelven una respuesta (HttpResponse). Permiten un control granular sobre el flujo de la petición y resultan especialmente útiles para vistas simples o personalizadas.
- **Vistas basadas en clases (CBV, Class-Based Views):** Django proporciona una jerarquía de clases base que encapsulan patrones comunes (listado, detalle, creación, edición, borrado, etc.), facilitando la reutilización de código y la extensión de funcionalidades mediante herencia y mezcla de *mixins*. Las CBV permiten una organización más modular y orientada a objetos del código.

En este proyecto se han empleado ambos enfoques, seleccionando el más adecuado para cada caso en función de la complejidad y los requisitos de la funcionalidad implementada.

- **Enrutamiento y conexión de vistas:** La conexión entre las peticiones HTTP del usuario y la lógica de negocio se realiza mediante el sistema de **rutas** o **URLs**. Cada aplicación define un archivo `urls.py` donde se asocian patrones de URL a vistas concretas (ya sean funciones o clases). Este mecanismo permite organizar la estructura del sitio web, establecer nombres semánticos para las rutas y facilitar la gestión de los permisos y la navegación interna. Las rutas se pueden parametrizar para capturar identificadores o aplicar filtros, habilitando una navegación dinámica y eficiente.

Endpoints principales de navegación. La aplicación define una serie de rutas clave para interactuar con la plataforma y acceder a sus funcionalidades principales. Estos endpoints permiten tanto la navegación por productos como la consulta de recomendaciones y resultados de evaluación:

```
1 urlpatterns = [  
2     path("recommend/<int:user_id>/", recommend_products, name="  
3     recommend_products"),  
4     path('dashboard/', evaluation_dashboard, name='evaluation_dashboard')  
5 ],  
6     path('product/<str:asin>/', recommend_product_page, name='  
7     recommend_product'),  
8 ]
```

Extracto de código 7.8: Rutas principales de la plataforma

- `recommend/<int:user_id>/`: genera recomendaciones personalizadas para un usuario específico.
- `dashboard/`: muestra un panel de evaluación con los resultados de los distintos algoritmos de recomendación.
- `product/<str:asin>/`: despliega la página de detalles de un producto y sus recomendaciones asociadas.

• **Templates:** Aunque el núcleo de la presentación se desarrolla en las plantillas (*templates*), el diseño modular de Django asegura que la lógica de negocio y los datos permanezcan desacoplados de la capa visual, siguiendo el patrón *Model-View-Template* (MVT). Las plantillas permiten una renderización dinámica del contenido, integrando datos del contexto proporcionado por las vistas y facilitando una experiencia de usuario coherente y adaptable.

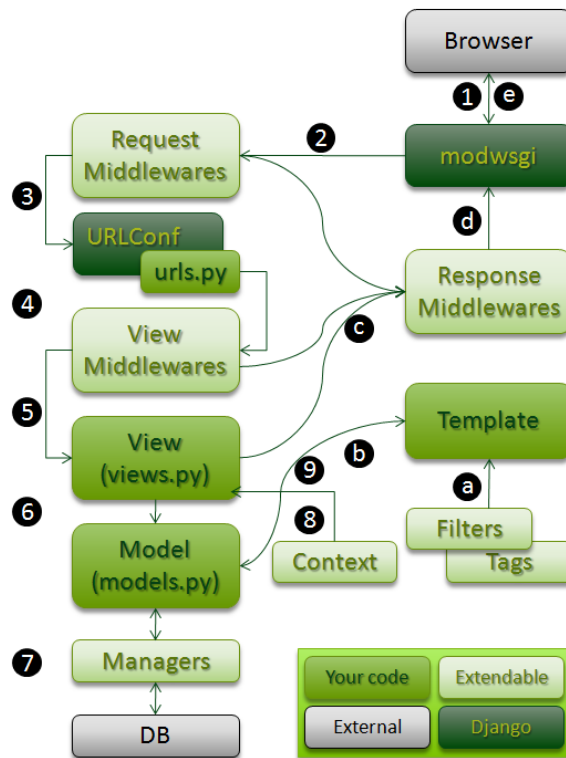


Figura 7.7: Flujo interno de procesamiento de peticiones en Django

La Figura 7.7 ilustra el ciclo completo de una petición en una aplicación Django. Este flujo refleja el recorrido de cualquier solicitud HTTP, desde que el usuario accede a la web hasta que obtiene una respuesta renderizada en el navegador, y se corresponde exactamente con la arquitectura seguida en el desarrollo de esta plataforma.

Breve explicación del flujo interno de procesamiento:

1. El proceso se inicia cuando el usuario realiza una petición desde el **navegador** (por ejemplo, accediendo a `/products/` para ver el catálogo).
2. La petición es recibida por el servidor (`mod_wsgi` en producción o el servidor de desarrollo en local) y pasa a través de los **middlewares de petición**. Estos componentes intermedios pueden modificar o validar la solicitud, por ejemplo, asegurando que el usuario esté autenticado o aplicando políticas de seguridad. En nuestro sistema, `AuthenticationMiddleware` se encarga de asociar la sesión de usuario a la petición.
3. El **enrutador** (`urls.py`) determina qué vista debe gestionarla. Por ejemplo, si la ruta es `/products/`, se invoca la vista correspondiente al listado de productos, implementada como una clase o función en `views.py` de la app `products`.
4. Opcionalmente, la petición puede pasar por **middlewares de vista**, aunque no es el caso en este proyecto.

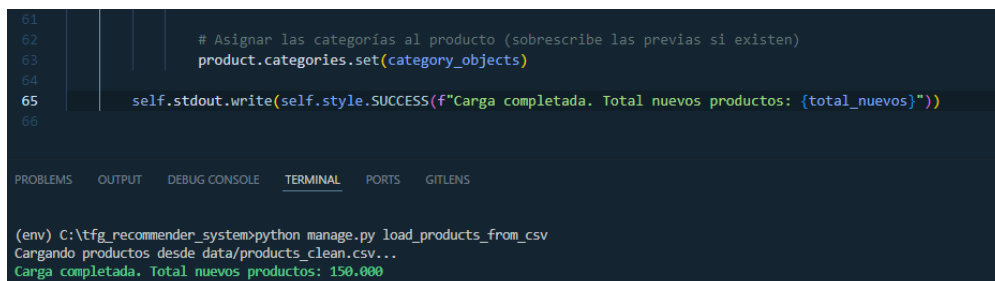
5. La lógica de negocio reside en las **vistas** (views.py). Por ejemplo, la vista ProductListView obtiene todos los productos, aplica filtros según los parámetros GET, y prepara el contexto para la plantilla.
6. Las vistas interactúan con los **modelos** (models.py) a través del ORM. Por ejemplo, al cargar el detalle de un producto, se consulta el modelo Product para obtener su información, o se crea una nueva valoración en Review si un usuario envía el formulario de comentarios.
7. Los **managers** son interfaces que permiten consultar o modificar los datos de la base de datos de forma avanzada, como Product.objects.filter(...). Django se encarga de transformar estas operaciones en consultas SQL a PostgreSQL, proporcionando eficiencia y abstracción.
8. Los datos extraídos de los modelos se organizan en un **contexto**, que se pasa a las plantillas para su visualización.
9. Finalmente, se renderiza la **plantilla** adecuada (product_list.html, recommend_product.html, etc.), utilizando filtros y etiquetas personalizadas si es necesario (por ejemplo, para mostrar la valoración media o truncar descripciones).
10. La respuesta es procesada por los **middlewares de respuesta** y devuelta al usuario, completando el ciclo.

3. Carga y validación de datos en el sistema

3.1. Carga de productos desde CSV y validaciones de entrada

Se desarrolló un comando de administración (load_products_from_csv) que importa productos y categorías desde archivos CSV generados durante la fase ETL. Este proceso valida los tipos de datos, la unicidad de los campos y la integridad de las relaciones antes de insertar los registros en la base de datos.

```
python manage.py runscript load_products_from_csv
```



```
61
62
63     # Asignar las categorías al producto (sobrescribe las previas si existen)
64     product.categories.set(category_objects)
65
66     self.stdout.write(self.style.SUCCESS(f"Carga completada. Total nuevos productos: {total_nuevos}"))
67
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS GITLENS

```
(env) C:\tfg_recommender_system>python manage.py load_products_from_csv
Cargando productos desde data/products_clean.csv...
Carga completada. Total nuevos productos: 150,000
```

Figura 7.8: Captura de la ejecución del comando en el entorno de desarrollo

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × public.products_product/ecommerce_db/postgres@PostgreSQL 17 ×

public.products_product/ecommerce_db/postgres@PostgreSQL

Query Query History Scratch Pad

```
1 SELECT * FROM public.products_product
2 ORDER BY id ASC
```

Data Output Messages Notifications

Showing rows: 1 to 1000 Page No: 1 of 150

	id [PK] bigint	asin character varying (50)	title text	price double precision	average_rating double precision	image_url character varying (200)
1	1	B07SM13SL5	Digi-Tattoo Decal Skin Compatible With MacBook Pro 13 inch (Model A2338/...	19.99	4.5	https://m.media-amazon.com/images/I/61Rpxmi+mPL_AC_SL1500_.jpg
2	2	B089CNGZCW	NotoCity Compatible with Vivoactive 4 band 22mm Quick Release Silicone B...	9.99	4.5	https://m.media-amazon.com/images/I/51ajKkb76L_AC_UL1000_.jpg
3	3	B004E2Z880	Motorola Droid X Essentials Combo Pack	14.99	3.8	https://m.media-amazon.com/images/I/51-DXSMH5aL_AC_.jpg
4	4	B07BJ7ZZL7	QGHXO Band for Garmin Vivofit 4, Soft Silicone Replacement Watch Band St...	14.89	4.4	https://m.media-amazon.com/images/I/61J-V53DqJL_AC_SL1000_.jpg
5	5	B00R6R82HS	Fishfinder, Depth Finder Poly Sun Cover for 3" - 4" Models - Protects Your Scr...	10.99	4.4	https://m.media-amazon.com/images/I/51VaoCIS-dL_AC_.jpg
6	6	B07Y6K1PRB	Novelty Cute Cartoon USB 2.0 Flash Drive Data Storage Memory Stick Carto...	8.98	4.7	https://m.media-amazon.com/images/I/51+NmbjgpgRL_AC_SL1000_.jpg
7	7	B07848ZT9T	ANNKE 4K 16CH Security DVR Recorder with Human & Vehicle Detection an...	909.99	5	https://m.media-amazon.com/images/I/71Qo4iazvel_AC_SL1500_.jpg
8	8	B00VB7JCJE	BIPRA U3 2.5 inch USB 3.0 NTFS Portable External Hard Drive - White (2500B)	98.49	5	https://m.media-amazon.com/images/I/61W5ksZFGAL_AC_SL1500_.jpg
9	9	B0BPLX8B2K	Outer Space Planets Stickers(50Pcs),Planetary Systems Waterproof Vinyl W...	3.99	4.5	https://m.media-amazon.com/images/I/71teibM9rriL_AC_SL1500_.jpg
10	10	B0BYBG1PPD	Gateway 15.6" FHD Ultra Slim Budget Notebook, Intel Pentium Processor(4...	189.99	4.1	https://m.media-amazon.com/images/I/61Y00u4-21L_AC_SL1500_.jpg
11	11	B0C2VCWPGY	VILTROX 24mm F1.8 f/1.8 FE Full-Frame Wide-Angle Prime Autofocus Lens f...	379	3.9	https://m.media-amazon.com/images/I/71nBfBD+PL_AC_SL1500_.jpg
12	12	B093N940PG	Polaroid Originals Now Viewfinder i-Type Instant Camera (White) Bundle wit...	179.99	4.8	https://m.media-amazon.com/images/I/71CPSn8b2gL_AC_SL1500_.jpg
13	13	B07QF5117Q	NVIDIA NVS 310 by PNY 1 GB DDR3 PCI Express Gen 2 x16 DisplayPort 1.2 ...	34.98	4.2	https://m.media-amazon.com/images/I/719zHGIn5sL_AC_SL1500_.jpg
14	14	B094C5ZNI1M	Iuo Cherry MX RGB Switch Mute Pink Switch Mute Gray Switch Genuine Ger...	13.99	4	https://m.media-amazon.com/images/I/81MIUAt1ES_AC_SL1500_.jpg
15	15	B01M0SLDHO	AntennaMastsRus - Made in USA - 4 Inch Black Aluminum Antenna is Comp...	22.99	4.3	https://m.media-amazon.com/images/I/61RSzeOyLbL_AC_SL1500_.jpg
16	16	B0178F72E6	HP Elitebook 8560W New Replacement LCD Screen for Laptop LED HD Matte	63.33	4.7	https://m.media-amazon.com/images/I/51wIAFpNGML_AC_SL1000_.jpg
17	17	B082ZSL7JX	May Chen Compatible with MacBook Pro 16 inch Case 2020 2019 Release A...	26.99	4.5	https://m.media-amazon.com/images/I/71q8TyANDOL_AC_SL1200_.jpg
18	18	B09M7TH3YC	Wenlaty Case Compatible with iPad 9th /8th /7th Generation Case(2021/202...	19.99	4.7	https://m.media-amazon.com/images/I/61FMIGpUNML_AC_SL1500_.jpg
19	19	B07VJXK7BV	Ares Vision Universal mounting CCTV Security Camera Wall Junction Box Br...	13.99	3.9	https://m.media-amazon.com/images/I/314+6f1-J5L_AC_.jpg
20	20	B00FS7W8BK	Centon Electronics Flash Memory Card (SI-SDHU1-32G)	8.99	4.8	https://m.media-amazon.com/images/I/71wtbPhgMvL_AC_SL1500_.jpg
21	21	B08862MBVY	Galaxy Tab A 10.1 Case 2019 - Techcircle Slim PU Leather Multiple Angle St...	17.99	4	https://m.media-amazon.com/images/I/71ZqwoiaqTL_AC_SL1000_.jpg
22	22	B0BSQL7M51	PNY 1TB PRO Elite Class 10 U3 V30 microSDXC Flash Memory Card - 100M...	109.99	4.7	https://m.media-amazon.com/images/I/61ajvgWBzL_AC_SL1000_.jpg
23	23	B0BKSTK986	LENTION USB C Docking Station, 10 Gbps USB C&USB A Ports, 4K 60Hz HD...	89.99	4.4	https://m.media-amazon.com/images/I/51jWbhpjRL_AC_SL1500_.jpg

Figura 7.9: Captura de pgAdmin4 donde se aprecian los productos cargados en la base de datos

3.2. Procesado y carga de reseñas, generación automática de usuarios

La importación de reseñas se realiza mediante el script `load_amazon_reviews.py`, que automatiza la creación de usuarios y su asociación con productos existentes. Si no existe un usuario correspondiente, se crea un usuario inactivo con una cuenta técnica vinculada al identificador original del dataset. Cada reseña se vincula correctamente a un producto y un usuario, garantizando la integridad referencial.

3.3. Optimización para pruebas: limpieza y reducción de datos

Con el objetivo de facilitar la ejecución de pruebas y demostraciones en entornos con recursos limitados, se implementó el comando `clean_for_demo.py`, que reduce la base de datos a un subconjunto manejable (por ejemplo, 5000 productos con suficientes reseñas y buena calidad de datos), eliminando entradas redundantes y categorías huérfanas.

3.4. Gestión y validación de reseñas

La importación de valoraciones se realiza en bloque desde archivos CSV, aplicando diversas validaciones que garantizan la coherencia de los datos:

- Validación del rango permitido de calificaciones (1 a 5).

- Verificación de la existencia previa del usuario y del producto.
- Aplicación de la restricción `unique_together` para evitar reseñas duplicadas.
- Creación de usuarios técnicos en caso de que no existan previamente.

Las reseñas duplicadas o con referencias inconsistentes son descartadas automáticamente, asegurando un dataset limpio y fiable.

4. Exposición de la API RESTful

4.1. Uso de Django REST Framework

Se eligió *Django REST Framework* (DRF) por su robustez, capacidad para manejar serialización compleja y soporte nativo para autenticación y permisos. Esto facilita la exposición de recursos a clientes externos y la integración con sistemas de terceros.

4.2. Endpoints principales

- `/api/products/`: listado y detalle de productos.
- `/api/users/`: gestión y consulta de usuarios.
- `/api/reviews/`: creación, actualización y consulta de valoraciones (CBV).
- `/api/recommend/<user_id>/`: obtención de recomendaciones para un usuario específico (FBV).
- `/api/dashboard/`: visualización y filtrado de evaluaciones de algoritmos.

Figura sugerida: Captura del endpoint de recomendaciones en acción y/o fragmento de respuesta JSON.

4.3. Control de acceso, autenticación y permisos

Se implementaron sistemas de autenticación basada en sesión y token. Los permisos restringen las operaciones de escritura y edición de reseñas únicamente a usuarios autenticados, previniendo manipulaciones externas. En endpoints sensibles como el de recomendaciones personalizadas, se exige autenticación, y el frontend notifica al usuario si no está logueado.

Ejemplo de endpoint

```

1 GET /api/recommend/42/
2
3 Response:
4 {
5   "user_id": 42,
```

```
6 "recommended_products": [10021, 10098, 10201]
7 }
```

Extracto de código 7.9: Ejemplo de respuesta del endpoint de recomendaciones

5. Pruebas y validación del back-end

Se llevaron a cabo diversas pruebas para asegurar el correcto funcionamiento de todo el sistema:

- Pruebas unitarias sobre importación y validación de datos.
- Pruebas funcionales del pipeline de carga masiva (productos y reseñas).
- Testeo de los endpoints REST mediante herramientas como Postman.
- Verificación de la restricción de unicidad y correcto funcionamiento de serialización/deserialización.
- Pruebas de rendimiento en la fase ETL y durante la carga concurrente.

Figura sugerida: Captura de un test funcional de la API, mostrando el flujo de obtención de recomendaciones para un usuario de prueba.

6. Resumen y retos encontrados

- **Carga masiva y limpieza de datos:** fue necesario optimizar procesos y aplicar muestreo para evitar cuellos de botella.
- **Gestión de integridad referencial:** se automatizó la creación de usuarios técnicos y se aplicaron restricciones de unicidad.
- **Validación y robustez de la API:** se incorporaron validaciones exhaustivas, mensajes de error claros y controles de acceso estrictos.
- **Entrenamiento local de modelos:** las limitaciones de recursos exigieron trabajar con subconjuntos de datos y modelos preentrenados.

El resultado final es un **back-end robusto y validado**, preparado tanto para escenarios de prueba como para futuras integraciones con algoritmos de recomendación avanzados y componentes del sistema frontend.

8. Cierre

8.1. Introducción

En este capítulo explicaremos...

8.2. Conclusiones

En este capítulo concluimos que...

.1. Anexo A: Elección del dataset

Este anexo recoge el análisis comparativo realizado para seleccionar el conjunto de datos más adecuado para el desarrollo del sistema recomendador. Se han evaluado tres recursos principales: *AmazonReviews2023*, el dataset de *Retailrocket*, y un estudio académico publicado en *arXiv* (referencia 2307.09688), con el objetivo de identificar la opción más completa, representativa y alineada con los objetivos del proyecto.

1. AmazonReviews2023

El repositorio *AmazonReviews2023* ofrece un amplio conjunto de herramientas y datos esenciales para investigaciones avanzadas en sistemas de recomendación y recuperación de información. Este conjunto de datos abarca reseñas de productos de Amazon desde 1996 hasta 2023, proporcionando una cobertura temporal y temática extensa.

- **Datos de reseñas y productos:** más de 570 millones de reseñas y 48 millones de productos distribuidos en 33 categorías distintas.
- **Procesamiento de datos:** scripts diseñados para transformar los datos en conjuntos de entrenamiento, validación y prueba, facilitando así la evaluación de modelos (disponible en GitHub).
- **Modelos preentrenados BLAIR:** modelos de lenguaje enfocados en capturar la relación entre descripciones de productos y su contexto lingüístico, lo que mejora la calidad de las recomendaciones (disponibles en GitHub).
- **Amazon-C4:** conjunto de datos adicional creado para evaluar la comprensión del contexto y la relevancia en la recuperación de productos (disponible en GitHub).

Valoración: AmazonReviews2023 se presenta como una opción idónea para el desarrollo de algoritmos de recomendación avanzados. Los modelos BLAIR y Amazon-C4 proporcionan oportunidades significativas para mejorar el entendimiento semántico y la recuperación de productos, lo que incrementa la precisión de las recomendaciones ofrecidas.

2. Retailrocket dataset

El conjunto de datos de *Retailrocket* proporciona información detallada sobre la actividad de los usuarios en una plataforma de comercio electrónico. Los datos están organizados de la siguiente manera:

- **Eventos web:** registro de interacciones de los usuarios, tales como visualizaciones de productos, adiciones al carrito y compras.
- **Propiedades de los artículos:** información detallada de los productos, incluyendo descripciones y características específicas.

- **Árbol de categorías:** estructura jerárquica que organiza los productos en diversas categorías para facilitar la navegación y búsqueda.

Valoración: Este conjunto de datos es particularmente útil para la construcción y evaluación de sistemas de recomendación en escenarios que involucren la navegación web y el comportamiento del usuario.

Justificación de la elección

Tras analizar en detalle las características y el enfoque de cada recurso, se ha optado por trabajar con **AmazonReviews2023** como fuente principal de datos. La decisión se basa en varios factores clave:

- **Orientación a producto y metadatos enriquecidos:** AmazonReviews2023 proporciona no solo valoraciones de usuarios, sino también una gran riqueza de metadatos asociados a los productos (descripciones, marcas, categorías, imágenes, fechas, etc.). Esto permite poner en práctica técnicas avanzadas de análisis y algoritmia, tanto para recomendación colaborativa como para enfoques basados en contenido y análisis de tendencias.
- **Facilidad de integración y flexibilidad experimental:** La estructura del dataset facilita la integración con el sistema, permitiendo adaptar el volumen y la granularidad de los datos a los recursos disponibles.
- **Alineamiento con los objetivos iniciales** El trabajo persigue desarrollar y evaluar distintos algoritmos de recomendación, analizar su rendimiento y explorar el valor que ofrecen en la mejora de la personalización, aumento de ventas y la experiencia de usuario. La disponibilidad de información tan detallada sobre los productos posibilita estos objetivos y aporta un mayor valor experimental ya que el segundo dataset se centra más en interacciones de los usuarios, la cuál es más difícil de procesar y recrear en un entorno con recursos limitados.
- **Reproducibilidad y relevancia académica:** La accesibilidad, documentación y reconocimiento académico de AmazonReviews2023, así como su mantenimiento y actualización regular, aportan garantías adicionales de replicabilidad y utilidad futura del sistema desarrollado.

Veredicto: Nos decantamos por elegir AmazonReviews2023 como dataset en el que se ha basado la plataforma ya que responde a criterios tanto de riqueza y variedad de datos, como de orientación técnica y experimental, asegurando que el sistema desarrollado pueda beneficiarse de un contexto realista y aprovechar el máximo potencial de análisis, modelado y evaluación de algoritmos recomendadores, y una alineación total con la motivación inicial del trabajo.

Bibliografía
